Unidade II

3 DETALHAMENTO DA LINGUAGEM C#

A linguagem de programação C# consolidou-se no cenário global como uma ferramenta poderosa e versátil, atraindo tanto novatos quanto profissionais experientes do mundo da programação. No coração dessa linguagem está sua sintaxe clara e lógica, projetada para tornar o código facilmente legível e compreensível. Cada instrução em C# culmina com um ponto-e-vírgula, e os blocos de código são coesamente delineados com o uso de chaves. No ecossistema do C#, temos os namespaces, que funcionam como recipientes, agrupando classes e outros tipos sob um nome comum. Eles oferecem uma maneira eficiente de organizar o código e, ao mesmo tempo, evitar potenciais conflitos de nomenclatura, garantindo que bibliotecas e programas possam coexistir de forma harmoniosa.

Quando nos debruçamos sobre o tratamento de dados em C#, percebemos uma rica tapeçaria de tipos numéricos à disposição. Desde a representação de números inteiros até números de ponto flutuante e decimais precisos, o C# garante flexibilidade e precisão para diferentes necessidades de cálculo. Em contraste, o tipo booleano – que representa valores verdadeiros ou falsos – serve como a pedra angular para a lógica e tomada de decisão na linguagem, permitindo avaliar condições e dirigir o fluxo de execução.

Já no domínio do texto, o C# oferece as cadeias de caracteres, mais conhecidas como **strings** – sequências de caracteres usadas primordialmente para representar texto. Uma característica distintiva em C# é a imutabilidade das strings, ou seja, uma vez que uma string é criada, seu valor original permanece inalterado.

Abordar a questão da armazenagem de informações também implica abordar **variáveis**, identificadores que apontam para locais de armazenamento na memória. No C#, antes de serem utilizadas, as variáveis precisam ser declaradas com um tipo específico. Essa etapa garante ao compilador uma compreensão clara de como interpretar os dados que elas detêm.

O poder real de qualquer linguagem de programação reside em sua capacidade de realizar operações e tomar decisões. C# é repleto de operadores para tarefas matemáticas, comparações e operações lógicas, e quando esses operadores são habilmente combinados em expressões, desbloqueiam a capacidade dos programas de fazer cálculos complexos e tomar decisões. Na esteira dessas decisões, encontramos declarações que especificam ações a serem executadas, desde simples declarações de variáveis até estruturas condicionais mais complexas.

A capacidade de armazenar coleções ordenadas de itens é possibilitada por vetores ou arrays. Em contrapartida, para expressar conjuntos fixos de valores, o C# fornece enumerações. Além destes, a linguagem introduz estruturas, que são tipos compostos capazes de conter diversas variáveis sob um

único nome. No pináculo dessa hierarquia de tipos está o tipo objeto, o ancestral comum de todos os outros tipos em C#.

Finalmente, para interagir com o mundo exterior e o usuário, o C# incorpora conceitos de entrada e saída, variando desde a simples leitura de teclado até a intrincada manipulação de arquivos. Em resumo, C# apresenta um ambiente de programação profundamente robusto e versátil. Dominar seus fundamentos desvenda o imenso potencial que essa linguagem oferece.

3.1 Sintaxe

Todo programa em C# tem uma estrutura básica, mesmo que seja o mais simples "Olá, Mundo!". A linguagem compartilha semelhanças sintáticas com linguagens como Java e C++, mas também apresenta características únicas, especialmente relacionadas à sua integração com a plataforma .NET. Um programa típico é composto de declarações de namespace, classes e métodos.

Na figura 66, Olimpo é um namespace usado para organizar e fornecer um nível de separação de códigos. A classe Deus abriga o método Main, que é o ponto de entrada do programa. Essa organização demonstra como C# valoriza a orientação a objetos.

```
1.  namespace Olimpo
2.  {
3.      class Deus
4.      {
5.          static void Main(string[] args)
6.          {
7.              Console.WriteLine("Olá, Zeus!");
8.          }
9.      }
10. }
```

Figura 66 – Exemplo de declaração de um programa simples

Muitos argumentam que C# é mais fácil de aprender para novatos do que algumas outras linguagens, graças à sua sintaxe clara e ao rico ambiente de desenvolvimento do Visual Studio. Quanto à produtividade, estudos como o de Prechelt (2000) indicam que linguagens como C# podem resultar em desenvolvimento mais rápido e menos erros quando comparado a linguagens de baixo nível.

Embora não haja um limite específico para o número de caracteres em uma única linha, é prática comum manter linhas de código relativamente curtas para melhor legibilidade. Além disso, o C# não impõe tamanho máximo para um programa, mas a memória e os recursos do sistema podem impor limites práticos. Outro fator relevante é que a linguagem é fortemente tipada. Ou seja, uma vez que uma variável é declarada para um tipo particular, seu tipo não pode ser alterado implicitamente.

Na figura 67, a variável hercules, que alude às 12 tarefas de Hércules, é do tipo int. Ela não pode, sem coerção explícita, ser retribuída a um valor de outro tipo, como uma string ou um booleano.

```
1.   int hercules = 12; // Declarando uma variável do tipo inteiro
```

Figura 67 – Exemplo de declaração tipada

Parênteses **()** são frequentemente usados para envolver listas de parâmetros em definições de método e chamadas. As chaves **{}** delimitam blocos de código, como classes, métodos ou estruturas condicionais (figura 68).

```
1.   void InvocarDeus(string nome)
2.   {
3.      if (nome == "Ares")
4.      {
5.         Console.WriteLine("O Deus da Guerra foi invocado!");
6.      }
7.   }
```

Figura 68 – Exemplos do uso de parênteses e chaves

Caracteres especiais, como ponto-e-vírgula (;), são usados para denotar o fim de uma instrução. O **#**, usado em diretivas como #using, faz com que instruções sejam processadas antes da compilação do código, utilizado para incluir namespaces ou definir condicionais. O quadro 4 apresenta uma lista não exaustiva com caracteres especiais, e há também um conjunto robusto de palavras-chave, como class, int, void e if. Essas palavras não podem ser usadas como nomes de variáveis ou métodos. Já o quadro 5 tem uma lista não exaustiva de palavras-chave.

Quadro 4 – Exemplos de caracteres especiais da linguagem C#

Caractere	Descrição
{	Início de um bloco de código
}	Fim de um bloco de código
;	Fim de uma declaração
:	Usado em herança e instruções switch
.	Acesso a membros de um objeto
,	Separação em listas e parâmetros
+	Operador de adição
-	Operador de subtração
*	Operador de multiplicação
/	Operador de divisão
%	Operador de módulo
&	Operador AND lógico/bit a bit
^	Operador XOR bit a bit
!	Operador NOT lógico
~	Operador complemento bit a bit
=	Operador de atribuição
==	Operador de igualdade
!=	Operador de desigualdade

Caractere	Descrição
<	Menor que
>	Maior que
<=	Menor ou igual a
>=	Maior ou igual a
++	Incremento
--	Decremento
&&	Operador AND lógico condicional
||	Operador OR lógico condicional
?	Operador condicional ternário
:	Usado no operador condicional ternário
[]	Define arrays
()	Contém parâmetros ou altera precedência
@	Indica string verbatim ou alias
$	Indica string interpolada
#	Usado em diretivas de pré-processamento
_	Indica um identificador ou descartável
\\	Caractere de escape para barra invertida
\"	Caractere de escape para aspas
\n	Quebra de linha
\t	Tabulação
\r	Retorno de carro
\0	Caractere nulo
=>	Indica uma expressão lambda

Quadro 5 – Exemplos de palavras-chave do C#

Palavra-chave	Descrição
abstract	Indica que uma classe ou método é abstrato
as	Realiza conversões de tipo seguras
base	Acessa membros da classe-base
bool	Tipo de dado booleano
break	Termina o loop ou switch
case	Usado em instruções switch
catch	Captura exceções em um bloco try
char	Tipo de dado caractere
checked	Habilita a verificação de overflow
class	Define uma classe
const	Declara uma constante
continue	Passa para a próxima iteração do loop
decimal	Tipo numérico com ponto decimal
default	Valor padrão em um switch ou tipo padrão em um generic
delegate	Define um tipo de delegado
do	Define a instrução de loop do-while

Palavra-chave	Descrição
double	Tipo numérico de ponto flutuante de dupla precisão
else	Bloco condicional if-else
enum	Declara uma enumeração
event	Declara um evento
explicit	Define uma conversão de tipo explícita
extern	Declara um método externo
false	Valor booleano falso
finally	Bloco que é executado após try-catch
fixed	Impede que o coletor de lixo mova um objeto
float	Tipo numérico de ponto flutuante de precisão simples
for	Define a instrução de loop for
foreach	Define a instrução de loop foreach
if	Define uma instrução condicional
implicit	Define uma conversão de tipo implícita
in	Palavra-chave para parâmetros de entrada
int	Tipo de dado inteiro
interface	Declara uma interface
internal	Modificador de acesso para um membro da assembly
is	Verifica o tipo de objeto
lock	Marca um bloco de declarações como crítico
long	Tipo numérico inteiro longo
namespace	Declara um namespace
new	Cria instâncias ou oculta membros herdados
null	Valor nulo
object	Tipo de base para todos os tipos
operator	Sobrecarrega um operador
out	Palavra-chave para parâmetros de saída
override	Indica que um método substitui um método-base
params	Indica um parâmetro que aceita um número variável de argumentos
private	Modificador de acesso privado
protected	Modificador de acesso protegido
public	Modificador de acesso público

Comentários são denotados por **//** para uma única linha, e **/* */** para múltiplas linhas. São essenciais para explicar a lógica do código, conforme a figura 69.

```
1.  // Isso é um comentário de uma linha
2.  /*
3.  Este é um comentário
4.  de múltiplas linhas
5.  */
```

Figura 69 – Exemplo do uso de comentários no código

Observação

Relativo à língua portuguesa, outro aspecto a considerar sobre a sintaxe do C# é que, em geral, ela permite usar caracteres com cedilha e acentos em nomes de identificadores. Assim, você poderia, teoricamente, ter nomes de variáveis como "número" ou "açúcar", mas isso não é comum em programação, principalmente porque pode causar problemas de codificação ou ser mal interpretado em contextos internacionais. A maioria das convenções de codificação sugere usar nomes em inglês sem acento. Espaços não são permitidos em identificadores em C#. Se você tentar nomear uma variável ou classe com um espaço, causará um erro de compilação. Por exemplo, uma variável nomeada int minha variável = 10; resultará em erro.

Outras convenções de nomenclatura não são exclusivas do C#, ou seja, são utilizadas em outras linguagens, mas também foram adotadas por um grande número de programadores C#. Essas convenções não são componentes integrantes da linguagem, mas seu uso é amplamente disseminado. O quadro 6 apresenta algumas nomenclaturas:

Quadro 6 – Convenções de nomenclatura comumente usadas em C#

Nome	Convenção	Exemplo
Classes e estruturas	PascalCase	PublicClass, MyStruct
Métodos	PascalCase (públicos e privados)	CalculateTotal(), PrintValues()
Propriedades	PascalCase	TotalAmount, LastItem
Eventos	PascalCase	ButtonClick, ValueChanged
Namespaces	PascalCase	System.Drawing, Microsoft.EntityFrameworkCore
Variáveis e parâmetros	camelCase	totalAmount, listItems
Membros de dados privados	camelCase com prefixo '_'	_privateValue, _initialCount
Interfaces	'I' seguido de PascalCase	IEnumerable, IDisposable
Constantes	PascalCase	MaxValue, DefaultSize
Campos readonly	'_' seguido de camelCase	_readonlyField

O camelCase refere-se à prática de escrever frases compostas ou expressões combinando várias palavras, onde cada palavra começa com uma letra maiúscula, exceto a primeira, que é minúscula. Por exemplo: **t**his**I**s**C**amel**C**ase. Há também o PascalCase, semelhante ao CamelCase, mas começa com letra maiúscula. Por exemplo: **T**his**I**s**P**ascal**C**ase. Apesar de não ser uma regra rígida da sintaxe de muitas linguagens, seguir convenções de nomenclatura é uma boa prática que facilita a leitura e manutenção do código. Essas convenções podem variar entre linguagens, empresas e projetos.

3.2 Namespaces

Ao trabalhar com linguagens de programação modernas, é essencial organizar o código de forma que não apenas melhore sua manutenibilidade, mas também previna possíveis conflitos de nomes e ajude na modularização. No C#, namespaces desempenham esse papel crucial. Neste tópico vamos explorar a natureza e a importância dos namespaces no C#, sua sintaxe, uso prático e como interagem com a organização global do código.

Em termos simples, namespace é um contêiner para um conjunto de identificadores que fornece uma maneira de agrupar funções, classes, estruturas e outros tipos sob um nome, evitando conflitos entre nomes em códigos maiores e mais complexos. Namespaces são usados principalmente para organizar o código e proporcionar uma forma de evitar colisões de nomes entre diferentes bibliotecas ou partes do software.

Em projetos de software maiores, é comum haver muitas classes, estruturas e interfaces. Sem uma forma adequada de organização, o código pode se tornar difícil de gerenciar e entender, e os namespaces oferecem uma estrutura lógica, permitindo que desenvolvedores agrupem componentes relacionados. Além disso, suponha que você esteja usando duas bibliotecas de terceiros em seu projeto; se ambas tiverem uma classe com o mesmo nome, surgirá um conflito. Os namespaces resolvem esse problema pois, mesmo que as classes tenham o mesmo nome, seus namespaces provavelmente serão diferentes. Outro aspecto importante é que, ao usar bibliotecas externas, eles ajudam a identificar de qual biblioteca determinado tipo é originário, tornando o código mais legível e menos propenso a erros.

```
1.  namespace Olimpo.Mar
2.  {
3.    public class Poseidon
4.    {
5.      // Implementação da classe Poseidon
6.    }
7.  }
8.  namespace Olimpo.Ceu
9.  {
10.   public class Zeus
11.   {
12.     // Implementação da classe Zeus
13.   }
14. }
15. namespace Olimpo.Underworld
16. {
17.   public class Hades
18.   {
19.     // Implementação da classe Hades
20.   }
21. }
```

Figura 70 – Organizando o Olimpo com namespaces

Imagine a vastidão do Olimpo e as muitas divindades e criaturas que habitam suas histórias. Caso todas fossem chamadas apenas pelo primeiro nome, sem qualquer categorização, um nome poderia se confundir com outro. Da mesma forma, em programação, quando lidamos com uma grande quantidade de classes e tipos, precisamos de uma forma de organizar e evitar conflitos de nomes.

Para organizar os deuses do Olimpo, podemos ter namespaces baseados em seus domínios, conforme a figura 70. Para invocar a classe Zeus em outro local, basta incluir a palavra-chave using (linha 1 da figura 71) seguida pelo namespace Olimpo.Ceu, domínio de Zeus.

```
1.  using Olimpo.Ceu;
2.
3.  // ...
4.
5.  Zeus reiDosDeuses = new Zeus();
```

Figura 71 – Invocando Zeus a partir do domínio Olimpo.Ceu

Também temos diversas criaturas na mitologia grega que podem ser categorizadas em seus respectivos namespaces. A figura 72 ilustra esse caso.

```
1.  namespace Criaturas.Mar
2.  {
3.     public class Kraken
4.     {
5.          // Implementação da classe Kraken
6.     }
7.  }
8.
9.  namespace Criaturas.Bosque
10. {
11.    public class Centauro
12.    {
13.         // Implementação da classe Centauro
14.    }
15. }
```

Figura 72 – Criaturas do mar e do bosque organizadas com namespaces

A figura 73 mostra como proceder se quisermos trabalhar com Kraken.

```
1.  using Criaturas.Mar;
2.
3.  // ...
4.
5.  Kraken monstroMarinho = new Kraken();
```

Figura 73 – Invocando Kraken a partir do namespace Criaturas.Mar

É importante diferenciar namespaces de assemblies no C#. Assembly é uma unidade de implantação, como um arquivo DLL ou EXE. Embora tanto os namespaces quanto os assemblies sejam usados para organizar código, servem a propósitos diferentes. O primeiro organiza o código logicamente, enquanto o segundo faz isso fisicamente. Um único assembly pode conter tipos de vários namespace.

3.3 Tipos numéricos

Uma das características fundamentais da linguagem C# é a variedade de tipos de dados que suporta, especialmente quando se trata de números. Compreender esses tipos é essencial para qualquer desenvolvedor que deseja manipular operações matemáticas, dados estatísticos, engenharia ou até mesmo jogos. Tipos numéricos podem ser categorizados em dois grupos: de **valor integral** e de **ponto flutuante**.

Observação

Antes de abordar esses dois grupos, é importante entender a diferença entre tipos numéricos de valor e tipos numéricos de referência. Os primeiros contêm o valor real na memória e não armazenam referências ou ponteiros para outros lugares nela; em vez disso, detêm o valor em si. Já os de referência são armazenados na memória como referência a locais onde os dados reais estão armazenados.

Tipos integrais representam números inteiros, ou seja, sem uma parte decimal. Em C#, temos vários tipos integrais, dependendo do tamanho do número e se ele é ou não assinado (capaz de representar valores negativos). **Tipos de ponto flutuante** representam números que têm uma parte decimal. O quadro 7 apresenta os tipos números em C#.

Quadro 7 – Tipos numéricos integrais e ponto flutuante

Tipo integral	Descrição	Tipo flutuante	Descrição
byte	8 bits, 0-255	float	32 bits, precisão simples
sbyte	8 bits, -128-127	double	64 bits, precisão dupla
int	32 bits, signed	decimal	Alta precisão, fixo
uint	32 bits, unsigned		
short	16 bits, signed		
ushort	16 bits, unsigned		
long	64 bits, signed		
ulong	64 bits, unsigned		

Lembre-se que diferentes cenários exigem diferentes tipos numéricos. Optar pelo tipo correto não apenas garante que os dados sejam representados corretamente, mas também otimiza o uso da memória (a figura 74 ilustra esse conceito). Além disso, foi usada a letra F após o número 88.7 (linha 6), e a letra M após o número 1000000.75 (linha 10). Esses sufixos, em C#, são essenciais para diferenciar e especificar

os tipos de dados numéricos literais, garantindo que sejam interpretados conforme o desejado; assim como ajudam os programadores a evitar erros de compilação e comportamentos inesperados.

```
1.  // Representando a idade de deuses gregos com tipos integrais.
2.  int idadeZeus = 3000; // Zeus viveu por um longo tempo, assim, o tipo
int é uma boa escolha.
3.  byte idadeEros = 20); // Eros é frequentemente representado como um
jovem, por isso um byte é suficiente.
4.  // Comparando a força de dois deuses usando ponto flutuante.
5.  double forcaHercules = 99.5; // Hercules é quase perfeito em força.
6.  float forcaAthena = 88.7F; // Athena, sendo mais estratégica, tem uma
força menor.
7.  // Calculando o total de ouro em Dracmas no Monte Olimpo usando
decimal para alta precisão.
8.  decimal ouroOlimpo = 1000000.75M;
```

Figura 74 – Usando tipos numéricos adequados para otimizar o uso da memória

A figura 75 apresenta diversos exemplos comentados de aplicação dos sufixos, e o quadro 8 aborda em detalhes os sufixos mais comuns.

```
1.  class SufixosNumericos
2.  {
3.    public static void Main()
4.    {
5.      // Usando sufixo F para float
6.      float velocidadeHermes = 5.7F; // Este é um float devido ao sufixo
    'F'
7.      Console.WriteLine($"Velocidade de Hermes: {velocidadeHermes}");
8.      // Usando sufixo M para decimal
9.  decimal precoAmbrosia = 12.99M; // Este é um decimal devido ao sufixo
    'M'
10.     Console.WriteLine($"Preço da Ambrosia: {preçoAmbrosia}");
11.     // Usando sufixo D para double (embora não seja estritamente
    necessário)
12.     double distanciaOlimpoTerra = 9876.543D; // Explicitamente double
    devido ao sufixo 'D'
13.     Console.WriteLine($"Distância do Olimpo à Terra:
{distanciaOlimpoTerra}");
14.     // Usando sufixos U para unsigned e L para long
15.     uint populacaoAtenas = 120000U; // Unsigned integer devido ao
    sufixo 'U'
16.     long anosTitanomachy = 5000000000L; // Long integer devido ao
sufixo 'L'
17.     Console.WriteLine($"População de Atenas: {populaçãoAtenas}");
18.     Console.WriteLine($"Anos da Titanomachy: {anosTitanomachy}");
19.   }
20. }
```

Figura 75 – Aplicação dos sufixos numéricos

Quadro 8 – Sufixos comumente usados

Sufixo	Descrição
F/f	Denota números de ponto flutuante do tipo float. Por padrão, valores numéricos com ponto decimal em C# são considerados double. O sufixo F explicita o tipo como float
M/m	Usado para números do tipo decimal, em situações que requerem alta precisão, como cálculos financeiros
D/d	Embora double seja o padrão para números com ponto decimal, o sufixo D explicita que estamos lidando com um valor double
U/u e L/l	Para números inteiros, U denota valores unsigned e L denota valores long; ambos especificam valores além dos limites-padrão de int

Além dos tipos numéricos primitivos em C#, o .NET Framework fornece classes numéricas que oferecem uma ampla gama de funcionalidades. Essas classes são particularmente úteis quando se precisa de operações matemáticas mais avançadas ou quando se trabalha com números de precisão arbitrariamente alta. São elas:

3.3.1 System.Math

A classe System.Math fornece métodos estáticos para operações matemáticas básicas, bem como algumas constantes matemáticas úteis.

- Propriedades:

  – **E**: retorna a base dos logaritmos naturais (aproximadamente 2,718).

  – **PI**: retorna o valor de pi (aproximadamente 3,14159).

- Métodos:

  – **Abs()**: retorna o valor absoluto de um número.

  – **Ceiling()**: retorna o menor número inteiro que seja maior ou igual ao número especificado.

  – **Floor()**: retorna o maior número inteiro menor ou igual ao número especificado.

  – **Max()**: retorna o maior de dois números.

  – **Min()**: retorna o menor de dois números.

  – **Pow()**: retorna um número elevado à potência especificada.

  – **Round()**: arredonda um valor para o inteiro mais próximo.

  – **Sqrt()**: retorna a raiz quadrada de um número.

3.3.2 System.Numerics.BigInteger

A estrutura BigInteger representa um número inteiro de precisão arbitrária. Isso significa que ele pode ser usado para armazenar e operar em números inteiros muito grandes para ser representados pelos tipos numéricos padrão.

- Métodos:
  - **Abs()**: retorna o valor absoluto.
  - **Add()**: adiciona dois valores BigInteger.
  - **Divide()**: divide um valor BigInteger por outro.
  - **Multiply()**: multiplica dois valores BigInteger.
  - **Subtract()**: subtrai um valor BigInteger de outro.
  - E muitos outros métodos para operações aritméticas, de comparação, bit a bit etc.

3.3.3 System.Decimal

Embora decimal seja um tipo primitivo em C#, também tem métodos associados que podem ser usados para operações com precisão.

- Métodos:
  - **Add()**: adiciona dois valores decimais.
  - **Ceiling()**: retorna o menor número inteiro que é maior ou igual ao número decimal especificado.
  - **Divide()**: divide um decimal por outro.
  - **Floor()**: retorna o maior número inteiro menor ou igual ao número decimal especificado.
  - **Multiply()**: multiplica dois valores decimais.
  - **Round()**: arredonda um valor decimal para o número inteiro mais próximo.
  - **Subtract()**: subtrai um decimal de outro.

A figura 76 apresenta diversos usos dessas ferramentas, muito úteis na manipulação numérica.

```
1.  double athenasWisdom = 2.7;
2.  double zeusPower = 3.9;
3.
4.  double greaterValue = Math.Max(athenasWisdom, zeusPower);// Aqui, Zeus
    prevalece!
5.
6.  System.Numerics.BigInteger trojanWarriors = new
    System.Numerics.BigInteger(1000000);
7.  System.Numerics.BigInteger greekGods = new
    System.Numerics.BigInteger(10);
8.  System.Numerics.BigInteger totalCombatants =
    System.Numerics.BigInteger.Add(trojanWarriors, greekGods);
9.
10. decimal goldenApples = 3.5M;
11. decimal silverApples = 2.8M;
12. decimal totalApples = decimal.Add(goldenApples, silverApples);
```

Figura 76 – Trabalhando com ferramentas numéricas

3.4 Tipos booleanos

O tipo booleano, representado pela palavra-chave **bool**, é fundamental para a programação e está enraizado em uma lógica binária que remonta a antigos filósofos, como o legendário Aristóteles, da Grécia Antiga. Com sua lógica de bivalência, ele propunha que qualquer proposição tem exatamente dois valores possíveis, verdadeiro ou falso. É essa simplicidade fundamental que torna o tipo booleano tão poderoso e versátil. Em C#, o tipo bool pode ter um de dois valores: true ou false. E, semelhante a como os deuses gregos muitas vezes tomavam decisões que moldavam o destino dos mortais, os valores booleanos em um programa determinam o fluxo de execução, especialmente quando combinados com estruturas condicionais.

```
1.  bool zeusEstaBravo = false;
2.  if (zeusEstaBravo)
3.  {
4.     Console.WriteLine("Melhor se abrigar, pois tempestades estão a
caminho!");
5.  }
6.  else
7.  {
8.     Console.WriteLine("Os céus estão calmos. É seguro prosseguir.");
9.  }
```

Figura 77 – zeusEstaBravo: exemplo de variável booleana

No código da figura, se Zeus estiver irritado (zeusEstaBravo for verdadeiro), um aviso é emitido; caso contrário, uma mensagem reconfortante é mostrada. Esse é o princípio fundamental de qualquer estrutura condicional: uma decisão é tomada com base em uma condição booleana. Além de sua representação direta, os valores booleanos em C# podem resultar de operações relacionais. Operadores como ==, !=, <, >, <=, e >= comparam valores e retornam um resultado booleano.

```
1.  int troianos = 1000;
2.  int gregos = 900;
3.
4.  bool gregosSaoNumerosos = gregos > troianos; // Isso será avaliado
como false
```

Figura 78 – Comparação de valores com resultado booleano

No contexto da figura, mesmo que os gregos sejam famosos por sua astúcia e habilidade em batalha, numericamente são menores que os troianos.

Juntamente aos operadores relacionais, C# oferece operadores lógicos para trabalhar com valores booleanos: **&&** (E), **||** (OU) e **!** (NÃO). Eles permitem combinar ou inverter valores booleanos.

```
1.  bool aresApoiaGregos = true;
2.  bool athenaApoiaGregos = false;
3.
4.  bool gregosTemApoioDivino = aresApoiaGregos || athenaApoiaGregos; //
Isso será avaliado como true
```

Figura 79 – gregosTemApoioDivino: operadores lógicos com valores booleanos

No trecho, mesmo que apenas um dos deuses (Ares) apoie os gregos, a expressão é avaliada como verdadeira, pois usamos o operador OU.

Observação

Enquanto os tipos booleanos podem parecer simples à primeira vista, sua importância não deve ser subestimada. Assim como na mitologia grega, onde as escolhas e alianças dos deuses determinavam o curso dos eventos humanos, os valores booleanos e suas operações determinam o fluxo de execução de um programa. É crucial entender que o tipo bool em C# não pode ser implicitamente convertido para outros tipos, como inteiros ou strings. Em algumas linguagens, é comum tratar 0 como falso e qualquer outro valor como verdadeiro, mas em C# isso é evitado para manter a clareza e evitar erros. Assim, tentativas de converter diretamente entre bool e int resultarão em erro de compilação.

Na linha 2 da figura 80, usamos o método Convert.ToInt32 para converter o valor booleano para um inteiro, resultando em 1 para true e 0 para false.

```
1.  bool hermesIsFast = true;
2.  int speedValue = Convert.ToInt32(hermesIsFast);  // Isso é válido
```

Figura 80 – Conversão de tipo booleano para inteiro

Finalmente, é bom lembrar que o tipo booleano é um tipo de valor em C#. Isso significa que, quando você passa um bool como argumento para um método, ele é passado por valor, não por referência. Para ilustrar, vale evocar a famosa lenda do toque do Rei Midas. Tudo que ele tocava virava ouro, e podemos usar uma função para simular isso na figura 81. Porém, ao passar uma variável booleana para essa função, seu valor original fora da função não será alterado, porque o booleano é passado por valor.

```
1.  public void ToqueDeMidas(bool virouOuro)
2.  {
3.      virouOuro = true;
4.  }
```

Figura 81 – ToqueDeMidas: booleano na passagem de parâmetro

Na figura 82, entradaUsuario representa a entrada fornecida, enquanto valorConvertido representa o valor booleano resultante da tentativa de conversão.

```
1.  string entradaUsuario = "True"; // Pode ser uma entrada do usuário
2.  bool valorConvertido;
3.
4.  if (Boolean.TryParse(entradaUsuario, out valorConvertido))
5.  {
6.    if (valorConvertido)
7.    {
8.        Console.WriteLine("O valor é verdadeiro, como a existência dos
    Deuses do Olimpo!");
9.    }
10.   else
11.   {
12.       Console.WriteLine("O valor é falso, como os rumores sobre a
    humanidade de Hércules!");
13.   }
14. }
15. else
16. {
17.       Console.WriteLine("Entrada inválida.");
18. }
```

Figura 82 – Boolean.TryParse: tentativa de converter valor booleano

No .NET Framework existe uma estrutura chamada Boolean que representa um valor booleano (verdadeiro ou falso). Entretanto, é importante notar que bool é simplesmente um alias (uma espécie de apelido) para System.Boolean:

- **Propriedades**: TrueString retorna a string "True"; e FalseString, a string "False".
- **Métodos**:
  - **ToString()**: converte o valor booleano do objeto atual em sua representação de string equivalente ("True" ou "False").

- **Equals(Object obj)**: retorna um valor que indica se uma instância é igual a um objeto especificado.
- **GetType()**: obtém o Type da instância atual.
- **Parse(String value)**: converte a representação de string especificada de um valor lógico em seu equivalente Boolean.
- **TryParse(String value, out Boolean result)**: tenta converter a representação de string especificada de um valor lógico em seu equivalente Boolean, retornando um valor que indica se a conversão foi bem-sucedida.

3.5 Cadeias de caracteres

Uma cadeia de caracteres – mais comumente chamada de string em linguagens de programação – caracteriza uma sequência de caracteres que pode ser usada para representar texto. Na verdade string é um objeto, ao contrário de algumas outras linguagens que a tratam como um tipo primitivo. Por ser um objeto, temos à disposição uma série de métodos e propriedades úteis para manipulação.

Mas antes de mergulharmos nessas funcionalidades, vamos entender a fundação. Imagine que cada string seja como uma tapeçaria, onde cada caractere é um fio individual. Na mitologia grega, poderíamos ter uma string que contém o nome Athena (linha 1 da figura 83). O valor Athena é armazenado na memória, e a variável deusa aponta para esse local. Cada caractere em Athena ocupa uma posição específica, começando em 0 e indo até n-1, onde n é o comprimento da string.

```
1.  string deusa = "Athena";
2.
3.  Console.WriteLine(deusa.Length); // Resultado: 6
4.
5.  string titulo = "Zeus, o Rei dos Deuses";
6.  titulo = titulo.Replace("Rei dos Deuses", "Poderoso");
7.  Console.WriteLine(titulo); // Resultado: Zeus, o Poderoso
```

Figura 83 – Exemplos de manipulação de cadeias de caracteres

O comprimento de uma string pode ser encontrado usando a propriedade Length. Na linha 3 da figura, o código irá imprimir 6, que é o número de caracteres em Athena.

Um aspecto fundamental das strings em C# é a **imutabilidade**; ou seja, uma vez criada uma string, ela não pode ser alterada. Qualquer operação que pareça modificar uma string está na verdade criando uma nova string. Para ilustrar isso, imaginemos o Olimpo, e Zeus quer mudar seu título. Ele começa como Zeus, o Rei dos Deuses, mas quer mudar para Zeus, o Poderoso. A linha 6 da figura 83 demonstra essa operação usando o método Replace da string título. Apesar de parecer que estamos modificando a string original, na verdade uma nova string é criada, ou seja, um novo endereço de memória é atribuído.

Também poderíamos ter atribuído o título Zeus, o Poderoso em vez de usar o método Replace. A imutabilidade não está na reatribuição da variável, mas na impossibilidade de modificar o objeto string em si após sua criação. Se tivéssemos uma variável string e atribuíssemos um novo valor a ela, estaríamos fazendo a variável apontar para um novo objeto string, e o objeto string anterior permaneceria inalterado (e seria recolhido pelo coletor de lixo se não houvesse mais referências a ele).

Por serem imutáveis, a linguagem pode otimizar operações de comparação de strings usando uma técnica chamada **interning**. Isso significa que, em vez de comparar cada caractere individualmente, a linguagem pode simplesmente verificar se duas referências apontam para a mesma instância de string na memória. Isso acelera extremamente as comparações de string.

Em computação, thread (mencionada brevemente no tópico 2.9 e abordada em mais detalhes no 7.6) é a menor unidade de processamento que pode ser gerenciada por um sistema operacional. Pode-se pensar nela como uma sequência independente de instruções que pode ser executada por um sistema, e que em muitos contextos pode ser executada em paralelo com outras threads. Em ambientes multithread, várias threads podem acessar uma string simultaneamente. Se fossem mutáveis, haveria um risco considerável de conflitos e condições de corrida, levando a erros e comportamento inesperado. Como são imutáveis, não há necessidade de bloqueios ou outras formas de sincronização ao acessar ou modificá-las.

A questão da imutabilidade também é importante, pois em muitos sistemas as strings são usadas como chaves em dicionários ou tabelas hash. Se fosse mutável, seu hash code poderia mudar depois de ser adicionada a uma coleção, tornando quase impossível localizá-la ou removê-la corretamente. Com strings imutáveis, uma vez que o hash code é calculado, ele nunca muda, garantindo comportamento previsível. Também são usadas em contextos críticos, como caminhos de arquivos, URLs, identificadores e outros. E permitir que essas strings sejam facilmente modificadas pode levar a erros, comportamento inesperado e falhas de segurança.

Se strings fossem mutáveis, passá-las como argumentos para funções ou métodos poderia ter efeitos colaterais inesperados; por exemplo, um método que alterasse inadvertidamente o valor da string afetaria todas as referências a essa string fora do método. Strings imutáveis garantem que um método não modificará inadvertidamente o valor da string original e, uma vez criada, sua alocação de memória é final, o que permite ao sistema otimizar o armazenamento e gerenciamento de strings na memória, sem se preocupar com possíveis realocações ou expansões.

Além disso, imutabilidade tem um custo. Cada operação que "modifica" uma string resulta na criação de uma nova instância, o que pode levar a um overhead de memória e a operações mais lentas, especialmente em manipulações intensivas de string. "Overhead de memória" refere-se ao custo adicional, em termos de utilização da memória, que é necessário para gerenciar ou administrar recursos em um sistema computacional, em vez de simplesmente armazenar os próprios dados. Em outras palavras, é a quantidade de memória usada pelo sistema para gerenciar a memória ou outros recursos, em vez de apenas armazenar os dados de que o programa realmente precisa. Para contornar isso, C# (e muitas outras linguagens) oferece tipos mutáveis especializados para manipulação intensiva de strings, como StringBuilder em C#, que pode ser usado quando modificações frequentes são necessárias.

StringBuilder é uma classe em C# encontrada no namespace System.Text e representa uma sequência de caracteres mutável, especialmente útil quando precisamos realizar operações de manipulação de string frequentes ou complexas. Também é capaz de indicar quanto espaço de memória reservou (o que, aliás, é diferente do comprimento, que indica quantos caracteres estão atualmente na sequência). A capacidade pode aumentar automaticamente à medida que os caracteres são adicionados, mas também podemos definir a capacidade manualmente para otimizações.

Seus principais métodos são:

- **Append**: adiciona uma string ao final da sequência atual.
- **Insert**: insere uma string em uma posição específica.
- **Remove**: remove caracteres em um intervalo específico.
- **Replace**: substitui todas as ocorrências de uma substring especificada.
- **Clear**: limpa o conteúdo do StringBuilder.

A figura 84 ilustra esses métodos. Quando um programador conclui suas manipulações e deseja obter uma string padrão imutável, pode chamar o método **ToString do StringBuilder**.

```
1.  using System.Text;
2.
3.  StringBuilder sb = new StringBuilder();
4.
5.  // Adicionando textos
6.  sb.Append("Zeus é conhecido como");
7.  sb.Append(" o rei dos deuses na mitologia grega.");
8.
9.  // Inserindo texto no início
10. sb.Insert(0, "Na Grécia antiga, ");
11.
12. // Substituindo parte do texto
13. sb.Replace("rei dos deuses", "principal deidade");
14.
15. // Obtendo a string resultante
16. string resultado = sb.ToString();
17. Console.WriteLine(resultado);
```

Figura 84 – StringBuilder: exemplo de uso e conversão para string

Como mencionado, strings são ricas em métodos que facilitam a manipulação de texto. Algumas das tarefas comuns incluem verificar se ela contém uma substring, transformar a string em maiúsculas ou minúsculas, dividir uma string em várias substrings e assim por diante.

Suponha que, em uma fábula, Hércules esteja à procura de hidras. Ele poderia verificar se a palavra hidra aparece em uma descrição usando o método Contains (como na linha 2 da figura 85). Outra operação comum é a concatenação, que é o ato de juntar duas ou mais strings para formar uma nova. No reino mitológico, se Hermes quisesse combinar mensagens de Atena e Apolo, ele poderia fazer algo conforme a linha 7 da figura 85.

A linguagem C# também oferece a facilidade de interpolação de string, utilizando-se o cifrão (**$**) antes de iniciar uma string e **{}** para delimitar as expressões ou variáveis que desejamos inserir nela, permitindo que os valores sejam inseridos diretamente dentro da string. Suponha que Poseidon queira proclamar quantos tridentes possui (a linha 11 da figura 85 demonstra um exemplo de interpolação):

```
1.  string descricao = "Hércules luta contra a hidra de Lerna.";
2.  bool temHidra = descricao.Contains("hidra");
3.  Console.WriteLine(temHidra); // Resultado: true
4.
5.  string mensagemAthena = "Defenda Atenas!";
6.  string mensagemApolo = "E ilumine o caminho.";
7.  string mensagemCombinada = mensagemAthena + " " + mensagemApolo;
8.  Console.WriteLine(mensagemCombinada); // Resultado: Defenda Atenas! E
ilumine o caminho.
9.
10. int numeroDeTridentes = 3;
11. string declaracao = $"Poseidon tem {numeroDeTridentes} tridentes.";
12. Console.WriteLine(declaracao); // Resultado: Poseidon tem 3 tridentes.
```

Figura 85 – Exemplos de manipulação de cadeias de caracteres

Também é possível usar caracteres de escape (como **\n** para nova linha ou **\t** para tabulação). Além disso, usando o prefixo @ antes de uma string, podemos criar strings verbatim, que não interpretam caracteres de escape e tratam cada caractere literalmente.

A classe String em C# representa uma sequência de caracteres e, em um nível mais técnico, uma instância da classe String é essencialmente um array de caracteres (char), mas a classe oferece uma ampla variedade de métodos que facilitam a manipulação de texto.

O quadro 9 elenca e descreve os principais métodos da classe String.

Quadro 9 – Classe String: principais métodos e propriedades

Nome	Tipo	Descrição
Clone()	Método	Cria uma cópia dessa instância
Compare()	Método	Compara duas strings e retorna um valor indicando se uma é menor, igual ou maior que a outra
Contains()	Método	Verifica se a string contém uma substring específica
EndsWith()	Método	Determina se a string termina com a substring especificada

Nome	Tipo	Descrição
Equals()	Método	Compara duas strings e determina se são iguais
IndexOf()	Método	Retorna a primeira ocorrência de uma substring
Insert()	Método	Insere uma string em uma posição especificada
IsNullOrEmpty()	Método	Verifica se a string é nula ou tem comprimento zero
IsNullOrWhiteSpace()	Método	Verifica se a string é nula, vazia ou contém apenas caracteres de espaço em branco
LastIndexOf()	Método	Retorna a última ocorrência de uma substring
Remove()	Método	Remove uma quantidade determinada de caracteres de uma posição específica
Replace()	Método	Substitui todas as ocorrências de uma substring especificada por outra substring
Split()	Método	Divide a string em substrings com base em caracteres de delimitação
StartsWith()	Método	Determina se a string começa com a substring especificada
Substring()	Método	Retorna uma substring de uma string
ToLower()	Método	Converte a string para minúsculas
ToUpper()	Método	Converte a string para maiúsculas
Trim()	Método	Remove todos os espaços em branco no início e no final da string
Length	Propriedade	Retorna o número de caracteres na string
Chars	Propriedade	Obtém um caractere de uma posição específica na string

Saiba mais

Para se aprofundar no uso de strings, uma referência importante se encontra no site oficial da Microsoft:

STRINGS and string literals. *Learn Microsoft*, 27 maio 2023. Disponível em: https://tinyurl.com/3633wrt2. Acesso em: 30 out. 2023.

Quando se compara strings em C#, o método Equals ou o operador **==** deve ser usado em vez do operador-padrão de igualdade de referência, especialmente se o que se deseja é comparar o conteúdo das strings, e não suas referências. Outra curiosidade é que alguns métodos da classe String – como Compare e ToLower – contêm variações que aceitam um argumento de "cultura" (ou *culture*, em inglês), que considera convenções linguísticas e culturais específicas ao realizar operações.

Por fim, uma característica notável em C# que envolve a manipulação de strings é o suporte a **expressões regulares**, sequências de caracteres que formam um padrão de pesquisa. Podem ser usadas para tarefas complexas, como validar o formato de uma entrada. Imagine que um oráculo precisa verificar se uma profecia está no formato correto, usando expressões regulares; ele pode garantir que a profecia tenha a estrutura desejada (detalharemos esse assunto no tópico 5.5).

3.6 Variáveis

As variáveis permitem que os desenvolvedores armazenem e manipulem dados em seus programas. Uma variável é um nome associado a uma localização de memória. No contexto da programação C#, declaramos uma variável para reservar espaço na memória, e esse espaço é usado para armazenar valores, que podem ser alterados durante a execução do programa.

Por exemplo, imagine o deus grego Hermes, e queremos armazenar a velocidade de voo dele em nosso programa. Em C#, poderíamos fazer isso da seguinte maneira: "double velocidadeHermes = 15.5;", sendo **double** o tipo da variável, e **velocidadeHermes** o nome da variável. O C# oferece uma variedade de tipos de variáveis, cada uma projetada para armazenar dados específicos. Temos tipos de valor, que incluem tipos numéricos, char e bool, e tipos de referência, como string, arrays e classes.

A figura 86 apresenta a saída no console com o valor de diversas variáveis, conforme programado na figura 87.

```
Número de labores de Hércules: 12
Altura do Monte Olimpo: 2.918 km
Zeus é o rei dos deuses? True
Inicial do nome de Athena: A
Cidade favorita de Athena: Atenas
Número de musas: 9
Distância corrida por Apolo: 400.5 metros
Idade de Prometeu quando roubou o fogo: 123
```

Figura 86 – Saída do console após as declarações das varáveis

O tamanho e o intervalo de valores que podem ser armazenados em variáveis numéricas dependem do tipo específico. Uma característica interessante do C# é sua capacidade de inferir tipos de variáveis. Utilizando a palavra-chave **var**, o compilador determina automaticamente o tipo de variável com base no valor atribuído. Por exemplo, se escrevermos "var idadeAthena = 23;", o C# inferirá que **idadeAthena** é do tipo int. A linha 30 da figura 87 faz uma inferência similar.

```
1.  using System;
2.  namespace MitologiaGrega
3.  {
4.  class Program
5.              {
6.     static void Main(string[] args)
7.     {
8.        // Declaração de variável do tipo int (inteiro)
9.        int numeroLaboresHercules = 12;
10.
11.       // Declaração de variável do tipo double (ponto flutuante de dupla
precisão)
12.       double alturaMonteOlimpoEmKm = 2.918;
13.
14.       // Declaração de variável do tipo bool (verdadeiro ou falso)
15.       bool zeusEhReiDosDeuses = true; // Zeus é considerado o rei dos
deuses na mitologia grega
16.
17.       // Declaração de variável do tipo char (caractere)
18.       char inicialNomeAthena = 'A';
19.
20.       // Declaração de variável do tipo string
21.       string nomeCidadeFavoritaDeAthena = "Atenas"; // Athena deu seu
nome à cidade de Atenas
22.
23.       // Declaração de variável do tipo byte (inteiro de 8 bits)
24.       byte numeroDeusasMusas = 9;
25.
26.       // Declaração de variável do tipo float (ponto flutuante de
precisão simples)
27.       float distanciaCorridaDeApoloEmMetros = 400.5F;
28.
29.       // Declaração usando a palavra-chave var - o compilador inferirá
o tipo
30.       var idadePrometeuQuandoRoubouOFogo = 123; // O tipo inferido é
int
31.       // Exibindo valores das variáveis
32.       Console.WriteLine($"Número de labores de Hércules:
{numeroLaboresHercules}");
33.       Console.WriteLine($"Altura do Monte Olimpo:
{alturaMonteOlimpoEmKm} km");
34.       Console.WriteLine($"Zeus é o rei dos deuses?
{zeusEhReiDosDeuses}");
35.       Console.WriteLine($"Inicial do nome de Athena:
{inicialNomeAthena}");
36.       Console.WriteLine($"Cidade favorita de Athena:
{nomeCidadeFavoritaDeAthena}");
37.       Console.WriteLine($"Número de musas: {numeroDeusasMusas}");
38.       Console.WriteLine($"Distância corrida por Apolo:
{distanciaCorridaDeApoloEmMetros} metros");
39.       Console.WriteLine($"Idade de Prometeu quando roubou o fogo:
{idadePrometeuQuandoRoubouOFogo}");
40.       }
41.    }
42. }
```

Figura 87 – Exemplos de declaração de variáveis

Em termos de convenção, ao nomear variáveis em C#, é comum seguir o padrão camelCase, e os nomes devem ser descritivos e claros sobre o que representam, como forcaHercules ou numeroSereias. É importante notar que C# é uma linguagem fortemente tipada, ou seja, o tipo de uma variável é verificado pelo compilador, e tentar atribuir o valor de um tipo incompatível resultará em erro de compilação; por exemplo, tentar atribuir uma string a velocidadeHermes, que foi declarado como double, resultaria em erro.

3.7 Operadores e expressões

Operadores são símbolos que realizam operações em variáveis e valores, e C#, como linguagem de POO, contém um rico conjunto de operadores que permitem manipular variáveis de diversos tipos; esses operadores são semelhantes a funções matemáticas e lógicas que se aplicam em operações básicas. Imagine que Atena, deusa da sabedoria, deseja calcular a idade média entre três grandes deuses: Zeus, Poseidon e Hades. Em C# essa operação pode ser feita com operadores aritméticos (conforme a figura 88), em que utilizamos os operadores **+** e **/** para adição e divisão, respectivamente (linha 5).

```
1.  int idadeZeus = 5000;
2.  int idadePoseidon = 4800;
3.  int idadeHades = 4900;
4.
5.  int idadeMedia = (idadeZeus + idadePoseidon + idadeHades) / 3;
```

Figura 88 – Operadores aritméticos: exemplo de uso

Assim, se Atena quisesse saber se Zeus é o mais velho entre eles, ela usaria operadores de comparação e lógica. Os operadores **>** e **&&** representam a comparação "maior que" e "e lógico", respectivamente.

```
1.  bool zeusEhOMaisVelho = idadeZeus > idadePoseidon && idadeZeus >
idadeHades;
```

Figura 89 – Operadores de comparação e lógica: exemplo de uso

Suponha que Afrodite queira oferecer uma rosa a Hércules caso ele não seja mortal. Em C# isso seria representado utilizando o operador condicional, conforme a figura 90. Nesse contexto, "**? :**" é o operador ternário que permite avaliar uma expressão e retornar um valor se for verdadeiro e outro se for falso.

```
1.  bool herculesEhMortal = true;
2.  string presente = herculesEhMortal ? "Nada" : "Rosa";
```

Figura 90 – Operador condicional: exemplo de uso

Há também operadores especiais, como "**??**", que retorna o valor da esquerda se ele não for nulo, ou o valor da direita se for.

```
1.  string destinatarioPrincipal = null;
2.  string destinatarioReserva = "Dionísio";
3.  string destinatarioMensagem = destinatarioPrincipal  ??
destinatarioReserva;
```

Figura 91 – Operador de coalescência nula: exemplo de uso

Imagine que Hermes, o mensageiro dos deuses, queira entregar uma mensagem a Apolo, mas, se Apolo não estiver disponível, ele entregará a Dionísio (figura 91). Diversos tipos de operador podem ser usados em cenários variados; a figura 92 exemplifica o uso de alguns desses operadores, já o quadro 10 apresenta alguns dos operadores mais usados pelos desenvolvedores em programas do mundo real.

```
1.using System;
2.
3.namespace MitologiaGrega
4.{
5.   class Program
6.   {
7.      static void Main(string[] args)
8.      {
9.         // O poder de Zeus é medido em raios por dia
10.        int poderZeus = 100;
11.
12.        // O poder de Poseidon é medido em ondas por dia
13.        int poderPoseidon = 80;
14.
15.        // O poder de Hades é medido em almas coletadas por dia
16.        int poderHades = 75;
17.
18.        // Quem é o mais poderoso?
19.        bool zeusMaisPoderoso = (poderZeus > poderPoseidon) && (poderZeus
> poderHades);
20.
21.        // A média de poder entre eles
22.        double mediaPoder = (poderZeus + poderPoseidon + poderHades)/
3.0;
23.
24.        // O poder combinado de Zeus e Poseidon
25.        int poderCombinado = poderZeus + poderPoseidon;
26.
27.        // Se Hades e Poseidon unirem forças, eles serão mais poderosos
que Zeus
28.        bool hadesPoseidonVsZeus = (poderHades + poderPoseidon) >
poderZeus;
29.
30.        // Poseidon se fortalece ao combinar seu poder com o de Zeus, mas
divide por dois
31.        poderPoseidon += poderZeus;
32.        poderPoseidon /= 2;
33.
34.        Console.WriteLine($"Zeus é o mais poderoso? {zeusMaisPoderoso}");
35.        Console.WriteLine($"Média de poder entre os deuses:
{mediaPoder}");
36.        Console.WriteLine($"Hades e Poseidon juntos são mais poderosos
que Zeus? {hadesPoseidonVsZeus}");
37.        Console.WriteLine($"Poder de Poseidon após se fortalecer:
{poderPoseidon}");
38.      }
39.   }
40.}
```

Figura 92 – Uso de operadores: diversos exemplos

Quadro 10 – Principais operadores: descrição e exemplos

Tipo de operador	Símbolo	Exemplo	Descrição
Aritmético	+	a + b	Adição
	-	a – b	Subtração
	*	a * b	Multiplicação
	/	a / b	Divisão
	%	a % b	Módulo (resto da divisão)
Comparação	>	a > b	Maior que
	<	a < b	Menor que
	>=	a >= b	Maior ou igual
	<=	a <= b	Menor ou igual
	==	a == b	Igualdade
	!=	a != b	Diferente
Lógico	&&	a && b	E lógico
	||	a || b	Ou lógico
	!	!a	Negação lógica
Atribuição	=	a = b	Atribui o valor de b a a
	+=	a += b	Atribui a soma de a e b a a
	-=	a -= b	Atribui a diferença entre a e b a a
	*=	a *= b	Atribui o produto de a e b a a
Bit a bit	&	a & b	E bit a bit
	|	a | b	Ou bit a bit
	^	a ^ b	Ou exclusivo (XOR) bit a bit
Condicional	? :	a ? b : c	Retorna b se a for verdadeiro, c caso contrário
Coalescência nula	??	a ?? b	Retorna a se não for nulo, senão retorna b
Operador de incremento	++	a++	Incrementa o valor de a
Operador de decremento	--	a--	Decrementa o valor de a

Expressões em C# são combinações de operandos (como literais, variáveis, propriedades etc.) e operadores que, quando avaliados, retornam um único valor. São fundamentais para qualquer linguagem de programação, pois são a base da lógica e do cálculo.

Suponha que Zeus queira lançar raios. A quantidade de raios que ele lança pode ser calculada por uma expressão que combina seu humor, sua energia e a presença de outros deuses. A expressão **(humorZeus * energiaZeus) + (poseidonPresente ? 5 : 0)**, da linha 5 da figura 93, é uma combinação de operandos e operadores que calcula o número total de raios lançados por Zeus, baseada em seu humor, sua energia e na presença de Poseidon.

```
1.  int humorZeus = 7; // numa escala de 1 a 10
2.  int energiaZeus = 80; // porcentagem de energia
3.  bool poseidonPresente = true;
4.
5.  int raiosLancados = (humorZeus * energiaZeus) + (poseidonPresente ? 5
: 0);
```

Figura 93 – raiosLancados por Zeus: exemplo de expressão

Outra expressão pode envolver a tentativa de Afrodite encantar seres humanos. A eficácia do seu encanto pode depender do local em que ela está (por exemplo, perto do mar ou no Olimpo) e da presença de certos artefatos mágicos. Conforme a linha 5 da figura 94, a expressão **charmeAfrodite + (pertoDoMar ? 10 : –10) + (colarMagico ? 20 : 0)** calcula a eficácia do encanto de Afrodite.

```
1.  int charmeAfrodite = 90; // numa escala de 1 a 100
2.  bool pertoDoMar = true;
3.  bool colarMagico = false;
4.
5.  int eficaciaEncanto = charmeAfrodite + (pertoDoMar ? 10 : -10) +
(colarMagico ? 20 : 0);
```

Figura 94 – eficaciaEncanto de Afrodite: outro exemplo de expressão

As expressões não se limitam a cálculos aritméticos ou lógicos. Elas podem ser usadas em muitos contextos, como:

- **Atribuições**: int totalRaios = raiosLancados;
- **Condições**: if (eficaciaEncanto > 100) {...}
- **Retornos de método**: return totalRaios;
- **Argumentos de método**: LancarRaios(raiosLancados);

Elas podem ser tão simples quanto um valor literal ou tão complexas quanto uma combinação de várias subexpressões, funções e propriedades, sendo avaliadas para produzir um valor que pode ser usado, por exemplo, para tomar uma decisão, realizar um cálculo ou simplesmente exibir informações.

Em resumo, expressões em C# são construções fundamentais, que permitem ao programador especificar lógica e cálculo, enquanto os operadores são as ferramentas que manipulam e combinam operandos dentro dessas expressões.

3.8 Declarações

As declarações em C# são instruções fundamentais que especificam como os dados são representados e manipulados no código. Incluem declarações de variáveis, constantes, condicionais, loops, métodos, classes e namespaces.

Lembrete

Como vimos no tópico 3.6, variáveis armazenam informações que podem ser usadas e manipuladas posteriormente no código. Em C#, cada variável tem um tipo e um nome. O tipo determina a natureza dos dados que a variável pode conter, enquanto o nome se refere à variável. Já as constantes (tópico 2.2) são semelhantes às variáveis, mas, uma vez atribuídas, não podem ser alteradas. Geralmente são usadas para armazenar valores que nunca mudam no programa.

A figura 95 exemplifica os diversos tipos de declaração.

```
1.  // Declaração de Namespace
2.  namespace MitologiaGrega
3.  {
4.  // Declaração de Classe
5.  class Program
6.  {
7.    // Declaração de Método
8.    static void Main(string[] args)
9.    {
10.     // Declaração de Variáveis
11.     string deus = "Zeus";
12.     int idade = 3000;
13.     // Declaração de Constante
14.     const string LUGAR_OLIMPO = "Monte Olimpo";
15.     // Uso de Método
16.     InvocarDeus(deus);
17.     // Uso de Classe (criação de objeto)
18.     Deus poseidon = new Deus("Poseidon", "mar");
19.     poseidon.MostrarInfo();
20.     Console.ReadLine();
21.   }
22.     // Declaração de Método
23.   static void InvocarDeus(string nomeDeus)
24.   {
25.   Console.WriteLine($"Invocando {nomeDeus}!");
26.   }
27. }
28.     // Declaração de Classe
29.     class Deus
30.   {
31.     public string Nome { get; set; }
32.     public string Domínio { get; set; }
33.
34.     // Declaração de Método
35.     public Deus(string nome, string domínio)
36.     {
37.     Nome = nome;
38.     Domínio = domínio;
39.     }
40.     // Declaração de Método
41.     public void MostrarInfo()
42.     {
43.     Console.WriteLine($"{Nome} é o deus(a) de {Domínio}.");
44.     }
45.   }
46.
47.     // Declaração de outra Classe dentro do mesmo Namespace
48.     class Titan
49.     {
50.       // Detalhes do Titan
51.     }
52.   }
```

Figura 95 – Declarações: variáveis, constantes, métodos, classes e namespaces

Lembrete

Os tópicos 2.1, 2.3 e 3.2 abordaram, respectivamente, declarações de classe, métodos e namespaces. Convém recordar que em C# as classes representam blueprints para criar objetos, os métodos das classes são blocos de código que realizam uma tarefa específica e podem ser chamados sempre que necessário. Blueprint é uma palavra em inglês que, no sentido literal, refere-se a um conjunto de planos ou desenhos técnicos usados para guiar a construção ou fabricação de algo, especialmente edifícios ou navios. Esses desenhos apresentam especificações, detalhes e procedimentos para seguir.

Na programação, dizer que uma classe é blueprint para criar objetos é uma maneira de ilustrar que a classe define a estrutura e o comportamento que os objetos criados a partir dela terão, mas sem ser uma instância real do objeto. Da mesma forma que o blueprint de um edifício mostra onde as portas e janelas estarão (mas não é uma casa real), uma classe define propriedades e métodos que seus objetos terão, mas a classe em si não é o objeto. Já namespaces são usados para organizar o código em unidades lógicas.

As **declarações condicionais** não são exclusividade das linguagens orientadas a objetos. Assim como nas linguagens estruturadas, elas são fundamentais para controlar o fluxo em uma aplicação, permitindo que o programa tome decisões baseadas em condições e execute diferentes blocos de código de acordo com o resultado dessa avaliação. A declaração **if** avalia uma expressão entre parênteses; se essa expressão resultar em verdadeiro (true), o bloco de código subsequente é executado. Caso contrário, se houver uma cláusula **else** após o if, o bloco de código associado ao else é executado.

A figura 96 ilustra esse condicional. A declaração if pode ser usada sozinha para executar um bloco de código se uma condição específica for verdadeira, ou seja, caso não seja necessário se preocupar com o que acontece se a condição for falsa.

```
1.  using System;
2.  class Program
3.  {
4.    static void Main(string[] args)
5.    {
6.      string deus = "Zeus";
7.      // Declarações Condicionais
8.      if (deus == "Zeus")
9.      {
10.       Console.WriteLine("Zeus é o deus do céu e do trovão!");
11.     }
12.     else
13.     {
14.       Console.WriteLine("Este não é Zeus.");
15.     }
16.   }
17. }
```

Figura 96 – Declaração de condicional: if e else

Por outro lado, para avaliar várias condições em sequência, podemos usar **else if**. A figura 97 ilustra uma sequência de "else ifs" encadeados (linhas 7, 11 e 15). Após a execução do código, será impresso "Deus da Guerra" na tela do Console.

```
1.  string deus = "Ares";
2.
3.  if (deus == "Zeus")
4.  {
5.      Console.WriteLine("Deus do céu e do trovão.");
6.  }
7.  else if (deus == "Poseidon")
8.  {
9.      Console.WriteLine("Deus dos mares.");
10. }
11. else if (deus == "Ares")
12. {
13.         Console.WriteLine("Deus da guerra.");
14. }
15. else if (deus == "Dionísio")
16. {
17.     Console.WriteLine("Deus do vinho e das festas.");
18. }
19. else
20. {
21.     Console.WriteLine("Deus desconhecido.");
22. }
```

Figura 97 – Declaração de condicional: else if

O else if é flexível em termos das condições que pode avaliar: consegue lidar com condições não apenas de igualdade, mas também de comparação, lógica e até mesmo combinações complexas, porém tende a ser menos legível se houver muitas condições a verificar; nesses casos a estrutura **switch** pode

ser mais conveniente. A declaração switch permite selecionar um entre vários blocos de código para ser executado. É frequentemente usada como alternativa para longas cadeias de if-else if-else.

Na figura 98 reescrevemos o exemplo da figura 97 usando switch. Compare as duas versões de código-fonte e perceba que a clareza e a legibilidade foram aprimoradas.

```
1.  string deus = "Ares";
2.
3.  switch (deus)
4.  {
5.      case "Zeus":
6.        Console.WriteLine("Deus do céu e do trovão.");
7.        break;
8.      case "Poseidon":
9.        Console.WriteLine("Deus dos mares.");
10.       break;
11.     case "Ares":
12.       Console.WriteLine("Deus da guerra.");
13.       break;
14.     case "Dionísio":
15.       Console.WriteLine("Deus do vinho e das festas.");
16.       break;
17.   default:
18.       Console.WriteLine("Deus desconhecido.");
19.       break;
20. }
```

Figura 98 – Declaração de condicional: switch

Portanto, se estiver verificando igualdade em uma única variável e tiver muitos casos pela frente, o switch é mais legível e organizado. Tradicionalmente é usado para avaliar a igualdade de uma expressão com valores constantes e, em versões mais recentes do C#, podemos usar padrões e flexibilizá-lo. Em geral, também é mais eficiente que else if se houver muitos casos a verificar; em contrapartida, se tivermos condições mais complexas (por exemplo, combinações de verificações de igualdade, comparações de magnitude etc.) ou apenas um pequeno número de verificações a fazer, o else if é mais indicado.

As **estruturas de repetição** (ou loops) são usadas, assim como nas linguagens estruturadas, para repetir um bloco de código enquanto uma condição específica for verdadeira ou para repetir um bloco por um número específico de vezes. Um loop do tipo **for** repete um bloco de código por um número específico de vezes e é estruturado em três partes: inicialização da variável de controle; condição a ser avaliada para término das repetições; e iteração que representa a forma de atualização da variável de controle após execução de cada repetição.

Na linha 9 da figura 99, a variável i foi inicializada com zero (i=0), sendo atualizada (incrementada) em uma unidade (i++) a cada execução do loop até que a condição (i<12) seja falsa. Nesse caso, quando i=12, o loop termina. Já o loop do tipo **while** repete um bloco de código enquanto uma condição for verdadeira. A linha 16 exemplifica essa estrutura: o loop executa enquanto a variável mortaisConvertidos for menor que 100. Até lá, Zeus continua convertendo mais mortais.

```
1.  using System;
2.  class Program
3.  {
4.    static void Main(string[] args)
5.    {
6.      string deus = "Zeus";
7.
8.      // Declarações de Loop: For
9.      for (int i = 0; i < 12; i ++)
10.     {
11.       Console.WriteLine($"O {i + 1}° deus olímpico.");
12.     }
13.
14.     // Declarações de Loop: While
15.     int mortaisConvertidos = 0;
16.     while (mortaisConvertidos < 100)
17.     {
18.       // Zeus converte mais mortais
19.       mortaisConvertidos++;
20.     }
21.     Console.WriteLine($"{mortaisConvertidos} mortais foram
convertidos por Zeus.");
22.     Console.ReadLine();
23.   }
24. }
```

Figura 99 – Declaração de estruturas de repetição: for e while

3.9 Vetores

Vetores, também conhecidos como arrays, são uma estrutura fundamental em muitas linguagens de programação, incluindo C#, e permitem armazenar vários itens do mesmo tipo em uma única variável, sendo usados extensivamente para armazenar, manipular e recuperar dados de forma eficiente. **Vetor** é uma coleção de variáveis do mesmo tipo, dispostas em sequência na memória e acessíveis por um índice. A declaração de um vetor envolve a especificação do tipo de seus elementos, seguido por colchetes, e então o nome do vetor. Para inicializar um vetor, o desenvolvedor pode definir explicitamente seu tamanho e/ou os valores que ele contém.

Na linha 10 da figura 100, vemos a declaração de um vetor para acomodar 5 inteiros.

```
1.  using System;
2.
3.  namespace VetoresExemplo
4.  {
5.     class Program
6.     {
7.        static void Main(string[] args)
8.        {
9.           // Declaração de um vetor com espaço para 5 inteiros.
10.          int[] numeros = new int[5];
11.
12.          // Declaração e inicialização de um vetor com nomes de deuses
gregos.
13.          string[] deusesGregos = { "Zeus", "Hera", "Poseidon", "Demeter",
"Ares" };
14.
15.          // Acesso ao primeiro deus grego (baseado em zero, então [0] é
"Zeus").
16.          string deus = deusesGregos[0];
17.          Console.WriteLine($"O primeiro deus é: {deus}");
18.
19.          // Iterando através de todos os deuses e imprimindo cada um.
20.          for(int i = 0; i < deusesGregos.Length; i++)
21.          {
22.          Console.WriteLine($"Deus #{i + 1}: {deusesGregos[i]}");
23.          }
24.
25.          // Declaração e inicialização de um vetor multidimensional
representando um tabuleiro com guerreiros dos deuses.
26.          string[,] tabuleiro =
27.          {
28.             { "Zeus", "Hera" },
29.             { "Poseidon", "Ares" }
30.          };
31.
32.          Console.WriteLine("Tabuleiro de Guerreiros:");
33.          for(int i = 0; i < 2; i++)
34.          {
35.             for(int j = 0; j < 2; j++)
36.             {
37.                Console.WriteLine($"Posição [{i},{j}]: {tabuleiro[i,j]}");
38.             }
39.          }
40.       }
41.    }
42. }
```

Figura 100 – Vetores: exemplos de declaração e utilização

Vetores são baseados em zero, ou seja, o primeiro elemento tem índice 0, o segundo tem índice 1 e assim por diante. Um elemento do vetor é acessado através de seu índice. A linha 16 (deusesGregos[0]) representa Zeus, pois está na primeira posição do vetor. Similarmente, deusesGregos[1] permite acessar Hera, que está armazenada na segunda posição do vetor. A tentativa de acessar um índice fora dos limites do vetor resultará em uma exceção IndexOutOfRangeException.

Observação

Veremos o tratamento de exceções no tópico 5.1, mas é prudente verificar o tamanho do vetor usando a propriedade Length antes de acessá-lo (vide linha 20 da figura 100).

Os vetores em C# são objetos e, portanto, contêm métodos associados a eles. Por exemplo, o método Array.Resize pode ser usado para alterar o tamanho de um vetor existente. Além disso, a classe Array fornece uma série de métodos estáticos úteis para manipular vetores. O quadro 11 apresenta os principais métodos.

Quadro 11 – Classe Array: principais métodos

Método	Descrição
Clear	Define um intervalo de elementos no array como o valor-padrão do tipo do array (normalmente zero)
Copy	Copia elementos de um array para outro
CopyTo	Copia todos os elementos do array unidimensional atual para um array unidimensional compatível
Exists	Determina se um elemento satisfaz as condições definidas por um predicado especificado
Find	Procura um elemento que corresponda às condições definidas por um predicado especificado
FindAll	Obtém todos os elementos que correspondem às condições definidas por um predicado especificado
FindIndex	Retorna o índice do primeiro elemento encontrado que corresponde às condições
FindLast	Retorna o último elemento encontrado que corresponde às condições
FindLastIndex	Retorna o índice do último elemento encontrado que corresponde às condições
ForEach	Executa a ação especificada em cada elemento de um array
IndexOf	Retorna o índice da primeira ocorrência de um valor em um array unidimensional
LastIndexOf	Retorna o índice da última ocorrência de um valor em um array unidimensional
Reverse	Inverte a ordem dos elementos em um array unidimensional inteiro ou em uma porção do array
Sort	Classifica os elementos em um array inteiro ou uma porção de um array unidimensional

A figura 101 aponta exemplos de alguns métodos dessa classe.

```
1.using System;
2.namespace ExemploMitologiaGrega
3.{
4.   class Program
5.   {
6.      static void Main(string[] args)
7.      {
8.         // 1. Declarando e inicializando o array.
9.         string[] deidades = { "Zeus", "Hera", "Ares", "Afrodite",
"Hermes", "Hades", "Artemis", "Apolo" };
10.
11.        // 2. Usando Array.Find para encontrar a primeira deidade que
começa com a letra 'A'.
12.        string primeiraDeidadeComA = Array.Find(deidades, deidade =>
deidade.StartsWith("A"));
13.        Console.WriteLine($"Primeira deidade com 'A': {primeiraDeidadeComA}");
// Resultado: Ares
14.
15.        // 3. Usando Array.FindAll para encontrar todas as deidades que
têm menos de seis letras.
16.        string[] deidadesCurtas = Array.FindAll(deidades, deidade =>
deidade.Length < 6);
17.        Console.WriteLine("Deidades com menos de 6 letras: " + string.
Join(", ", deidadesCurtas));
18.
19.        // 4. Usando Array.Exists para verificar se há alguma deidade com
nome "Apolo".
20.        bool existeApolo = Array.Exists(deidades, deidade => deidade==
"Apolo");
21.        Console.WriteLine($"Existe Apolo? {existeApolo}"); // Resultado: True
22.
23.        // 5. Usando Array.IndexOf para encontrar o índice de "Hades".
24.        int indiceHades = Array.IndexOf(deidades, "Hades");
25.        Console.WriteLine($"Índice de Hades: {indiceHades}");// Resultado: 5
26.
27.        // 6. Usando Array.Sort para ordenar as deidades em ordem alfabética.
28.        Array.Sort(deidades);
29.        Console.WriteLine("Deidades em ordem alfabética: " + string.Join)
(", ", deidades));
30.
31.        // 7. Usando Array.Reverse para inverter a ordem das deidades.
32.        Array.Reverse(deidades);
33.        Console.WriteLine("Deidades em ordem reversa: " + string.Join)
(", ", deidades));
34.
35.        // 8. Usando Array.ForEach para imprimir todas as deidades.
36.        Console.WriteLine("Lista de Deidades:");
37.        Array.ForEach(deidades, deidade => Console.WriteLine($"- {deidade}"));
38.
39.        // 10. Usando Array.LastIndexOf para encontrar o último índice de
"Zeus" (útil após duplicatas).
40.        int ultimoIndiceZeus = Array.LastIndexOf(deidades, "Zeus");
41.        Console.WriteLine($"Último índice de Zeus: {ultimoIndiceZeus}");
42.     }
43.  }
44.}
```

Figura 101– Métodos da classe Array: exemplos de utilização

Na linha 9 da figura, criamos um vetor com deidades gregas e executamos uma variedade de operações para demonstrar os diferentes métodos da classe Array. As operações incluem buscar, verificar existência, ordenar, inverter e imprimir elementos do vetor.

Dada a imutabilidade do tamanho dos vetores após sua criação, em muitos cenários as coleções dinâmicas, como List<T>, podem ser mais apropriadas.

Observação

Estudaremos coleções apenas no tópico 5.4, mas já vale frisar que vetores têm a vantagem de ser mais leves em termos de sobrecarga de memória e geralmente oferecem acesso mais rápido a seus elementos.

C# também suporta vetores multidimensionais e vetores de matrizes. Enquanto um vetor unidimensional é uma linha de elementos, um vetor multidimensional pode ser visualizado como matriz ou até mesmo como cubo de dados. Por exemplo, a figura 100 (linha 25 à 30) representa um tabuleiro de jogo com guerreiros dos deuses gregos. A figura 102 ilustra o que será exibido na tela do console após a execução da linha 32 à 39.

```
Tabuleiro de Guerreiros:

Posição [0,0]: Zeus

Posição [0,1]: Hera

Posição [1,0]: Poseidon

Posição [1,1]: Ares
```

Figura 102 – Exibição do tabuleiro na tela do console

Poderíamos substituir o código da linha 32 à 39 pelo código da figura 103 para uma exibição mais espacial (duas dimensões) do tabuleiro no console.

```
Console.WriteLine("Tabuleiro de Guerreiros:\n");
Console.WriteLine("+---------+---------+");
for (int i = 0; i < 2; i++)
    {
      for (int j = 0; j < 2; j++)
        {
        // Alinhar o texto ao centro
        int espacoEsquerdo = (9 - tabuleiro[i, j].Length) / 2;
        int espacoDireito = 9 - tabuleiro[i, j].Length - espacoEsquerdo;
        Console.Write("|");
        Console.Write(new string(' ', espacoEsquerdo) + tabuleiro[i, j] +
new string(' ', espacoDireito));
        }
        Console.WriteLine("|\n+---------+---------+");
    }
```

Figura 103 – Trecho de código para exibir o tabuleiro em duas dimensões

A figura 104 apresenta a nova saída do console, após a execução do código-fonte.

```
Tabuleiro de Guerreiros:

+---------+---------+
|  Zeus   |  Hera   |
+---------+---------+
| Poseidon|  Ares   |
+---------+---------+
```

Figura 104 – Nova exibição do tabuleiro na tela do console (em duas dimensões)

3.10 Enumeração

Enumeração é um tipo distinto, que consiste em um conjunto de constantes nomeadas, essencialmente grupos de constantes inteiras com nomes que facilitam a leitura e manutenção de um programa. A figura 105 apresenta a sintaxe básica para declarar uma enumeração, e a figura 106 utiliza os dias da semana de acordo com a sintaxe.

```
enum NomeDaEnumeração
{
    Constante1,
    Constante2,
    Constante3,
    // ...
}
```

Figura 105 – Enumeração: sintaxe básica

```
enum DiaDaSemana
{
    Domingo,
    Segunda,
    Terça,
    Quarta,
    Quinta,
    Sexta,
    Sábado
}
```

Figura 106 – Enumeração: exemplo com dias da semana

Por padrão, o primeiro membro de um enum tem valor 0, e o valor de cada membro subsequente é incrementado em 1. No exemplo anterior, domingo tem valor 0, segunda tem valor 1, e assim por diante. É possível, entretanto, especificar valores para os membros se necessário. A figura 107 ilustra essa flexibilidade.

```
enum DiaDaSemana
{
    Domingo = 0,
    Segunda = 1,
    Terca = 2,
    Quarta = 3,
    Quinta = 4,
    Sexta = 5,
    Sábado = 6
}
```

Figura 107 – Especificação de valores para a enumeração

Enums são úteis porque melhoram a legibilidade do código e reduzem a possibilidade de erro ao restringir valores a um conjunto finito de opções. Por exemplo, em vez de passar um número inteiro para representar um dia da semana, você pode usar a enumeração DiaDaSemana, tornando o código mais claro e menos propenso a erros. Para usar um valor enum, podemos fazer referência ao tipo de enum e ao valor que se deseja, conforme a figura 108.

```
DiaDaSemana hoje = DiaDaSemana.Sexta;
```

Figura 108 – Tipos enumerados por referência

```
int numeroDoDia = (int)DiaDaSemana.Sexta; // Isso retornará 5
```

Figura 109 – Conversão de um tipo enumerado para inteiro

A figura 110 ilustra a criação e utilização de um tipo enumerado, bem como sua conversão para inteiro e vice-versa.

```
1.  using System;
2.
3.  namespace MitologiaGrega
4.  {
5.     // Declaração da enumeração representando os principais deuses
olímpicos.
6.     enum DeusOlimpico
7.     {
8.     Zeus,        // 0
9.     Hera,        // 1
10.    Poseidon,    // 2
11.    Demeter,     // 3
12.    Ares,        // 4
13.    Atena,       // 5
14.    Apollo,      // 6
15.    Artemis,     // 7
16.    Hefesto,     // 8
17.    Afrodite,    // 9
18.    Hermes,      // 10
19.    Dionisio     // 11
20.    }
21.
22.    class Program
23.    {
24.       static void Main()
25.       {
26.          // Atribuindo um valor do enum a uma variável.
27.          DeusOlimpico deusFavorito = DeusOlimpico.Atena;
28.
29.       Console.WriteLine($"Meu deus olímpico favorito é: {deusFavorito}");
30.
31.       // Convertendo um valor enum para inteiro.
32.       int posicaoNoOlimpo = (int)deusFavorito;
33.       Console.WriteLine($"{deusFavorito} está na posição
{posicaoNoOlimpo + 1} no Olimpo.");
34.
35.       // Convertendo um inteiro de volta para um valor enum.
36.       int numeroDeus = 4;
37.       DeusOlimpico deusPorNumero = (DeusOlimpico)numeroDeus;
38.       Console.WriteLine($"O deus na posição {numeroDeus + 1} é:
{deusPorNumero}");
39.
40.       Console.ReadLine();
41.          }
42.       }
43.    }
```

Figura 110 – Exemplo de tipo enumerado utilizando a mitologia grega

Em resumo, enumerações são uma maneira de agrupar e nomear constantes em C#, tornando o código mais legível e mantendo um conjunto definido de valores possíveis para variáveis de determinado tipo.

3.11 Estruturas

Estrutura (ou struct) é um tipo de valor que pode ser usado para encapsular pequenos grupos de variáveis relacionadas. Uma das principais diferenças entre uma struct e uma classe (class) em C# é que estruturas são tipos de valor, enquanto classes são tipos de referência; ou seja, quando uma estrutura é passada para uma função, ela é passada por valor (ou seja, uma cópia dos dados é feita), enquanto uma classe é passada por referência. Estruturas e classes têm propósitos diferentes, e escolher entre elas depende das necessidades do problema que desejamos resolver.

Algumas vantagens do uso de estruturas em comparação com classes:

- **Eficiência de alocação de memória**: structs são tipos de valor e alocados na pilha, enquanto classes são tipos de referência e alocadas no heap. A alocação na pilha é geralmente mais rápida do que a alocação no heap, e não há custo associado à coleta de lixo, uma vez que os valores da pilha são desalocados automaticamente quando saem do escopo.

- **Semântica de passagem por valor**: ao passarmos uma struct como argumento para uma função ou método, também passamos uma cópia da struct. Isso pode ser útil quando queremos garantir que o original não seja modificado. Com classes a referência é passada, e o objeto original pode ser alterado através dessa referência.

- **Imutabilidade**: é comum (e muitas vezes aconselhado) projetar structs para ser imutáveis, o que pode tornar o código mais previsível e menos propenso a erros. Classes, embora possam ser feitas imutáveis, são frequentemente projetadas para ser mutáveis.

- **Representação de conceitos leves**: structs são em geral usados para representar conceitos simples ou tipos de dados leves, como um ponto em um espaço bidimensional ou um intervalo. A semântica de valor de uma struct pode ser mais intuitiva para tais conceitos.

A respeito da eficiência de alocação de memória, é importante mencionar as diferenças entre pilha (stack) e heap. Pilha é uma região da memória que opera de maneira automática e organizada. Quando uma função é chamada em um programa, suas variáveis locais são armazenadas na pilha. À medida que a função executa suas operações e eventualmente retorna, essas variáveis são automaticamente removidas, disponibilizando a memória para outras operações. Esse comportamento é chamado *last in first out* (Lifo), ou seja, a última informação colocada na pilha é a primeira a ser retirada.

Uma das características distintivas da pilha é sua eficiência; a alocação de memória na pilha é incrivelmente rápida porque envolve o movimento de apenas um ponteiro. No entanto, essa eficiência tem limitações. O tamanho da pilha é fixo e, se um programa tentar usar mais espaço que o disponível, pode resultar em um erro conhecido como estouro de pilha.

Em contraste temos o heap, uma área de memória utilizada para armazenamento dinâmico; diferente da pilha, a memória nele deve ser alocada e desalocada explicitamente. Em linguagens como C#, isso geralmente é feito por comandos como new para alocar memória (conforme o tópico 2.5). Nessa área

são armazenados objetos e dados dinâmicos, como arrays, cujo tamanho é determinado em tempo de execução.

O heap não tem a estrutura Lifo da pilha; em vez disso, a memória pode ser alocada e desalocada em qualquer sequência, o que ocasionalmente pode fragmentar a memória. Isso significa que pode haver pequenos espaços não utilizados espalhados pelo heap; além disso, a alocação de memória nele é geralmente mais lenta que na pilha. Uma particularidade do heap em muitas linguagens modernas é que ele é gerenciado por um sistema chamado coletor de lixo, que procura e libera memória que não é mais referenciada pelo programa, mas isso pode causar pausas ocasionais na execução.

Em resumo, enquanto a pilha é uma área de memória de gerenciamento mais direto e automático – frequentemente usada para variáveis de curta duração –, o heap é utilizado para gerenciar objetos e dados com vida útil mais longa ou um tamanho conhecido somente em tempo de execução. Ambas são essenciais para uma execução eficiente e eficaz.

Quadro 12 – Comparativo entre estruturas e classes

Característica	Estruturas	Classes
Alocação	Pilha (stack)	Heap
Coleta de lixo	Não está sujeita	Está sujeita
Herança	Suporta apenas herança de interfaces	Suporta herança completa
Construtor- padrão	Tem construtor-padrão não modificável	Pode ou não ter
Imutabilidade	Frequentemente projetada para ser imutável	Pode ser mutável ou imutável
Semântica de passagem	Passada por valor (cópia)	Passada por referência
Eficiência em dados (pequenos)	Geralmente mais eficiente em pequenos volumes de dados	Potencialmente menos eficiente
Eficiência em dados (grandes)	Pode ser menos eficiente (cópia de dados)	Mais eficiente (passagem de referência) para grandes volumes de dados
Uso recomendado	Tipos leves que representam um único valor	Tipos mais complexos ou que necessitam de herança, polimorfismo etc.

É importante notar que as estruturas também têm suas desvantagens e limitações, dentre as quais:

- não suportam herança (exceto herança de interfaces);
- passagem por valor, que pode ser menos eficiente para structs grandes, pois copiar grandes quantidades de dados pode ser mais lento do que copiar uma única referência.

Exemplo: para representar os deuses do Olimpo e suas principais características, podemos criar uma estrutura Deus que representa um deus da mitologia grega com nome, domínio e símbolo (linha 5 à 25 da figura 111). No programa principal (linha 27 à 41), instanciamos três deuses (Zeus, Poseidon e Hades) e exibimos seus detalhes.

```
1.  using System;
2.
3.  namespace MitologiaGrega
4.  {
5.    public struct Deus
6.    {
7.      public string Nome { get; set; }
8.      public string Dominio { get; set; }
9.      public string Simbolo { get; set; }
10.
11.     public Deus(string nome,string dominio,string simbolo)
12.     {
13.       Nome = nome;
14.       Dominio = dominio;
15.       Simbolo = simbolo;
16.     }
17.
18.     public void MostrarDetalhes()
19.     {
20.       Console.WriteLine($"Nome: {Nome}");
21.       Console.WriteLine($"Domínio: {Dominio}");
22.       Console.WriteLine($"Símbolo: {Simbolo}");
23.       Console.WriteLine("-------------------");
24.       }
25.     }
26.
27.     class Program
28.     {
29.       static void Main(string[]args)
30.       {
31.         Deus zeus = new Deus("Zeus", "Deus dos céus e trovão",
"Raio");
32.         Deus poseidon = new Deus("Poseidon", "Deus dos mares",
"Tridente");
33.         Deus hades = new Deus("Hades", "Deus do submundo", "Elmo da
escuridão");
34.
35.         zeus.MostrarDetalhes();
36.         poseidon.MostrarDetalhes();
37.         hades.MostrarDetalhes();
38.
39.         Console.ReadKey();
40.     }
41.   }
42. }
```

Figura 111 – Estrutura Deus: exemplo de declaração e utilização

Para exibir os detalhes de cada deus, usamos o método MostrarDetalhes, afinal uma estrutura pode ter métodos, propriedades, indexadores, eventos e até mesmo definir construtores, assim como classes. É importante destacar que ela não pode ter um construtor explícito sem parâmetros (um que não leva argumentos), e não pode ter um destruidor. Além disso, ao contrário das classes, as structs não permitem inicializar os campos no momento da declaração, a menos que esse campo seja declarado como const ou static.

3.12 Tipo objeto

Object é o tipo raiz na hierarquia de tipos; ou seja, todas as classes, independentemente de sua posição ou complexidade na hierarquia de herança, derivam direta ou indiretamente do tipo Object. Esse tipo raiz é definido no namespace System. Como resultado dessa herança universal, o tipo Object contém um conjunto mínimo de métodos que todos os objetos em C# herdam e podem sobrescrever, incluindo funções como:

- **ToString**: retorna uma representação de string do objeto.
- **GetHashCode**: obtém um código hash para o objeto atual.
- **Equals**: determina se um objeto é igual a outro.

Citamos o GetHashCode pois as funções de hash são fundamentais em diversas áreas da computação. Hash é uma função que converte uma entrada (ou mensagem) em uma cadeia de caracteres de comprimento fixo, que tipicamente parece aleatória. A saída, frequentemente referida como código hash, é única (dentro de limites razoáveis) para a entrada única fornecida. Portanto, pequenas variações na entrada produzirão valores hash drasticamente diferentes. Nesse contexto, o método GetHashCode facilita a rápida recuperação e comparação de objetos em estruturas de dados complexas.

O quadro 13 descreve alguns métodos da classe Object.

Quadro 13 – Classe Object: principais métodos

Método	Descrição
Equals(Object obj)	Determina se o objeto atual é igual ao objeto especificado
Finalize()	Permite que um objeto tente liberar recursos e realizar outras operações de limpeza antes de ser recuperado pelo coletor de lixo
GetHashCode()	Serve como a função de hash padrão e retorna um código hash para o objeto atual
GetType()	Obtém o Type do objeto atual
MemberwiseClone()	Cria uma cópia superficial do objeto atual
ReferenceEquals(Object objA, Object objB)	Determina se as duas referências de objeto especificadas se referem ao mesmo objeto
ToString()	Retorna uma string que representa o objeto atual

Na figura 112 armazenamos instâncias das classes Deidade e Criatura em variáveis do tipo Object (obj1 e obj2). Em seguida, usamos o operador **is** para verificar o tipo real de cada objeto e o cast apropriado para acessar os métodos e propriedades do tipo real – neste caso, a representação ToString personalizada.

```
1.  using System;
2.
3.  public class Deidade
4.  {
5.     public string Nome { get; set; }
6.     public string Dominio { get; set; }
7.
8.     public override string ToString()
9.     {
10.       return $"{Nome}, deus(a) de {Dominio}.";
11.    }
12. }
13.
14. public class Criatura
15. {
16.    public string Nome { get; set; }
17.    public string Caracteristica { get; set; }
18.
19.    public override string ToString()
20.    {
21.       return $"{Nome}, conhecido por {Caracteristica}.";
22.    }
23. }
24.
25. public class Program
26. {
27.    public static void Main()
28.    {
29.       Deidade zeus = new Deidade { Nome = "Zeus", Dominio = "Céu e
trovão" };
30.       Criatura medusa = new Criatura { Nome = "Medusa", Caracteristica
= "transformar pessoas em pedra com seu olhar" };
31.
32.          // Armazenando entidades em variáveis do tipo Object
33.          Object obj1 = zeus;
34.          Object obj2 = medusa;
35.
36.          // Verificando os tipos e exibindo
37.          if (obj1 is Deidade)
38.          {
39.             Console.WriteLine(((Deidade)obj1).ToString());
40.          }
41.          else if (obj1 is Criatura)
42.          {
43.             Console.WriteLine(((Criatura)obj1).ToString());
44.          }
45.
46.          if (obj2 is Deidade)
47.          {
48.             Console.WriteLine(((Deidade)obj2).ToString());
49.          }
50.          else if (obj2 is Criatura)
51.          {
52.             Console.WriteLine(((Criatura)obj2).ToString());
53.          }
54.       }
55.    }
```

Figura 112 – Tipo Object: exemplo de utilização

Quando se trabalha com tipos desconhecidos em tempo de compilação, é comum a necessidade de uma operação chamada **type casting** (ou conversão de tipo) para acessar os membros específicos de um tipo. Casting é o processo de converter um tipo de dado em outro. No contexto de linguagens de programação orientadas a objetos (como C#), muitas vezes nos referimos ao cast como o processo de tratar um objeto de sua classe-base ou de uma interface como se fosse de uma classe derivada ou de uma implementação específica.

Na figura 112 temos variáveis do tipo Object (a classe-base para todos os objetos em C#), mas sabemos que essas variáveis na verdade continham instâncias de classes mais específicas (Deidade ou Criatura). Para acessar os membros específicos dessas classes, precisamos dizer ao compilador "trate esse objeto como se fosse dessa classe específica". Em C#, usamos o casting colocando o tipo desejado entre parênteses antes do objeto, como no exemplo da linha 39, na qual "(Deidade)obj1" está dizendo ao compilador "Trate obj1 como se fosse do tipo Deidade e me dê acesso aos seus membros". Da mesma forma, também foi feito casting nas linhas 43, 48 e 53. Antes de realizar um cast, é aconselhável verificar se o objeto pode ser convertido para o tipo desejado. Nas linhas 37, 41, 46 e 50 usamos o operador **is** para fazer essa verificação.

Se um programador tentar fazer um cast para um tipo incompatível sem verificar, receberá uma exceção em tempo de execução chamada InvalidCastException. Por fim, há também conceitos relacionados, chamados boxing e unboxing, específicos para conversão entre tipos de valor (como int, double) e referências de tipo Object. Boxing é o processo de converter um tipo de valor em Object, e unboxing é o processo de converter de volta.

Fazer a sobrescrita (ou override) do método ToString permite que o programador forneça uma representação em string mais apropriada e legível para um objeto do tipo customizado. Note que nas linhas 8 e 19 da figura 112 sobrescrevemos o método ToString, que é um membro da classe-base System.Object em C#. Por padrão, esse método retorna o nome completo do tipo do objeto. Para muitos tipos customizados (ou seja, tipos que o programador cria), essa representação-padrão pode não ser muito útil ou informativa. Sem a sobrescrita do ToString na linha 8, se tentássemos imprimir um objeto da classe Deidade diretamente (como Console.WriteLine(zeus);), obteríamos algo como o nome completo da classe, que seria algo como Namespace.Deidade, o que não é muito informativo. Ao sobrescrever ToString(), podemos retornar uma string mais significativa, como "Zeus, deus(a) de Céu e trovão", que fornece informações claras sobre o objeto e é mais útil para a maioria dos cenários nos quais o objeto pode ser convertido em uma string (como em logs, exibições em interfaces gráficas, entre outros).

O tipo Object desempenha um papel fundamental no sistema de tipos do C#, fornecendo os meios básicos pelos quais os objetos se comunicam, representam a si mesmos e interagem com o sistema. Isso é essencial para operações como comparação, representação textual e gerenciamento de memória, sendo frequentemente utilizado em cenários onde a informação sobre um tipo não é conhecida em tempo de compilação. Quando uma variável é declarada do tipo Object, ela pode armazenar referências a instâncias de qualquer tipo, permitindo uma grande flexibilidade, embora a custo de perder informações específicas do tipo e possivelmente de desempenho.

```
1.  public class Deus
2.  {
3.      public string Nome { get; set; }
4.      public string Dominio { get; set; }
5.      public override string ToString()
6.      {
7.          return $"{Nome}, deus(a) de {Dominio}.";
8.      }
9.  }
10. public class Monstro
11. {
12.     public string Nome { get; set; }
13.     public string Caracteristica { get; set; }
14.     public override string ToString()
15.     {
16.         return $"{Nome}, monstro conhecido por {Caracteristica}.";
17.     }
18. }
19. public class EventoMitico
20. {
21.     public string Evento { get; set; }
22.     public string Detalhes { get; set; }
23.     public override string ToString()
24.     {
25.         return $"Evento: {Evento}, detalhes: {Detalhes}.";
26.     }
27. }
28. public class CatalogadorMitologico
29. {
30.     private Object[] registros;
31.     public CatalogadorMitologico(Object[] entradas)
32.     {
33.         this.registros = entradas;
34.     }
35.     public void ProcessarRegistros()
36.     {
37.         foreach (Object registro in registros)
38.         {
39.             if (registro is Deus)
40.             {
41.                 Console.WriteLine($"Deus registrado: {registro}");
42.             }
43.             else if (registro is Monstro)
44.             {
45.                 Console.WriteLine($"Monstro registrado: {registro}");
46.             }
47.             else if (registro is EventoMitico)
48.             {
49.                 Console.WriteLine($"Evento mítico registrado: {registro}");
50.             }
51.         }
52.     }
53. }
```

Figura 113 – Tipo Object: declaração das classes (com o CatalogadorMitologico)

Para melhor visualizar o exemplo da figura 113, imagine que estamos catalogando diferentes entidades da mitologia grega: deuses, monstros e eventos míticos. Cada tipo de entidade possui propriedades distintas, e queremos armazená-las em um único registro antes de processá-las conforme seu tipo. Nesse cenário, o tipo Object permite registrar deuses, monstros e eventos míticos em um único array, processando-os posteriormente com base em seu tipo específico. Essa flexibilidade é útil para lidar com diferentes tipos de entidade sob um único sistema de catalogação.

O trecho de código-fonte da figura 114 é a continuação do trecho de código da figura 113. Foi criado um vetor de objetos do tipo Object chamado **entradas** para armazenar deuses, monstros e eventos míticos, que são de tipos diferentes.

Lembrete

Vimos no tópico 3.9 que um vetor somente pode ser declarado para objetos de um tipo (e no caso esse tipo comum é Object).

```
1.  public class Program
2.  {
3.     public static void Main()
4.     {
5.        Object[] entradas =
6.        {
7.           new Deus { Nome = "Zeus", Dominio = "Céu e trovão" },
8.           new Monstro { Nome = "Medusa", Caracteristica = "transformar
pessoas em pedra com seu olhar" },
9.           new EventoMitico { Evento = "Guerra de Troia", Detalhes = "Uma
guerra épica resultante do rapto de Helena." },
10.          new Deus { Nome = "Athena", Dominio = "Sabedoria" },
11.       };
12.
13.       CatalogadorMitologico catalogador = new CatalogadorMitologico(entradas);
14.       catalogador.ProcessarRegistros();
15.    }
16. }
```

Figura 114 – Tipo Object: utilização das classes, em especial do CatalogadorMitologico

O código-fonte resultante da junção da figura 113 com a 114 começa a iterar pelo array entradas e, para cada objeto, ele verifica seu tipo. Se o objeto for da classe Deus, imprimirá uma mensagem indicando que é um deus e exibirá a representação em string daquele deus (graças ao método ToString sobrescrito na classe Deus). Ele seguirá um padrão similar para monstros e eventos míticos conforme a figura 115, que mostra o resultado na tela do console.

```
Deus registrado: Zeus, deus(a) de Céu e trovão.

Monstro registrado: Medusa, monstro conhecido por transformar pessoas em
pedra com seu olhar.

Evento mítico registrado: Evento: Guerra de Troia, detalhes: Uma guerra
épica resultante do rapto de Helena.

Deus registrado: Athena, deus(a) de Sabedoria.
```

Figura 115 – Tipo Object: saída na tela do console após a execução

Em suma, o tipo Object em C# serve como ancestral comum de todos os outros tipos e fornece a base para interação entre os objetos em todo o ambiente .NET. É tanto um reflexo da natureza orientada a objetos do C# quanto uma ferramenta prática para desenvolvedores trabalharem em cenários polimórficos.

3.13 Entrada e saída

Entrada e saída (E/S), muitas vezes referidas como I/O (do inglês *input/output*), são uma parte fundamental de qualquer linguagem de programação, pois determinam como os programas interagem com fontes de dados externas, como arquivos, bancos de dados, redes ou entrada-padrão/saída-padrão de um sistema. Em C#, entrada e saída são tratadas principalmente através da Biblioteca de Classes Base (BCL) do .NET Framework (abordada com mais abrangência no tópico 4.1). Essa biblioteca fornece uma ampla gama de classes e métodos que permitem aos desenvolvedores ler e escrever dados de e para várias fontes.

Há dois grandes domínios de entrada e saída: I/O de arquivos e I/O de streams. A operação de arquivos é facilitada por uma série de classes que permitem ler e escrever em arquivos no sistema. Essas operações podem ser tão simples quanto ler ou escrever todo o conteúdo de um arquivo de texto, ou tão complexas quanto manipular arquivos binários ou serializar e desserializar objetos. Para essas operações, arquivo é uma entidade que pode ser aberta, lida, escrita e fechada, por isso cuidados devem ser tomados para tratar exceções, já que operações de E/S são propensas a erro, como tentar abrir um arquivo que não existe ou tentar escrever em um dispositivo de armazenamento cheio.

Os dois principais tipos de arquivo são **textuais** e **binários**. Os primeiros contêm texto; se abrirmos um deles com um editor de texto, seremos capazes de ler e compreender seu conteúdo. Arquivos de texto são frequentemente usados para armazenar configurações, logs, documentos e outros dados que podem ser representados como texto. Já os arquivos binários contêm dados não necessariamente legíveis como texto – imagens, vídeos e alguns formatos de documento são bons exemplos. Podem conter qualquer sequência de bytes, e sua interpretação depende da especificação do formato do arquivo e do programa que está lendo ou escrevendo esses dados.

O acesso ao sistema de arquivos é, por natureza, mais lento que muitas outras operações em um programa, como fazer cálculos ou acessar a memória. Portanto, quando se trabalha com I/O de arquivos, é crucial ser eficiente. Isso pode significar, por exemplo, ler ou escrever dados em blocos, em vez de

um byte de cada vez, ou minimizar o número de operações de arquivo abrindo um arquivo uma vez, realizando todas as operações necessárias e, em seguida, fechando-o.

O namespace System.IO contém as classes primárias para operações de arquivo. Dentre as principais, estão:

- **File**: oferece métodos estáticos para operações de arquivo, incluindo ler e escrever em arquivos, verificar a existência de um arquivo, excluí-lo, copiá-lo e muitas outras operações.

- **FileInfo**: representa um arquivo no sistema de arquivos. Oferece propriedades para obter dados sobre o arquivo, como seu caminho, tamanho e datas de criação e modificação. Além disso, fornece métodos de instância para operações, como criação, exclusão, movimentação e abertura.

- **Directory e DirectoryInfo**: essas classes são para operações relacionadas a diretórios. Directory fornece métodos estáticos, enquanto DirectoryInfo fornece métodos de instância.

O quadro 14 apresenta os principais métodos da classe File e FileInfo, e o quadro 15, métodos úteis de Directory e DirectoryInfo.

Quadro 14 – Classes File e FileInfo: principais métodos

Método	Descrição
Copy	Copia um arquivo existente para um novo local
Create	Cria ou sobrescreve um arquivo no caminho especificado
Delete	Exclui o arquivo especificado
Exists	Determina se o arquivo especificado existe
ReadAllText	Lê todo o conteúdo de um arquivo de texto em uma string
WriteAllText	Cria um novo arquivo, escreve a string especificada e, em seguida, fecha o arquivo
AppendAllText	Anexa texto a um arquivo existente ou cria o arquivo se ele não existir
Create	Cria um arquivo
Delete	Exclui um arquivo
MoveTo	Move um arquivo especificado para um novo local
Open	Abre um arquivo em um dos modos especificados (por exemplo, leitura e gravação)
Refresh	Atualiza as propriedades do objeto FileInfo para refletir as propriedades reais no sistema de arquivos
CopyTo	Copia um arquivo para um novo caminho

Quadro 15 – Classes Directory e DirectoryInfo: principais métodos

Método	Descrição
CreateDirectory	Cria todos os diretórios no caminho especificado
Delete	Exclui o diretório especificado
Exists	Determina se o diretório especificado existe
GetFiles	Retorna nomes de arquivos em diretório especificado

Método	Descrição
GetDirectories	Retorna o nome dos subdiretórios em um diretório especificado
Move	Move um diretório para um novo local
Create	Cria um diretório
Delete	Exclui um diretório
GetFiles	Retorna uma lista de arquivos em um diretório
GetDirectories	Retorna uma lista de diretórios em um diretório
MoveTo	Move o diretório para um novo local
Refresh	Atualiza as propriedades do objeto DirectoryInfo para refletir as propriedades reais no sistema de arquivos

Na figura 116 invocamos métodos de conveniência da classe File, que tratam automaticamente do processo de abertura e fechamento do arquivo. Na linha 18 usamos File.WriteAllLines para gravar um vetor de strings diretamente em um arquivo. Esse método abre o arquivo, escreve todas as linhas e fecha-o.

```
1.  using System;
2.  using System.IO;
3.
4.  class Program
5.  {
6.     static void Main()
7.     {
8.        string path = "deusesGregos.txt"
9.        string[] deuses =
10.       {
11.          "Zeus - Rei dos deuses e governante do Monte Olimpo.",
12.          "Hera - Rainha dos deuses e esposa de Zeus.",
13.          "Poseidon - Deus dos mares, terremotos e cavalos.",
14.          "Athena - Deusa da sabedoria, da coragem e da guerra.",
15.          "Ares - Deus da guerra."
16.       };
17.    // Escrevendo informações sobre deuses gregos no arquivo usando
File
18.    File.WriteAllLine(path, deuses);
19.    Console.WriteLine("Informações sobre deuses gregos gravadas com
sucesso!");
20.
21.    // Lendo e exibindo informações sobre deuses gregos do arquivo
usando File
22.    Console.WriteLine("\nLendo informações do arquivo:");
23.    string[] linesRead = File.ReadAllLines(path);
24.    foreach (var line in linesRead)
25.    {
26.       Console.WriteLine(line);
27.    }
28  }
29.}
```

Figura 116 – I/O de arquivo: exemplo de utilização

Posteriormente, para ler todas as linhas do arquivo e exibi-las, usamos File.ReadAllLines (linha 23). Ele abre o arquivo, lê todas as linhas até o fim e, em seguida, fecha-o. Esses métodos são muito úteis para operações rápidas e diretas em que você queira escrever ou ler todo o conteúdo de uma vez.

Outras duas classes interessantes são Path e DriveInfo; o quadro 16 descreve alguns métodos e propriedades úteis dessas classes. Nas linhas 10 e 11 da figura 117 usamos o método Combine, que serve para combinar duas ou mais strings de caminho de arquivo em um único caminho.

```
1.  using System;
2.  using System.IO;
3.
4.  class Program
5.  {
6.    static void Main()
7.    {
8.      // Caminhos de diretórios e arquivos para o exemplo.
9.      string olimpoPath = "Olimpo";
10.     string deusesFilePath = Path.Combine(olimpoPath, "Deuses.txt");
11.     string titasFilePath = Path.Combine(olimpoPath, "Titas.txt");
12.
13.     // 1. Usando Directory.CreateDirectory para criar um diretório
        chamado "Olimpo"
14.     if (!Directory.Exists(olimpoPath))
15.     {
16.       Directory.CreateDirectory(olimpoPath);
17.       Console.WriteLine("O Monte Olimpo foi criado!");
18.     }
19.
20.     // 2. Usando File.WriteAllText para criar e escrever em um
        arquivo chamado "Deuses.txt"
21.     File.WriteAllText("deusesFilePath, "Zeus, Hera, Poseidon, Athena");
22.     Console.WriteLine("Os principais Deuses foram registrados no
        Monte Olimpo.");
23.
24.     // 3. Usando File.WriteAllText para criar e escrever em um
        arquivo chamado "Titas.txt"
25.     File.WriteAllText("titasFilePath, "Cronos, Réia, Atlas");
26.     Console.WriteLine("Os Titas foram registrados.");
27.
28.     // 4. Usando DirectoryInfo para listar todos os arquivos no
        diretório "Olimpo"
29.     DirectoryInfo olimpoDirectory = new DirectoryInfo(olimpoPath);
30.     Console.WriteLine("\nArquivos no Monte Olimpo:");
31.     foreach (var file in olimpoDirectory.GetFiles())
32.     {
33.       Console.WriteLine(file.Name);
34.     }
35.
36.     Console.ReadKey();
37.   }
38. }
```

Figura 117 – Trabalhando com arquivos utilizando diversas classes de Sytem.IO

A vantagem de usar Path.Combine em vez de simplesmente concatenar strings para criar caminhos é que ele gerencia de maneira adequada os separadores de diretório (como \ no Windows e/em sistemas Unix-like) independentemente do sistema operacional em que o código está sendo executado. Isso

ajuda a garantir que os caminhos sejam formados corretamente e evita problemas potenciais com caminhos mal formatados.

Na linha 16 da figura 117 usamos o método Directory.CreateDirectory para criar um diretório chamado Olimpo, que cria o diretório somente se ele não existir, evitando potenciais erros. Para adicionar uma lista de deuses a um arquivo chamado Deuses.txt dentro do diretório Olimpo, empregamos o método File. WriteAllText (linha 21), que cria o arquivo (ou sobrescreve se ele já existir) e escreve a string fornecida; assim, "Zeus, Hera, Poseidon, Athena" são registrados no arquivo.

Depois, na linha 25, usando o mesmo método File.WriteAllText, gravamos uma lista de titãs em um arquivo separado chamado Titas.txt. Como no passo anterior, o método cria ou sobrescreve o arquivo com os nomes "Cronos, Reia, Atlas". Por fim, para ver todos os registros no Monte Olimpo, utilizamos na linha 29 a classe DirectoryInfo para obter uma representação do diretório Olimpo. Em seguida, chamamos o método GetFiles para recuperar todos os arquivos dentro desse diretório e listá-los na tela.

Quadro 16 – Path e DriveInfo: exemplos de métodos e propriedades

Classe	Método	Descrição
Path	GetFileName(string path)	Retorna o nome do arquivo (e sua extensão) de um caminho especificado
Path	GetDirectoryName(string path)	Retorna o diretório de um caminho especificado
Path	GetExtension(string path)	Retorna a extensão de um caminho especificado
Path	GetFullPath(string path)	Retorna o caminho absoluto para um caminho especificado
Path	Combine(string path1, string path2)	Combina dois caminhos de string
DriveInfo	DriveType (propriedade)	Obtém o tipo de unidade, como CD-ROM, removível, fixo etc.
DriveInfo	IsReady (propriedade)	Obtém um valor indicando se a unidade está pronta
DriveInfo	Name (propriedade)	Obtém o nome da unidade, por exemplo "C:"

Apresentaremos agora o conceito de **stream**, uma abstração de uma sequência de bytes. Pode-se pensar nele como um fluxo contínuo de dados que pode ser lido ou escrito. Essa abstração é poderosa porque permite que os desenvolvedores tratem diversas fontes de dados – sejam arquivos, redes, memória etc. – de maneira consistente. A BCL (Base Class Library, que será apresentada no tópico 4.1) fornece diversas classes para trabalhar com streams, e está principalmente no namespace System.IO.

Através dessas classes, podemos ler de ou escrever em fluxos de dados, bem como encadear múltiplos fluxos juntos para realizar operações complexas, como comprimir dados ou criptografia. A principal classe que representa um fluxo contínuo de dados em .NET é a classe Stream, da qual todas as outras classes de stream derivam. Essa classe abstrata define métodos básicos, como Read, Write, Flush, Seek, entre outros, fundamentais para manipular dados.

Em sua essência, streams leem e escrevem dados. O método Read é usado para ler dados de um fluxo e colocá-lo em um vetor de bytes, enquanto Write é usado para escrever dados de um vetor de bytes para um fluxo de dados. Streams permitem que o programador consiga manipular os dados, movendo as

posições para diferentes locais dentro do fluxo usando o método Seek. Isso é útil especialmente quando é necessário voltar a um ponto anterior no stream e ler ou escrever novamente.

Existem três tipos básicos de manipulação de fluxos contínuos de dados: MemoryStream, NetworkStream e BufferedStream. O primeiro representa um stream que utiliza a **memória** como armazenamento, sendo útil quando precisamos armazenar temporariamente dados em memória antes de movê-los para um destino diferente. O segundo é usado para ler e escrever **em conexões de rede**, comumente usado em combinação com classes como TcpClient e TcpListener para comunicações baseadas em *transmission control protocol* (TCP). O último dá suporte para buffering, o que pode melhorar a performance de operações de I/O ao ler ou escrever grandes quantidades de **dados em blocos** em vez de byte a byte. Essa técnica consiste em usar uma área temporária de armazenamento, chamada **buffer**, para armazenar temporariamente os dados antes que eles sejam enviados para o destino final ou processados.

O método Flush garante que todos os dados em um buffer sejam escritos em um fluxo de dados subjacente. Isso é crucial para garantir a integridade dos dados, especialmente se estivermos escrevendo em um stream mas os dados ainda não foram completamente enviados.

Também é importante fechar ou descartar um stream quando ele não é mais necessário, liberando recursos associados. Em muitos casos, o padrão using garante que os recursos sejam liberados corretamente. No tópico 3.2, introduzimos o conceito de que a diretiva using permite o uso de tipos em um namespace sem precisar qualificar o uso do tipo pelo seu namespace completo. Por exemplo, se inserirmos "using System" no início do código, podemos escrever simplesmente Console.WriteLine("oi") em vez de System.Console.WriteLine("oi").

Adicionalmente, a palavra-chave using cumpre outra função importante relacionada a recursos externos: garantir a um objeto que implementa a interface IDisposable ter seu método Dispose chamado quando não for mais necessário, permitindo que quaisquer recursos sejam liberados de forma adequada. A interface IDisposable é uma parte central do .NET para manejar recursos não gerenciados, como descritores de arquivo, conexões de banco de dados ou qualquer outro recurso que precisa ser explicitamente liberado quando não é mais necessário. Ela declara um único método, Dispose, que quando implementado por uma classe fornece a lógica para liberar esses recursos.

Quando uma classe no .NET, como muitas no System.IO, trabalha com recursos não gerenciados, é uma prática recomendada implementar IDisposable para garantir que esses recursos sejam limpos corretamente. Isso ajuda a prevenir vazamentos de recursos e outros problemas potenciais. A classe FileSystemWatcher é um exemplo de classe em System.IO que implementa a interface IDisposable.

```
1.  using System;
2.  using System.IO;
3.
4.  public class ServicoDeMitologia
5.  {
6.     public void Main()
7.     {
8.        // Criar uma história da mitologia grega
9.        string historia = "Na mitologia grega, Zeus é o deus do céu e
governante do Olimpo.";
10.
11.       // Converter a história em bytes
12.       byte[] bytesDaHistoria = new byte[historia.Length * sizeof(char)];
13.       System.Buffer.BlockCopy(historia.ToCharArray(), 0,bytesDaHistoria,
0,bytesDaHistoria.Length);
14.
15.       // Usar MemoryStream para armazenar a história em bytes
16.       using (MemoryStream streamDaMemoria = new MemoryStream
(bytesDaHistoria))
17.       {
18.       // Criar um arquivo chamado "historia.txt" e usar BufferedStream
para escrever nele
19.       using (FileStream streamDoArquivo = new FileStream ("historia.txt",
FileMode.Create, FileAccess.Write))
20.       using (BufferedStream streamBufferizado = new
BufferedStream(streamDoArquivo, 4096))
21.       {
22.          byte[] buffer = new byte[4096];
23.          int bytesLidos;
24.
25.       // Ler a história do MemoryStream e escrever no BufferedStream
26.       while ((bytesLidos = streamDaMemoria.Read(buffer, 0, buffer.
Length)) > 0))
27.       {
28.          streamBufferizado.Write(buffer, 0, bytesLidos);
29.       }
30.
31.       streamBufferizado.Flush();
32.       Console.WriteLine("História salva no arquivo!");
33.       }
34.    }
35. }
36.
37. public static void Main()
38. {
39.    ServicoDeMitologia servico = new ServicoDeMitologia();
40.    servico.Iniciar();
41.    }
42. }
```

Figura 118 – Streams: exemplo de utilização dos três tipos básicos

Iniciamos criando uma história sobre Zeus e a convertemos em bytes utilizando o método Buffer. BlockCopy, que copia a representação em bytes dos caracteres da história para um vetor de bytes (linha 13). Depois dessa conversão, a história é armazenada em um MemoryStream, uma classe que

permite manipular bytes em memória como se estivessem em um fluxo de entrada/saída (linha 16). A vantagem do MemoryStream é que ele opera inteiramente na memória, acelerando as operações de leitura e escrita.

Para gravar essa história em um arquivo de forma eficiente, empregamos o BufferedStream (linha 20) com um FileStream (linha 19). O FileStream representa um arquivo no sistema e nos permite ler e escrever nele, enquanto o BufferedStream serve como camada intermediária, armazenando temporariamente os dados em um buffer antes de gravá-los no destino (nesse caso, o arquivo). Isso pode melhorar significativamente a performance, especialmente quando se escreve no arquivo em pequenos incrementos. O quadro 17 fornece uma relação com as principais classes para manipular streams.

Quadro 17 – Principais classes de I/O stream

Classe	Descrição
Stream	Classe-base abstrata para todos os fluxos de I/O
FileStream	Fornece suporte para ler e gravar arquivos
MemoryStream	Usa uma matriz de bytes na memória para E/S
NetworkStream	Facilita a comunicação através de soquetes de rede
BufferedStream	Adiciona buffer a um fluxo existente para melhorar o desempenho
CryptoStream	Fornece criptografia ou descriptografia em um fluxo
GZipStream	Comprime ou descomprime fluxos usando o formato GZip
DeflateStream	Comprime ou descomprime fluxos usando o formato Deflate
StreamReader	Lê caracteres de um fluxo de bytes de forma eficiente
StreamWriter	Escreve caracteres em um fluxo de bytes de forma eficiente
BinaryReader	Lê tipos de dados primitivos em um fluxo binário
BinaryWriter	Escreve tipos de dados primitivos em um fluxo binário
StringReader	Lê strings de um objeto TextReader
StringWriter	Escreve strings em um objeto TextWriter

Na linha 31 da figura 118, o método streamBufferizado.Flush é invocado para garantir que todos os dados que ainda possam estar no buffer sejam efetivamente escritos no arquivo. Isso é importante pois o BufferedStream pode conter dados ainda não transferidos para o arquivo se o buffer não estiver cheio. Finalmente, ao término das operações, os fluxos de dados são fechados automaticamente graças ao padrão using. Esse padrão garante que o método Dispose de cada stream seja chamado (linhas 16, 19 e 20), liberando recursos e garantindo que todas as operações pendentes sejam concluídas.

As principais classes também oferecem uma gama de propriedades e métodos importantes. O quadro 18 relaciona, de modo não exaustivo, alguns deles. Embora as listas não incluam os métodos construtores, eles também são relevantes, e suas assinaturas devem ser conhecidas para uma correta utilização. Por exemplo, na linha 17 da figura 118, o construtor FileStream pressupõe três parâmetros: identificação do arquivo, modo de leitura e forma de acesso.

Quadro 18 – Principais métodos de I/O stream

Métodos	Descrição
Stream.Read(byte[] buffer, int offset, int count)	Lê bytes do fluxo para o buffer especificado
Stream.Write(byte[] buffer, int offset, int count)	Escreve bytes do buffer no fluxo
Stream.Flush()	Limpa todos os buffers para o fluxo subjacente e o faz escrever todos os dados pendentes
Stream.Close()	Fecha o fluxo atual e libera todos os recursos associados a ele
Stream.Seek(long offset, SeekOrigin origin)	Move o ponteiro de leitura/gravação para a posição especificada
Stream.SetLength(long value)	Define o comprimento do fluxo para o valor especificado
FileStream.Lock(long position, long length)	Bloqueia uma região específica do arquivo
FileStream.Unlock(long position, long length)	Desbloqueia uma região previamente bloqueada do arquivo
MemoryStream.ToArray()	Retorna a matriz de bytes contendo os dados atuais do fluxo de memória
MemoryStream.GetBuffer()	Obtém a matriz de bytes subjacente do fluxo de memória
BufferedStream.Flush()	Descarrega todos os dados nos buffers para o fluxo subjacente
BufferedStream.ReadByte()	Lê o próximo byte do fluxo como um valor inteiro
StreamReader.Read()	Lê o próximo caractere do fluxo como um inteiro Unicode
StreamWriter.WriteLine(string value)	Escreve uma string seguida por uma nova linha no fluxo
StreamWriter.AutoFlush	Obtém ou define um valor que indica se o StreamWriter descarrega automaticamente seu buffer após cada chamada de escrita
BinaryReader.ReadInt32()	Lê um valor inteiro de quatro bytes do fluxo binário
BinaryWriter.Write(string value)	Escreve uma cadeia de caracteres no fluxo binário
StringReader.ReadLine()	Lê uma linha de texto do objeto StringReader
StringWriter.ToString()	Retorna o conteúdo atual do objeto StringWriter como uma string

O código-fonte da figura 119 explora os métodos de entrada e saída de fluxo de dados presentes no namespace System.IO, especificamente para lidar com a escrita e leitura de dados em formato binário.

```
1.  using System;
2.  using System.IO;
3.
4.  public class Program
5.  {
6.     public static void Main()
7.     {
8.        string filename = "mitologia.dat";
9.
10.       // Escreve os detalhes de alguns deuses gregos no arquivo usando
FileStream e BinaryWriter.
11.       using (FileStream fs = new FileStream(filename, FileMode.Create))
12.       using (BinaryWriter writer = new BinaryWriter(fs))
13.       {
14.       writer.Write("Zeus");
15.       writer.Write("Rei dos deuses e governante do Monte Olimpo.");
16.       writer.Write("Hera");
17.       writer.Write("Rainha dos deuses e esposa de Zeus.");
18.       writer.Write("Athena");
19.       writer.Write("Deusa da sabedoria, coragem e inspiração.");
20.       }
21.
22.          Console.WriteLine("Detalhes dos deuses gregos gravados no
arquivo.");
23.
24.       // Lê os detalhes dos deuses gregos do arquivo usando FileStream
e BinaryReader e os exibe.
25.    Console.WriteLine("\nLendo os detalhes do arquivo:");
26.       using (FileStream fs = new FileStream(filename, FileMode.Open))
27.       using (BinaryReader reader = new BinaryReader(fs))
28.       {
29.          while(fs.Position < fs.Length)
30.          {
31.             string nomeDeus = reader.ReadString();
32.             string descricao = reader.ReadString();
33.             Console.WriteLine($"{nomeDeus}: {descricao}");
34.          }
35.       }
36.    }
37. }
```

Figura 119 – Streams: exemplo de alguns métodos

Na linha 11 utilizamos a classe FileStream para criar uma conexão direta com um arquivo denominado mitologia.dat, que permite acesso de baixo nível ao sistema de arquivos, o que significa que temos um controle mais granular sobre como os dados são lidos e escritos. Em muitas situações de programação, precisamos de um método eficiente para salvar e recuperar dados, e o FileStream serve perfeitamente a esse propósito.

Para escrever dados no arquivo, usamos o BinaryWriter (linha 12), que nos permite gravar dados primitivos em formato binário. No contexto do nosso programa, ele foi usado para registrar informações sobre algumas figuras mitológicas gregas – como Zeus, Hera e Athena – e suas respectivas descrições. Optar por um formato binário tem suas vantagens; é frequentemente mais compacto e mais rápido em

operações de E/S, embora não seja humanamente legível como arquivo de texto. Em contrapartida, para ler esses dados binários gravados, utilizamos o BinaryReader (linha 27), que funciona como o inverso do BinaryWriter, lendo os dados primitivos gravados anteriormente e transformando-os de volta em uma forma utilizável pelo programa. No nosso código, o BinaryReader recupera e exibe informações previamente armazenadas sobre as deidades gregas (note também que a palavra-chave using está presente em diversas linhas para garantir o Dispose).

Exemplos de classes em System.IO que implementam IDisposable incluem: FileStream, StreamReader, StreamWriter, StringReader, StringWriter, BinaryReader, BinaryWriter. Em resumo, o código-fonte fornece um exemplo de como classes e métodos do namespace System.IO podem ser empregados para manipular dados em formato binário, ilustrado pelo armazenamento e recuperação de detalhes sobre deidades da mitologia grega.

Outro tema relevante é a **I/O assíncrona**, que se refere a operações de entrada/saída (I/O) realizadas sem bloquear a execução do programa. Dada a natureza potencialmente demorada das operações de E/S, o C# oferece mecanismos para essas operações de forma assíncrona; ou seja, podemos iniciar uma operação de E/S e, em vez de esperar que ela termine, seu programa pode executar outras tarefas. Quando a operação de I/O estiver concluída, seu programa será notificado e poderá processar os resultados. Essas operações são comuns quando trabalhamos com arquivos, redes ou qualquer outra operação que possa levar tempo significativo e não queremos bloquear a thread principal (ou qualquer outra thread) enquanto esperamos a operação concluir (abordaremos esse tema com mais detalhes no tópico 7.6).

O .NET Framework e o .NET Core (e o .NET 5 e versões posteriores) fornecem *application programming interfaces* (APIs) para I/O assíncrona. A partir do C# 5, com a introdução das palavras-chave **async** e **await**, tornou-se ainda mais fácil escrever código-fonte de forma limpa e legível. Dadas as inúmeras variáveis envolvidas na manipulação de arquivos (por exemplo, permissões, espaço em disco, arquivos bloqueados), é vital se preparar para tratar exceções. A IOException é uma das exceções mais comuns, mas há outras – como FileNotFoundException e UnauthorizedAccessException – às quais os desenvolvedores também devem se atentar.

O quadro 19 descreve as exceções mais comuns associadas ao namespace System.IO (o tratamento das exceções será objeto do tópico 5.1).

Quadro 19 – Exceções do namespace System.IO

Exceção	Descrição
IOException	Classe-base para a maioria das exceções relacionadas ao I/O (indica um erro de E/S)
DirectoryNotFoundException	Lançada quando há tentativas de acessar um diretório que não existe
DriveNotFoundException	Lançada quando há tentativas de acessar um drive ou disco não disponível
EndOfStreamException	Lançada ao tentar ler além do final de um stream
FileLoadException	Lançada quando um arquivo é encontrado, mas não pode ser carregado
FileNotFoundException	Lançada ao tentar acessar um arquivo que não existe no sistema

Enquanto o namespace System.IO é amplamente reconhecido por hospedar várias classes fundamentais para operações de I/O, não é o único local onde podemos encontrar funcionalidades relacionadas a entrada/saída. Diversas outras partes do framework .NET fornecem mecanismos para interação de I/O. Por exemplo, o namespace System.Net.Sockets é essencial para comunicações de rede. Dentro dele encontramos classes que permitem a comunicação utilizando sockets, facilitando o envio e recebimento de dados pela rede.

Sockets são a base de muitas formas de comunicação na internet, incluindo *hypertext transfer protocol* (HTTP), *file transfer protocol* (FTP) e muitos outros. Eles podem ser usados para comunicação tanto em redes locais quanto em redes de longa distância. Uma classe notável nesse contexto é a Socket, que estabelece conexões de rede e gerencia a transmissão de dados. Outro recurso valioso, especialmente no contexto da web, é a classe WebClient, que está sob o domínio do namespace System.Net. Ela é projetada especificamente para **enviar e recuperar informações de recursos da web**, simplificando a comunicação com serviços digitais.

No cenário de **bancos de dados**, particularmente com o SQL Server, o namespace System.Data.SqlClient é uma ferramenta crucial. Ele abriga classes como SqlConnection e SqlCommand, que juntas possibilitam uma ampla gama de operações de I/O com bancos de dados. Para quem trabalha com desenvolvimento de **interfaces gráficas** em Windows Forms, a classe System.Windows.Forms.TextBox é um exemplo de componente que facilita a entrada e saída de texto. Além disso, embora não se enquadre nas operações de E/S no sentido mais tradicional, o System.Text.StringBuilder (abordado no tópico 3.5) desempenha um papel relevante na manipulação de strings, que é em si uma forma de operar I/O, já que envolve a manipulação de uma sequência de caracteres.

Por fim, temos a classe System.Console, amplamente utilizada para operações básicas de E/S em aplicativos de console. Essa classe proporciona uma interação direta com o usuário por texto, permitindo ler e escrever na janela do console. Em resumo, o .NET oferece uma vasta gama de funcionalidades para operações de I/O, espalhadas por vários namespaces e classes que vão muito além do tradicional System.IO.

4 RELAÇÃO ENTRE C# E .NET

C# é uma linguagem de programação moderna (Albahari, 2021), orientada a objetos, desenvolvida pela Microsoft como parte de sua iniciativa .NET e projetada para ser uma plataforma multilinguagem (Troelsen; Japikse, 2021). Isso significa que várias linguagens de programação podem ser usadas para desenvolver aplicações para a plataforma .NET, dentre as quais:

- Visual Basic .NET (VB.NET)
- F#
- C++/CLI
- IronPython
- IronRuby
- ASP.NET

Porém a relação entre C# e .NET é intrínseca, já que ambas **foram concebidas para ser a linguagem principal** dessa plataforma. A plataforma .NET, por sua vez, é um framework (ou arcabouço) de desenvolvimento que proporciona um vasto conjunto de bibliotecas e ferramentas para auxiliar os desenvolvedores a criar e executar aplicações para Windows, web, mobile, entre outros (Freeman, 2019).

A palavra framework significa um conjunto coerente e reutilizável de código e conceitos projetados para fornecer uma **fundação** sobre a qual os desenvolvedores podem construir aplicações específicas. Ele define uma estrutura (ou esqueleto, arcabouço) para a aplicação, estabelecendo certas regras e convenções, que ajuda a determinar a arquitetura do software e pode incluir código predefinido, ferramentas e bibliotecas que os desenvolvedores podem estender ou personalizar conforme necessário.

Ao usar um framework, os desenvolvedores frequentemente se beneficiam de padrões de projeto estabelecidos, reduzindo a quantidade de decisões de baixo nível que precisam ser tomadas. Além disso, muitas vezes eles incorporam melhores práticas e padrões da indústria, promovendo a criação de um software mais robusto e eficiente. Exemplo notável é o próprio .NET, mas há outros para Python (como o Django) e para Ruby (como o Ruby on Rails).

É importante esclarecer que o termo biblioteca é uma coleção de funções, classes e métodos que podem ser utilizados por outros programas. Ao contrário de um framework – que dita uma estrutura e um modo específico de fazer as coisas –, biblioteca é como uma ferramenta ou recurso que os desenvolvedores podem escolher usar conforme necessário, sem impor uma estrutura específica. Por exemplo, se um desenvolvedor precisa de funcionalidades para manipular datas e horas, ele pode usar uma biblioteca especializada nisso, sem que essa biblioteca determine como o restante do programa deve ser estruturado ou operar.

Bibliotecas são criadas para ser reutilizáveis, e frequentemente se concentram em resolver problemas ou tarefas específicas, como manipulação de imagens, conexões de rede ou operações matemáticas. A grande vantagem do .NET é permitir que várias linguagens de programação sejam usadas em conjunto, embora o C# seja frequentemente a mais associada e reconhecida.

Em suma, enquanto um framework fornece uma estrutura e conjunto de regras para desenvolver aplicações, estabelecendo uma abordagem ou arquitetura específica, uma biblioteca é um conjunto de funcionalidades que os desenvolvedores podem integrar a seus programas conforme necessário, sem uma imposição sobre a arquitetura geral da aplicação.

A biblioteca de classes do .NET Framework (ou .NET Core/.NET 5+ em versões mais recentes) fornece uma vasta gama de funcionalidades e serviços que os desenvolvedores podem usar. Ao programar em C#, os desenvolvedores em geral usam intensamente essas bibliotecas, evitando a necessidade de reinventar a roda e acelerando o desenvolvimento.

Em resumo, C# e .NET estão profundamente entrelaçados. C# foi criado para ser a linguagem principal para o .NET, que fornece ferramentas e serviços que tornam as aplicações escritas em C# robustas, seguras e eficientes. A combinação dos dois oferece uma plataforma poderosa para desenvolver software.

4.1 CLR e BCL

Um código escrito em C# é compilado para uma linguagem intermediária chamada Common Intermediate Language (CIL). Em vez de ser diretamente transformado em código de máquina específico para um sistema operacional ou hardware, CIL é uma abstração interpretada pela **máquina virtual .NET** (conhecida como CLR, ou Common Language Runtime). Compilado o código C# para CIL, ele pode ser executado em qualquer sistema onde o CLR esteja presente, garantindo a portabilidade do .NET.

Portanto, ao discutir CLR, nos aprofundamos na maquinaria interna que faz o .NET funcionar. O CLR atua como essa máquina virtual e é responsável por carregar e executar o código CIL (Richter, 2012). Ao executarmos um programa .NET, ele compila just in time (JIT) o CIL para o código de máquina específico da arquitetura em que o programa está sendo executado (processo feito em tempo de execução). O compilador JIT traduz o CIL em instruções que o sistema operacional e o hardware do computador podem entender – ou seja, o mesmo código binário .NET pode ser executado em diferentes sistemas operacionais e hardwares, desde que tenham uma implementação do CLR.

Além de gerenciar a execução do código, o CLR fornece vários outros serviços importantes, como o gerenciamento de memória através do coletor de lixo. Em muitas linguagens de programação, os desenvolvedores devem alocar e desalocar manualmente a memória. Com o CLR e o .NET, a maior parte desse trabalho é automatizada: o coletor de lixo monitora os objetos em memória, determina quais não são mais necessários e recupera a memória que eles ocupavam, reduzindo erros comuns, como vazamentos de memória.

O CLR também oferece suporte a funcionalidades relacionadas à segurança, como a verificação de tipos e a execução de código em ambientes restritos (sandboxing). Também fornece serviços para facilitar a interoperabilidade entre códigos gerenciados (.NET) e não gerenciados (como APIs escritas em C ou C++). A plataforma .NET também inclui uma biblioteca extensa de classes, conhecida como Framework Class Library (FCL). Embora a FCL não faça parte do CLR em si, ela funciona com ele para fornecer uma ampla variedade de funcionalidades, desde operações básicas de entrada e saída até serviços web avançados.

Em resumo, CLR é a base sobre a qual os programas .NET são executados. Ele não apenas facilita a execução de códigos escritos em várias linguagens, mas também oferece serviços essenciais que tornam o desenvolvimento mais eficiente e seguro. A existência do CLR e a abstração que ele fornece são fundamentais para a promessa do .NET de permitir que os desenvolvedores escrevam uma vez e executem em qualquer lugar.

A Base Class Library (BCL) é uma componente vital da plataforma .NET, da qual o C# é uma das linguagens principais. Basicamente, BCL é uma **coleção extensiva de classes, interfaces e estruturas** que fornecem funcionalidades predefinidas para tarefas comuns de programação. Podemos pensá-la como um conjunto de ferramentas e utilitários que os desenvolvedores podem aproveitar sem precisar "reinventar a roda" para tarefas básicas, como manipulação de strings, acesso a arquivos, operações de rede, coleções e muitos outros.

Como plataforma, o .NET foi projetado para permitir o desenvolvimento e a execução de aplicações de maneira mais simplificada e eficaz. Para atingir esse objetivo, possui duas componentes principais: CLR e BCL. A primeira, como já discutimos, é responsável por executar programas .NET, compilar just in time, gerenciar memória e outros serviços. Mas, para um desenvolvedor, ter apenas o CLR não é suficiente para criar aplicações ricas e completas de maneira eficiente; e aqui a BCL entra em cena, pois fornece instrumentos que os desenvolvedores utilizam para construir aplicações.

Por exemplo, se um desenvolvedor quiser manipular strings, ele não precisará criar suas próprias funções ou classes para realizar tarefas comuns, pois a BCL já fornece uma classe String com inúmeros métodos úteis. Se ele quiser trabalhar com coleções de objetos (como veremos no tópico 5.4), a BCL fornece classes como List, Dictionary e Queue. Ao lidar com arquivos, existem classes na BCL que facilitam a leitura, escrita e manipulação de arquivos (como vimos no tópico 3.13) e assim por diante, para várias tarefas de programação.

Pode-se dizer que a BCL e o CLR são os dois pilares que sustentam a experiência de desenvolvimento no .NET. O CLR garante que o código seja executado de maneira eficiente e segura, enquanto a BCL fornece os recursos e ferramentas que os desenvolvedores usam para construir aplicações. Ambos trabalham juntos a fim de fazer da plataforma .NET um ambiente poderoso, eficiente e produtivo para o desenvolvimento de software.

Quadro 20 – BCL: classes e namespaces

Classe/namespace	Descrição
System.Object	Base para todas as classes, fornece métodos fundamentais
System.String	Representa e manipula sequências de caracteres
System.Array	Classe-base para todos os vetores
System.Int32, System.Double, System Boolean	Representam tipos primitivos e suas operações
System.Collections.Generic.List<T>	Estrutura de dados para lista de objetos
System.Collections.Generic. Dictionary<TKey, TValue>	Estrutura de dados para mapear chaves a valores
System.IO.File & System.IO.Directory	Manipulam arquivos e diretórios
System.IO.Stream & System.IO.FileStream & System.IO.TextWriter	Classes relacionadas à entrada/saída de dados, como leitura e escrita em arquivos
System.Net.Http.HttpClient	Usado para enviar e receber solicitações HTTP
System.Linq.Enumerable	Fornece capacidades de consulta para coleções com LINQ
System.Threading.Tasks.Task	Representa operações assíncronas
System.DateTime & System.TimeSpan	Representa pontos no tempo e durações
System.Exception	Base para todas as exceções em .NET
System.Data.DataTable	Representa uma tabela de dados na memória
System.Xml.XmlDocument	Fornece métodos para trabalhar com XML
System.Diagnostics.Debug	Ferramentas para depurar código
System.Globalization.CultureInfo	Fornece informações sobre culturas específicas, como formatos de data e moeda
System.Reflection.Assembly	Permite inspecionar tipos em assemblies e criar instâncias de tipos

O quadro apresenta uma visão geral da utilidade e multiplicidade de aplicações possíveis da BCL. Lembre-se que uma das vantagens da POO é justamente o reúso de código, e a BCL é um manancial de código úteis, já testados e amplamente utilizados por diversos programadores no mundo inteiro. Essas classes e namespaces destacam a vastidão e versatilidade da BCL. Dependendo da aplicação que se está desenvolvendo, muitas outras classes poderiam ser consideradas igualmente importantes. Sua beleza reside na variedade de funcionalidades que oferece, cobrindo quase todos os aspectos da programação.

Também podemos observar no quadro que System é um namespace fundamental, contendo uma vasta coleção de classes, estruturas, interfaces, enumerações e delegados que formam o núcleo da BCL do .NET. Muitas das funcionalidades mais básicas e essenciais da plataforma .NET são definidas dentro do namespace System. Já System.Object é a classe-base para todas as classes no .NET, o que significa que todas as outras classes herdam direta ou indiretamente dela, tornando-a fundamental para o .NET funcionar. Para desenvolvedores que estão começando a estudar C# e .NET, familiarizar-se com os tipos dentro do namespace System é um bom ponto de partida.

4.2 Camadas de aplicação

No desenvolvimento de software, em especial no ambiente do C# e do ecossistema .NET, a forma como estruturamos e organizamos o código não é apenas uma questão de estética ou preferência, mas uma decisão que tem implicações diretas na escalabilidade, modularidade, manutenção e até mesmo na performance das aplicações. Uma das abordagens clássicas é a estruturação por uma arquitetura dividida em **três camadas** distintas: apresentação, lógica de negócios e dados.

Camada de apresentação é a interface com a qual o usuário tem contato direto. Pode ser uma interface gráfica tradicional em um aplicativo desktop, uma página web em um navegador ou até mesmo os endpoints de uma API em serviços mais modernos. Ela é crucial pois define como o usuário percebe e interage com a aplicação. Uma boa camada de apresentação é intuitiva e responsiva às ações do usuário.

Em C#, especificamente no contexto de desenvolvimento de APIs usando ASP.NET Core, endpoints referem-se aos pontos finais de uma API ou serviço web, que definem onde e como as solicitações da API podem ser acessadas. Quando um cliente – como um navegador ou um aplicativo móvel – faz uma solicitação a uma API, ele direciona essa solicitação a um endpoint específico, em seguida executa a lógica correspondente (como recuperar, atualizar, inserir ou excluir dados) e retorna uma resposta ao cliente.

Em aplicações para desktop tradicionais, como as criadas usando Windows Forms ou Windows Presentation Foundation (WPF), C# é a linguagem principal. Para aplicações web, a camada de apresentação muitas vezes é construída usando HTML, CSS e JavaScript, no entanto, com o uso de tecnologias como ASP.NET Core MVC ou Blazor, a lógica do lado do servidor dessa camada é escrita em C#. Blazor, em particular, permite que você escreva a lógica do lado do cliente em C# também, em vez de JavaScript. Em aplicações móveis usando Xamarin, C# é a linguagem principal para criar a interface e a lógica do aplicativo.

Quadro 21 – Exemplo de arquitetura: arquivos organizados por camadas

1. Camada de apresentação (usando WPF)	
DeusView.xaml	Define a interface gráfica para visualizar detalhes de um deus grego
DeusView.xaml.cs	É o código-behind que interage com a interface gráfica
2. Camada de lógica de negócios (domínio)	
ServicoDeDeus.cs	Contém a classe ServicoDeDeus, que define os métodos para buscar e manipular informações relacionadas aos deuses gregos
DeusGrego.cs	Contém a definição do modelo (classe) DeusGrego, que representa um deus grego com propriedades como nome, domínio e símbolo
3. Camada de acesso a dados	
IRepositorioDeDeus.cs	Define a interface IRepositorioDeDeus, que especifica os métodos que um repositório de deuses deve implementar
RepositorioDeDeus.cs	Implementação concreta da interface IRepositorioDeDeus, que contém a lógica para buscar um deus na fonte de dados

O quadro mostra a organização dos arquivos do exemplo em cada uma das três camadas, e a seguir analisaremos todas elas. No WPF, o arquivo .xaml define a interface do usuário com botões, caixas de texto e outros controles. O eXtensible Application Markup Language (XAML) é uma linguagem de marcação desenvolvida pela Microsoft, sua parte visual (ou frontend) que define a estrutura e aparência da interface do usuário, normalmente usando linguagem de marcação (em ASP.NET Web Forms costuma-se combinar HTML e controles de servidor ASP.NET).

```
1.  <Window x:Class="MitologiaGrega.DeusView"
2.        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.        Title="Detalhes do Deus Grego" Height="200" Width="300">
5.     <Grid>
6.        <Label Content="Nome do Deus:" HorizontalAlignment="Left"
VerticalAlignment="Top" Margin="10,10,0,0"/>
7.        <TextBox Name="nomeTextBox" HorizontalAlignment="Left"
VerticalAlignment="Top" Margin="10,30,10,0" Height="25" Width="260"/>
8.        <Button Content="Buscar" HorizontalAlignment="Right"
VerticalAlignment="Top" Margin="0,60,10,0" Width="80" Click="BuscarDeus"/>
9.        <TextBlock Name="detalhesTextBlock" HorizontalAlignment="Left"
VerticalAlignment="Top" Margin="10,100,10,0"/>
10.    </Grid>
11. </Window>
```

Figura 120 – Camada de apresentação: arquivo DeusView.xaml (em XAML)

Quando o usuário executa a aplicação (cujo código está na figura 120), a janela DeusView é exibida; nela o usuário pode digitar o nome de um deus e clicar no botão Buscar. Não se preocupe com a sintaxe, ela não é C#, é XAML. Ao clicar no botão Buscar, o método BuscarDeus (linha 10 da figura 121) é invocado. Ele chama o ServicoDeDeus para buscar detalhes sobre o deus. O arquivo .xaml.cs é o código-behind associado ao arquivo .xaml e contém a lógica para lidar com eventos e interações que ocorrem na interface.

```
1.  public partial class DeusView : Window
2.  {
3.    private ServicoDeDeus servicoDeDeus;
4.
5.    public DeusView()
6.    {
7.      InitializeComponent();
8.      servicoDeDeus = new ServicoDeDeus(new RepositorioDeDeus());
9.    }
10.   private void BuscarDeus(object sender, RoutedEventArgs e)
11.   {
12.     var deus = _servicoDeDeus.BuscarDeusPorNome(nomeTextBox.Text);
13.     if (deus != null)
14.     {
15.       detalhesTextBlock.Text = $"Nome: {deus.Nome}\nDomínio:
{deus.Dominio}\nSímbolo: {deus.Simbolo}";
16.     }
17.     else
18.     {
19.       detalhesTextBlock.Text = "Deus não encontrado.";
20.     }
21.   }
22. }
```

Figura 121 – Camada de apresentação: arquivo DeusView.xaml.cs (em C#)

Código-behind é um termo frequentemente usado no contexto de algumas tecnologias de desenvolvimento de interface do usuário, especialmente em ambientes como ASP.NET Web Forms e WPF em .NET. O conceito separa a representação visual da lógica de interação.

O coração de muitas aplicações reside na **camada de lógica de negócios**. Aqui as regras são definidas, os cálculos são feitos e as operações centrais da aplicação acontecem. Funciona como uma mediadora: recebe informações da camada de apresentação, processa-as conforme as regras estabelecidas e interage com a camada de dados quando necessário. Independente da plataforma ou do tipo de aplicação, o C# é frequentemente usado para codificar a lógica de negócios, especialmente em soluções .NET.

A figura 122 define a classe DeusGrego que será usada na aplicação.

```
1.  public class DeusGrego
2.  {
3.    public string Nome { get; set; }
4.    public string Dominio { get; set; }
5.    public string Simbolo { get; set; }
6.  }
```

Figura 122 – Camada de lógica de negócios: arquivo DeusGrego.cs (em C#)

Na figura 123 o ServicoDeDeus invoca o RepositorioDeDeus para buscar o deus na fonte de dados.

```
1.  public class ServicoDeDeus
2.  {
3.     private IRepositorioDeDeus _repositorioDeDeus;
4.
5.     public ServicoDeDeus(IRepositorioDeDeus repositorio)
6.     {
7.        _repositorioDeDeus = repositorio;
8.     }
9.
10.    public DeusGrego BuscarDeusPorNome(string nome)
11.    {
12.       return _repositorioDeDeus.ObterPorNome(nome);
13.    }
14. }
```

Figura 123 – Camada de lógica de negócios: arquivo ServicoDeDeus.cs (em C#)

Por fim, **camada de dados** é o repositório de informações da aplicação. Seja em bancos de dados relacionais tradicionais, sistemas de arquivos ou soluções mais contemporâneas baseadas em nuvem, essa camada lida com a persistência, recuperação e atualização dos dados. A figura 124 ilustra a interface que define os métodos para interagir com a fonte de dados.

```
1.  public interface IRepositorioDeDeus
2.  {
3.     DeusGrego ObterDeusPorNome(string nome);
4.  }
```

Figura 124 – Camada de dados: arquivo IRepositorioDeDeus.cs (em C#)

De forma complementar, a figura 125 é uma implementação concreta que, para simplificar o exemplo, busca um deus em uma lista fixa. Em uma aplicação real, essa classe acessaria um banco de dados ou outra fonte. Obtido o deus (ou não), a resposta é mostrada na janela através do textblock chamado detalhesTextBlock (linha 9 da figura 120).

```
1.  public class RepositorioDeDeus : IRepositorioDeDeus
2.  {
3.     private List<DeusGrego> _deuses = new List<DeusGrego>
4.     {
5.        new DeusGrego { Nome = "Zeus", Dominio = "Deus dos céus", Simbolo
= "Raio" },
6.        new DeusGrego { Nome = "Athena", Dominio = "Deusa da sabedoria",
Simbolo = "Coruja" },
7.        // ... outros deuses podem ser adicionados aqui.
8.     };
9.
10.    public DeusGrego ObterDeusPorNome(string nome)
11.    {
12.       return _deuses.FirstOrDefault(deus => deus.Nome.Equals(nome,
StringComparison.OrdinalIgnoreCase));
13.    }
14. }
```

Figura 125 – Camada de dados: arquivo RepositorioDeDeus.cs (em C#)

O exemplo traz uma clara separação entre as camadas de lógica de negócios (domínio) e acesso a dados. A classe ServicoDeDeus não sabe como os detalhes do deus são armazenados ou recuperados, apenas que existe algum repositório que pode fornecer essas informações. Por outro lado, o RepositorioDeDeus sabe como obter os detalhes (nesse caso, de uma lista fixa, mas poderia ser de um banco de dados), mas não conhece as regras de negócios ou lógica de alto nível da aplicação.

O C# pode ser utilizado para interagir com bancos de dados usando tecnologias como ADO.NET ou Entity Framework Core. Com essas bibliotecas, podemos escrever código C# para consultar, inserir, atualizar ou deletar informações de bancos de dados como SQL Server, PostgreSQL, MySQL e outros. No entanto, o próprio banco de dados (por exemplo, procedimentos armazenados ou funções definidas pelo usuário) pode usar uma linguagem diferente, como T-SQL para SQL Server ou PL/SQL para Oracle.

Como vimos no exemplo (cujos arquivos foram definidos no quadro 21), a fim de manter a organização em uma aplicação real, muitos desenvolvedores preferem agrupar os arquivos relacionados em pastas específicas, como Views, Services, Models e Repositories. Além disso, em projetos maiores, as camadas de Lógica de Negócios e Acesso a Dados poderiam estar em projetos separados (por exemplo, bibliotecas de classes) para separar responsabilidades e facilitar a manutenção.

Nosso exemplo, organizado em uma arquitetura em três camadas, começou com a representação de um deus grego, descrita no arquivo DeusGrego.cs. Esse arquivo define a estrutura básica de um deus com propriedades como nome, domínio e símbolo. Para interagir com os dados dos deuses, temos o ServicoDeDeus.cs, que se comunica com uma interface de repositório para buscar informações sobre um deus específico. Esse serviço não sabe exatamente de onde esses dados estão vindo, apenas que eles podem ser recuperados através da interface que utiliza. Isso é importante para manter a modularidade e permitir a troca da fonte de dados no futuro sem afetar a lógica central da aplicação.

A interface com que o ServicoDeDeus.cs se comunica é definida em IRepositorioDeDeus.cs e estabelece um contrato sobre quais operações um repositório de deuses deve ser capaz de realizar, garantindo consistência e expectativas claras sobre o comportamento dessa camada de acesso a dados. Finalmente, a implementação concreta dessa interface é encontrada no arquivo RepositorioDeDeus.cs. Atualmente esse repositório obtém seus dados de uma lista predefinida de deuses gregos, mas a beleza desse design é que, se decidíssemos mudar para um banco de dados ou outra fonte de dados no futuro, apenas essa parte do código precisaria ser modificada.

Assim, o RepositorioDeDeus.cs contém a lógica específica de como obter informações de um deus, enquanto o restante da aplicação permanece agnóstico sobre a fonte desses dados. Essa organização não apenas promove uma clara separação de responsabilidades, mas também facilita a manutenção e a expansão da aplicação no futuro.

No contexto das aplicações web desenvolvidas em C#, o padrão Model-View-Controller (MVC) é amplamente reconhecido (Reenskaug, 2003) e segmenta a aplicação em três pilares:

- **Model**: abriga os dados e a lógica de negócios.
- **View**: foca a apresentação e a interação com o usuário.
- **Controller**: age como orquestrador entre o Model e a View.

Esse padrão é intrinsecamente associado ao framework ASP.NET Core MVC da Microsoft, uma das principais ferramentas para desenvolvimento web no universo .NET. Esse padrão de design tem suas origens no final dos anos 1970. Foi inicialmente desenvolvido e utilizado em projetos de software no Xerox PARC, renomado centro de pesquisa em Palo Alto, Califórnia. Trygve Reenskaug é creditado por introduzir o conceito do MVC durante seu tempo no Xerox PARC em 1978-1979, tendo sido originalmente concebido para sistemas de computação pessoal, que na época estavam na vanguarda da tecnologia. Portanto não foi inicialmente projetado especificamente para aplicações web, mas para aplicações desktop. Com a evolução da tecnologia e a ascensão da internet, o MVC foi adaptado e se popularizou para desenvolvimento web devido à sua capacidade de separar responsabilidades, facilitar a manutenção e melhorar a escalabilidade das aplicações.

Não há uma correlação exata entre os componentes do MVC e a arquitetura de três camadas. A distinção às vezes pode ser um pouco nebulosa, pois o MVC é um padrão de design, e a arquitetura de três camadas é uma estrutura de base. A camada de apresentação é parecida com a View no MVC; já a camada de lógica de negócios pode incluir tanto o Controller quanto o Model, dependendo de como o MVC é implementado e de como as responsabilidades são distribuídas. Ademais, a camada de dados seria uma camada separada, responsável por armazenar e recuperar dados. O Model do MVC frequentemente interage com essa camada, mas não é a própria camada de dados.

Vamos repetir o exemplo anterior (mesmo cenário), mas estruturado com o MVC. Na figura 126, DeusGrego é a representação simplificada de um deus grego, contendo propriedades como nome, domínio e símbolo.

```
1.  public class DeusGrego
2.  {
3.     public int Id { get; set; }
4.     public string Nome { get; set; }
5.     public string Dominio { get; set; } // Ex: "Mar", "Guerra",
"Sabedoria"
6.     public string Simbolo { get; set; } // Ex: "Tridente", "Coruja",
"Elmo"
7.  }
```

Figura 126 – MVC: exemplo de Model (em C#)

```
1.  <h1>Detalhes do Deus Grego</h1>
2.  <p>Nome: [DeusGrego.Nome]</p>
3.  <p>Domínio: [DeusGrego.Dominio]</p>
4.  <p>Símbolo: [DeusGrego.Simbolo]</p>
```

Figura 127 – MVC: exemplo de View (em pseudocódigo)

A "Visão" da figura 127 mostra os detalhes do deus grego. Não está em C#; é uma representação simples em pseudocódigo (pois a visualização real em uma aplicação ASP.NET Core usaria a sintaxe Razor junto ao HTML).

```
1.  public class DeusesGregosController : Controller
2.  {
3.    private readonly IRepositorioDados _repositorio;
4.
5.    public DeusesGregosController(IRepositorioDados repositorio)
6.    {
7.      repositorio = repositorio;
8.    }
9.
10.   public IActionResult Detalhes(int id)
11.   {
12.     DeusGrego deus = _repositorio.ObterDeusPorId(id);
13.     if (deus == null)
14.     {
15.       return NotFound();
16.     }
17.     return View(deus);
18.   }
19. }
```

Figura 128 – MVC: exemplo de Controller (em C#)

A figura anterior mostra o DeusesGregosController, capaz de uma ação chamada Detalhes, que recebe um ID como parâmetro.

Em uma aplicação web MVC típica, como uma criada usando ASP.NET Core, quando a aplicação é inicializada, o ASP.NET Core não procura um método Main. Em vez disso, a execução começa com o método ConfigureServices e Configure na classe Startup, que configura serviços, middlewares e define o pipeline de requisição. Quando uma requisição é feita para, digamos, /DeusesGregos/Detalhes/1, o ASP.NET Core roteia essa requisição para o método Detalhes no DeusesGregosController. O controlador então interage com o modelo para recuperar dados. No nosso exemplo, ele busca detalhes de um deus específico usando _repositorio.ObterDeusPorId(id). Recuperados os detalhes, o controlador passa o modelo (nesse caso, um DeusGrego) para a Visão. Ela então renderiza o HTML usando os detalhes do modelo, e o HTML renderizado é enviado de volta ao cliente (navegador).

Não é objetivo nosso detalhar a programação ASP.NET, mas a View geralmente incorpora outras linguagens e tecnologias, como HTML, CSS, JavaScript e, no caso do .NET, Windows Forms, WPF e a estrutura ASP.NET MVC. Como vimos no exemplo, o Controller e o Model geralmente são escritos em C#. Dessa forma, por enquanto não se preocupe com os detalhes (nomes, termos técnicos, sintaxe etc.) do ASP.NET ou de outras tecnologias e linguagens recém-mencionadas; elas foram colocadas de maneira ilustrativa para você compreender o cenário das aplicações reais e visualizar, com mais detalhes, o funcionamento e a integração com o C#, que é o foco deste livro-texto.

Também é possível incluir C# na View com o auxílio da Razor, uma sintaxe de template para incorporar código C# dentro de páginas HTML, permitindo que desenvolvedores misturem HTML com código C# para facilitar a leitura e a escrita. Razor é frequentemente usada em aplicações ASP.NET Core MVC e ASP.NET Core Razor Pages para criar views (ou páginas) renderizadas no lado do servidor e enviadas para o cliente. O processamento no lado servidor é necessário pois a sintaxe Razor por si só não é reconhecida nem processada pelos navegadores, ou seja, o navegador não entende nem processa diretamente o código.

A figura 129 ilustra como ficaria a View da figura 127 usando a sintaxe Razor. Quando uma solicitação é feita para essa View, o servidor ASP.NET Core processa o arquivo, ou seja, executa o código Razor incorporado e gera um arquivo HTML puro como resultado.

```
@model DeusGrego

<h1>Detalhes do Deus Grego</h1>
<p>Nome: @Model.Nome</p>
<p>Domínio: @Model.Dominio</p>
<p>Símbolo: @Model.Simbolo</p>
```

Figura 129 – MVC: exemplo View (em Razor)

A estrutura do .NET também oferece outras abordagens. O Model-View-ViewModel (MVVM) é uma escolha muito usada para aplicações WPF e Xamarin, voltadas respectivamente para os ambientes desktop e mobile. O padrão foi popularizado por John Gossman em 2005, quando trabalhava na Microsoft. Ele introduziu o MVVM como uma adaptação especializada do padrão Presentation Model para ser usado com o WPF, a nova tecnologia de UI da Microsoft na época.

O MVVM foi desenvolvido com a ideia de aproveitar ao máximo os recursos de vinculação de dados (data binding) oferecidos pelo WPF, permitindo uma separação mais clara da lógica de apresentação dos componentes da interface do usuário. Como o WPF (e posteriormente o Silverlight e o Xamarin) permite vinculações de dados complexas, a estrutura MVVM se encaixou naturalmente, proporcionando uma maneira eficiente de construir aplicações interativas com uma clara separação de responsabilidades. Desde então, o padrão MVVM ganhou popularidade não apenas em aplicações WPF, mas também em outras tecnologias e plataformas, incluindo desenvolvimento móvel e aplicações web de página única – esta, frequentemente abreviada como *single page application* (SPA), é uma aplicação ou site que interage com o usuário dinamicamente, reescrevendo a página atual, em vez de carregar páginas inteiras a partir do servidor. Ou seja, uma vez que a página web é carregada inicialmente, não são feitos mais carregamentos completos de página à medida que o usuário interage com a aplicação. E não podemos esquecer do Blazor, o promissor framework da Microsoft que propõe uma revolução ao permitir o desenvolvimento de aplicações web utilizando C# no lugar do convencional JavaScript.

Vamos repetir o exemplo anterior (mesmo cenário), mas estruturado com o MVVC; para efeitos comparativos, manteremos a nomenclatura. A figura 130 aponta o modelo que representa um deus grego (arquivo Deus.cs).

```
1.  public class Deus
2.  {
3.     public string Nome { get; set; }
4.     public string Dominio { get; set; }
5.     public string Simbolo { get; set; }
6.  }
```

Figura 130 – MVVM: exemplo Model (em C#)

Na sequência, a figura 131 representa a interface do usuário (arquivo DeusView.xaml), escrita em XAML, cuja sintaxe é usada principalmente para definir interfaces de usuário em aplicações Windows, particularmente em tecnologias como Windows Presentation Foundation (WPF), Universal Windows Platform (UWP) e Xamarin.Forms (a mesma linguagem que usamos na figura 120). Lembre-se que esse código é ilustrativo, para fornecer uma visão completa do MVVM, portanto não se preocupe com os detalhes da sintaxe nesse código-fonte.

```
1.  <Window x:Class="MitologiaGrega.DeusView"
2.       xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3.       xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4.       xmlns:local="clr-namespace:MitologiaGrega"
5.       Title="Detalhes do Deus Grego" Height="200" Width="300">
6.
7.     <Grid Margin="10">
8.       <Grid.RowDefinitions>
9.         <RowDefinition Height="Auto"/>
10.        <RowDefinition Height="Auto"/>
11.        <RowDefinition Height="Auto"/>
12.        <RowDefinition Height="Auto"/>
13.        <RowDefinition Height="Auto"/>
14.      </Grid.RowDefinitions>
15.
16.      <TextBlock Text="Detalhes do Deus Grego" FontWeight="Bold" Grid.
Row="0"/>
17.
18.      <TextBox Text="{Binding Nome}" Grid.Row="1" Margin="0,10,0,0"/>
19.      <TextBox Text="{Binding Dominio}" Grid.Row="2"
Margin="0,10,0,0"/>
20.      <TextBox Text="{Binding Simbolo}" Grid.Row="3"
Margin="0,10,0,0"/>
21.
22.      <Button Content="Salvar" Command="{Binding SalvarComando}" Grid.
Row="4" Margin="0,10,0,0"/>
23.    </Grid>
24. </Window>
```

Figura 131 – MVVM: exemplo View (em XAML)

Na arquitetura MVVM, a View (a parte gráfica, criada em XAML) se comunica apenas com o ViewModel. A lógica principal e os comandos que podem ser executados pela interface estão localizados no ViewModel, possibilitando à View permanecer mais "limpa", centrando-se apenas na apresentação

e interação do usuário. O ViewModel, por outro lado, interage com o Model, que é a representação dos dados. Portanto, quando o usuário interage com a View, ele está indiretamente alterando o Model através do ViewModel.

Para finalizar o exemplo, considere o arquivo DeusViewModel.cs, escrito em linguagem C# (figura 132). Ele atua como ponte entre o Model e a View, expondo os dados e comandos que a View utiliza.

```
1.  using System.Windows.Input;
2.
3.  public class DeusViewModel
4.  {
5.    private Deus deus;
6.
7.    public DeusViewModel()
8.    {
9.      _deus = new Deus();
10.     SalvarComando = new RelayCommand(Salvar);
11.   }
12.
13.   public string Nome
14.   {
15.     get { return deus.Nome; }
16.     set { _deus.Nome = value; }
17.   }
18.
19.   public string Dominio
20.   {
21.     get { return _deus.Dominio; }
22.     set { _deus.Dominio = value; }
23.   }
24.
25.   public string Simbolo
26.   {
27.     get { return deus.Simbolo; }
28.     set { _deus.Simbolo = value; }
29.   }
30.
31.   public ICommand SalvarComando { get; private set; }
32.
33.   private void Salvar()
34.   {
35.     // Lógica para salvar informações do deus
36.   }
37. }
```

Figura 132 – MVVM: exemplo ViewModel (em C#)

Se um usuário inserir o nome de um deus na interface e clicar no botão Salvar, a View passará esse comando para o ViewModel (através do binding com SalvarComando). O ViewModel então processa o comando, atualizando ou salvando o Model conforme necessário. A separação clara entre a View, ViewModel e Model permite uma melhor organização, testabilidade e reutilização do código – características centrais da arquitetura MVVM.

O MVVM não é um substituto do MVC: é adotado quando se mostra mais adequado e eficiente. Ambos os padrões de design têm seus méritos e são mais apropriados para certos tipos de aplicações e tecnologias. O MVC é tradicionalmente associado ao desenvolvimento web, no qual uma requisição é feita, processada pelo Controller – que interage com o Model – e finalmente retorna uma View para o usuário. Frameworks como ASP.NET MVC e Ruby on Rails são exemplos típicos. Já o MVVM é bastante popular em aplicações onde a vinculação de dados é predominante, sendo comumente utilizado em aplicações WPF, Xamarin (para desenvolvimento móvel) e em algumas aplicações JavaScript mais modernas, que usam frameworks como Angular ou Knockout.js.

Em resumo, para o desenvolvimento web tradicional, o MVC é frequentemente a escolha preferida devido à sua natureza request-response. Em contrapartida, se o foco do desenvolvimento são rich client applications (como desktop e mobile com interfaces complexas), o MVVM pode ser mais apropriado devido à sua forte separação de responsabilidades e capacidade de vincular dados, facilitando a atualização da interface do usuário em resposta a mudanças nos dados ou no modelo.

A expressão rich client applications refere-se a um tipo de aplicação que executa tarefas significativas no lado cliente, ou seja, no dispositivo do usuário, em contraste com aplicações que dependem de um servidor para processamento (como muitas aplicações web tradicionais).

Quadro 22 – Comparativo: três camadas, MVC, MVVM

Modelo/estrutura	Quando é indicado	Exemplo de uso
Estrutura em três camadas	Quando a aplicação tem forte componente de processamento de negócios separável das demais camadas	Sistemas de ERP, sistemas de CRM e outras aplicações empresariais complexas
	Quando a aplicação precisa ser escalável horizontalmente	
	Quando é necessário isolar a lógica de negócios para reutilização e manutenção	
Model-View-Controller (MVC)	Principalmente para aplicações web	Aplicações web baseadas em ASP.NET MVC, Ruby on Rails, Spring MVC
	Quando se busca uma clara separação entre lógica de apresentação, dados e controle de fluxo	
	Quando a testabilidade é importante	
	Foco em ações orientadas a controladores (manipulação de solicitações HTTP)	
Model-View-ViewModel (MVVM)	Principalmente para rich client applications, como aplicações desktop ou móveis	Aplicações desenvolvidas em WPF, Xamarin.Forms, SPA (Angular, Vue.js)
	Quando você está usando frameworks com vinculação de dados bidirecional (data binding)	
	Quando a lógica da interface do usuário é complexa e precisa ser separada da visualização	

O quadro 22 mostra que a seleção da arquitetura ou do padrão a ser adotado é variável. Cada projeto tem peculiaridades e necessidades, e a estrutura escolhida deve refletir e atender a essas demandas. A constante em todas essas escolhas é a busca por uma aplicação coesa, eficiente e sustentável. Além disso, os limites entre essas arquiteturas não são estritamente definidos; em algumas situações é possível combinar características ou até mesmo usar MVC com uma estrutura em três camadas. A escolha deve se basear nos requisitos do projeto, na natureza da aplicação e nas tecnologias que o projeto demanda.

Saiba mais

As camadas de aplicação são muito usadas no desenvolvimento de software moderno e se inter-relacionam com práticas de gerenciamento ágil de projetos. Para se aprofundar no assunto, leia o livro a seguir, muito citado na área:

MARTIN, R.; MARTIN, M. *Princípios, padrões e práticas ágeis em C#*. Porto Alegre: Bookman, 2011.

4.3 Trabalhando com números

Trabalhar com números em C# envolve entender tipos de dados numéricos, operações matemáticas básicas e algumas funções disponíveis na biblioteca-padrão. Esses tipos são fundamentais para uma gama de aplicações, desde a simples manipulação de números até operações complexas em sistemas financeiros e de engenharia.

Como vimos no tópico 3.3, os tipos podem ser divididos em duas categorias principais: os **integrais** e os **de ponto flutuante**. Os tipos integrais representam números sem casas decimais e podem ser tanto positivos quanto negativos. Em C#, temos desde o byte (um tipo de 8 bits sem sinal) até o long (com 64 bits e pode representar valores muito grandes; há uma lista detalhada no quadro 7). A capacidade de escolher entre diferentes tamanhos de tipos integrais permite que os programadores otimizem o uso da memória, selecionando o tipo apropriado conforme a necessidade; por exemplo, se sabemos que um valor nunca será negativo e não ultrapassará 255, o tipo byte seria o mais adequado.

Já os tipos de ponto flutuante representam números com casas decimais, que incluem o float (32 bits) e o double (64 bits). Esses tipos são cruciais para cálculos que necessitam de precisão fracional. Além deles há o tipo decimal, com 128 bits e frequentemente usado em aplicações financeiras, já que sua alta precisão e representação interna o tornam ideal para evitar erros de arredondamento.

É essencial entender as diferenças e particularidades dos tipos numéricos. Ao escolher adequadamente o tipo de dado numérico para determinada situação, o desenvolvedor não apenas otimiza o desempenho do programa, mas também garante que os cálculos sejam realizados de maneira precisa e confiável.

Operações matemáticas básicas constituem a pedra angular de muitos programas, permitindo aos desenvolvedores manipular e processar dados numéricos para atingir os objetivos desejados. Assim como em muitas outras linguagens de programação, C# oferece um conjunto-padrão de operadores aritméticos para lidar com adição, subtração, multiplicação, divisão e obtenção de um módulo. Essas operações básicas são autoexplicativas, mas serão comentadas a seguir, incluindo alguns detalhes.

A adição usa o operador **+**, frequentemente adotado não apenas para somar números, mas também para concatenar strings. Já a subtração usa o operador **–**, que diminui o valor do operando à direita em

relação ao operando à esquerda, não se vinculando a operações de string. A multiplicação, representada pelo operador *****, realiza essa operação entre dois números, enquanto a divisão, que utiliza o operador **/**, faz o cálculo com o operando à esquerda em relação ao operando à direita. É vital estar ciente de que dividir dois inteiros em C# resultará em uma divisão inteira, ou seja, qualquer resto será descartado, e o resultado será um número inteiro.

Outra operação básica em C# é o **módulo**, representado pelo operador %, que retorna o resto de uma divisão. É especialmente útil em situações onde precisamos verificar se um número é divisível por outro (como determinar se um ano é bissexto ou não). Além dessas operações primárias, temos os operadores bitwise, que permitem aos programadores manipular individualmente os bits dentro das representações binárias dos números. Esses operadores são frequentemente usados em programação de baixo nível, operações de otimização ou quando se trabalha diretamente com estruturas de dados binárias.

Operadores bitwise do quadro 23, a seguir, e operadores lógicos não são a mesma coisa; têm funções distintas e operam em diferentes níveis de dados. Embora ambos sejam usados para avaliar e manipular valores baseados em condições, têm propósitos e aplicações diferentes.

Quadro 23 – Operadores bitwise

Operador	Descrição
& (AND)	Retorna 1 para cada posição de bit em que ambos os operandos correspondentes são 1
| (OR)	Retorna 1 para cada posição de bit em que pelo menos um dos operandos correspondentes é 1
^ (XOR)	Retorna 1 para cada posição de bit em que exatamente um dos operandos correspondentes é 1
~ (NOT)	Inverte os bits do operando
<< (Deslocamento à esquerda)	Desloca os bits do primeiro operando para a esquerda pelo número de posições especificado pelo segundo operando
>> (Deslocamento à direita)	Desloca os bits do primeiro operando para a direita pelo número de posições especificado pelo segundo operando

Vamos usar a mitologia grega para ilustrar o uso desses operadores: imagine uma representação binária dos poderes dos deuses, composta de quatro bits, e a posição de cada bit representa um poder específico:

- O primeiro bit representa poder sobre os raios.
- O segundo bit representa poder sobre o vento.
- O terceiro bit representa poder sobre a sabedoria.
- O quarto bit representa poder sobre o mar.

Assim, se determinado deus tiver a sequência 1001, isso significa que tem poder sobre os raios e o mar. Da mesma forma, 0011 significa poder da sabedoria e do mar.

```
1.  int zeus = 0b1100; // Zeus tem poder sobre raios e vento
2.  int atena = 0b0010; // Atena tem poder sobre a sabedoria
3.  int poseidon = 0b0001; // Poseidon tem poder sobre o mar
4.
5.  // Para encontrar os poderes comuns entre Zeus e Atena:
6.  int poderesComuns = zeus & atena; // Resultado seria 0b0000, pois eles
não têm poderes em comum
7.
8.  // Para combinar os poderes de Zeus e Poseidon:
9.  int combinacao = zeus | poseidon; // Resultado seria 0b1101
10.
11. // Para descobrir quais poderes são exclusivos de Zeus quando
comparado a Atena:
12. int exclusivoZeus = zeus ^ atena; // Resultado seria 0b1110
```

Figura 133 – Exemplo de utilização dos operadores bitwise

No exemplo, os operadores bitwise representam, combinam e comparam os poderes dos deuses da mitologia grega; é uma forma simplificada de visualizar como esses operadores funcionam em contexto prático. As linhas 1, 2 e 3 usam a notação **0b** antes do número, que é um prefixo usado em C# (e em outras linguagens de programação modernas), para indicar que o número que segue é uma representação binária. Assim, em vez de fornecer o valor em decimal, hexadecimal ou qualquer outro sistema numeral, fornecemos o valor diretamente em binário, o que é útil se estivermos lidando com operações a nível de bit, como operações bitwise, porque permitem visualizar e manipular diretamente a sequência de bits.

Aproveitando o ensejo, algumas notações e seus significados:

- **0b**: para representar em binário.
  - Exemplo: **0b**1010 representa o número decimal 10.
- **0x**: para representar em hexadecimal.
  - Exemplo: **0x**0A também representa o número decimal 10.
- Sem prefixo: representação em decimal.
  - Exemplo: 10 é, naturalmente, o número decimal 10.

A linguagem C# é fortemente tipada, ou seja, cada variável e objeto tem um tipo específico associado a ele, e o compilador impõe regras rigorosas sobre como esses tipos podem interagir. Essa característica da linguagem tem implicações importantes para o desenvolvimento e a segurança do código. Primeiro, a tipagem forte ajuda a garantir a integridade dos dados. Ao tentar atribuir o valor de

um tipo a uma variável de tipo incompatível, o compilador emitirá um erro. Por exemplo, não é possível atribuir diretamente uma string a uma variável do tipo int sem uma conversão adequada. Esse nível de rigor protege de erros inadvertidos que poderiam introduzir bugs ou comportamentos indesejados no programa.

Além disso, a tipagem forte permite que ferramentas de desenvolvimento, como ambientes de desenvolvimento integrados (IDEs), ofereçam funcionalidades como conclusão inteligente de código e refatoração automática. Ao saber exatamente de que tipo é determinada variável, o IDE pode sugerir métodos ou propriedades relevantes. Contudo, enquanto a tipagem forte traz benefícios em termos de segurança e clareza, também implica em certa verbosidade e na necessidade de se atentar às conversões de tipo, especialmente quando é necessário interação entre diferentes tipos.

Por esse motivo é comum a conversão entre diferentes tipos de dados, especialmente ao lidar com números. Essas conversões são cruciais porque permitem aos desenvolvedores manipular dados de formas diversas, adaptando-os às necessidades de diferentes operações ou funções.

Existem dois tipos principais de conversão em C#: **implícitas** e **explícitas**. As primeiras ocorrem automaticamente quando se passa o valor de um tipo menor (que ocupa menos espaço em bytes na memória) para um tipo maior (que requer mais espaço em memória); por exemplo, é seguro converter um int em um double sem perder informação. O compilador entende essa operação e a realiza sem precisar de qualquer intervenção adicional do programador. Por outro lado, conversões explícitas, frequentemente referidas como casting (conceito introduzido no tópico 3.12), são necessárias quando há risco de perder dados; por exemplo, ao converter um double em um int, a parte fracional do double será descartada. Em tais casos, o programador deve informar explicitamente o compilador de que está ciente da potencial perda de dados, utilizando a sintaxe de casting.

A figura 134 ilustra a conversão de forma explícita usando abre-fecha parênteses à esquerda da variável meuDouble.

```
1.   int meuInteiro = (int)meuDouble;
```

Figura 134 – Conversão explícita de double para int

Além dessas conversões básicas, os próprios tipos numéricos têm métodos como ToString, Parse e TryParse, vitais para converter números em strings e vice-versa. O primeiro método citado converte o valor numérico atual em sua representação de string, sendo útil quando precisamos apresentar um valor numérico em formato legível ou para combiná-lo com outras strings. O segundo converte a representação de string de um número em seu valor numérico equivalente; se a string não puder ser convertida, lançará uma exceção.

Em aplicações do cotidiano, a conversão entre números e strings é uma necessidade recorrente. Em um sistema bancário online, quando um cliente realiza uma transação, é comum que o sistema precise apresentar os detalhes. Suponha que um cliente acabou de transferir certa quantia para outra conta; para informar ao cliente o montante transferido, o sistema – que armazena esse valor como um

número – precisará convertê-lo em texto e incorporá-lo em uma mensagem mais amigável, como "Você transferiu R$ 1.000,00 com sucesso". Aqui o método ToString é fundamental para transformar o valor numérico em uma representação textual que possa se integrar a uma interface ou mensagem.

Imagine ainda um aplicativo de e-commerce cujos usuários possam filtrar produtos por preço. O usuário digita um valor no campo de pesquisa (como 50) para encontrar produtos até esse preço. No entanto esse valor é inicialmente recebido pelo sistema como uma string, pois é assim que os dados de entrada do usuário geralmente são capturados. Para o sistema realmente poder filtrar os produtos por esse valor, é preciso converter essa string em número; e aqui o método Parse entra em ação. Se o usuário inserir algo que não seja um número, como "cinquenta", o sistema pode lançar um erro, indicando que o valor inserido não é válido.

Considere também, como exemplo do cotidiano, um software de registro de pacientes em um hospital. Ao inserir a idade do paciente, o sistema precisa verificar se o valor inserido é realmente um número. Usar o método Parse diretamente pode não ser a melhor escolha, pois se o usuário inserir algo como "vinte e três" em vez de "23" o programa pode falhar. E aqui o TryParse se mostra útil, pois tenta fazer a conversão e, em caso de falha, fornece um feedback mais amigável, como "Por favor, insira a idade em algarismo", sem interrupções inesperadas no programa.

Além dessas conversões, temos uma rica **biblioteca-padrão** com várias funções adicionais, garantindo que os programadores tenham as ferramentas necessárias para abordar problemas complexos com eficiência e precisão, oferecendo uma variedade de funções que facilitam a manipulação e o processamento de números. A capacidade de trabalhar com números de forma eficiente e precisa é essencial para muitas aplicações, e o C#, reconhecendo essa importância, fornece ferramentas robustas. Analisaremos, em especial, Convert, Numerics e Math.

4.3.1 Convert

A classe **Convert** fornece métodos para transformar tipos de dados de forma mais controlada, o que pode ser útil quando lidamos com conversões entre tipos não numéricos e tipos numéricos, como strings que representam números. A classe Convert é parte do namespace System e fornece métodos estáticos para converter tipos de dados-base (como números, booleans e datas) para e de outros tipos de dados-base.

A figura 135 apresenta exemplos de ToString, Parse e TryParse para conversões.

```
1.  using System;
2.
3.  namespace MitologiaGrega
4.  {
5.     class Program
6.     {
7.        static void Main(string[] args)
8.        {
9.           // Utilizando o método ToString
10.          // Convertendo um número em uma string para exibir uma mensagem
sobre Zeus
11.          int numeroDeRaios = 100;
12.          string mensagem = "Zeus lançou " + numeroDeRaios.ToString() +
" raios hoje!";
13.          Console.WriteLine(mensagem); // Saída: Zeus lançou 100 raios
hoje!
14.
15.          // Utilizando o método Parse
16.          // Convertendo uma string para um número para calcular quantos
dias Atlas carregou o céu
17.          string anosTexto = "1000"; // Suponha que lemos isso do
pergaminho
18.          int anos = int.Parse(anosTexto);
19.          int dias = anos * 365;
20.          Console.WriteLine($"Atlas carregou o céu por {dias} dias!");
21.
22.          // Utilizando o método TryParse
23.          // Tentando converter uma string para um número e lidando com
possíveis erros
24.          string textoPergaminho = "Ares"; // Pode ou não ser um número
25.          int valor;
26.          if (int.TryParse(textoPergaminho, out valor))
27.          {
28.             Console.WriteLine($"O valor do pergaminho é: {valor}");
29.          }
30.          else
31.          {
32.             Console.WriteLine($"O texto '{textoPergaminho}' não é um
número válido.");
33.          }
34.          // Saída esperada: O texto 'Ares' não é um número válido.
35.       }
36.    }
37. }
```

Figura 135 – ToString, Parse e TryParse: exemplo de conversões

Ao contrário dos métodos Parse e ToString – específicos para determinados tipos (como int.Parse ou dateTime.ToString) –, a classe Convert fornece métodos como ToInt32, ToBoolean e ToString, que aceitam parâmetros de tipo object. Essa abordagem universal facilita conversões sem se preocupar muito com a origem. Outra característica interessante é a forma como a classe Convert lida com valores nulos; por exemplo, se tentarmos converter um objeto nulo (null) em um tipo numérico usando Convert, ele retornará o valor zero, e caso uma conversão não seja possível (por exemplo, tentando converter a string "olá" para um número) a classe Convert lançará uma exceção – semelhante ao comportamento do método Parse.

O quadro 24 oferece uma lista descritiva com diversos métodos úteis para converter tipos em C#.

Quadro 24 – Classe Convert: principais métodos

Método	Descrição
ToBoolean	Converte um valor especificado em um booleano. Pode converter uma string "true" ou "false" em seu valor booleano correspondente
ToByte	Converte um valor especificado no menor tipo inteiro, que é byte
ToChar	Converte o valor especificado em um caractere
ToDateTime	Converte o valor especificado em um DateTime. Muito útil para conversões a partir de strings que representam datas
ToDecimal	Converte um valor especificado em um número decimal
ToDouble	Converte o valor especificado em um número de ponto flutuante de dupla precisão
ToInt16	Converte o valor especificado em um inteiro de 16 bits
ToInt32	Converte o valor especificado em um inteiro de 32 bits
ToInt64	Converte o valor especificado em um inteiro de 64 bits
ToSByte	Converte o valor especificado em um inteiro com sinal de byte
ToSingle	Converte um valor especificado em um número de ponto flutuante de precisão simples
ToString	Converte o valor especificado em uma string. Se o valor for null, esse método retorna uma string vazia, evitando a NullReferenceException
ToUInt16	Converte o valor especificado em um inteiro sem sinal de 16 bits
ToUInt32	Converte o valor especificado em um inteiro sem sinal de 32 bits
ToUInt64	Converte o valor especificado em um inteiro sem sinal de 64 bits

A figura 136 contém alguns exemplos do uso dos métodos de conversão:

```
1.  using System;
2.
3.  namespace MitologiaGrega
4.  {
5.     class Programa
6.     {
7.        static void Main(string[] args)
8.        {
9.           // Uma string representando o número de trabalhos realizados
por Hércules
10.          string dozeTrabalhos = "12";
11.
12.          // Convertendo a string para um inteiro usando Convert.ToInt32
13.          int numeroDeTrabalhos = Convert.ToInt32(dozeTrabalhos);
14.
15.          // Representando o fato de que Medusa tinha originalmente belos
cabelos antes de ser amaldiçoada
16.          string cabeloMedusa = "true";
17.
18.          // Convertendo a string para um booleano usando Convert.ToBoolean
19.          bool tinhaCabeloBonito = Convert.ToBoolean(cabeloMedusa);
20.
21.          // Convertendo um valor numérico para byte - representando o
número de cabeças da Hidra de Lerna
22.          byte cabecasHidra = Convert.ToByte(9);
23.
24.          // Uma string representando a data da primeira aparição de
Pégaso
25.          string nascimentoPegaso = "2000-01-01";
26.
27.          // Convertendo a string para DateTime usando Convert.ToDateTime
28.          DateTime dataNascimento = Convert.ToDateTime(nascimentoPegaso);
29.
30.          // Convertendo o valor decimal de ouro que o Rei Midas possuía
para double
31.          double quantidadeOuro = Convert.ToDouble(1000.25m);
32.
33.          // Exibindo os resultados
34.          Console.WriteLine($"Hércules realizou {numeroDeTrabalhos}
trabalhos.");
35.          Console.WriteLine($"É {tinhaCabeloBonito} que Medusa tinha
cabelos belos antes de ser amaldiçoada.");
36.          Console.WriteLine($"A Hidra de Lerna tinha {cabecasHidra}
cabeças.");
37.          Console.WriteLine($"Pégaso nasceu em {dataNascimento.
ToShortDateString()}.");
38.          Console.WriteLine($"Rei Midas possuía {quantidadeOuro} unidades
de ouro.");
39.          Console.ReadLine();
40.       }
41.    }
42. }
```

Figura 136 – Classe Convert: utilizando alguns métodos

A string dozeTrabalhos representa o número de trabalhos que Hércules realizou. Na linha 13, convertemos essa string para um inteiro usando Convert.ToInt32 e armazenamos o valor resultante na variável numeroDeTrabalhos. A string cabeloMedusa representa o fato conhecido de que Medusa, antes de ser transformada em um monstro, tinha belos cabelos. Convertemos essa string para um valor booleano na linha 19 usando Convert.ToBoolean e armazenamos o resultado na variável tinhaCabeloBonito.

Na linha 22 utilizamos Convert.ToByte para transformar o número 9, que representa o número de cabeças da Hidra de Lerna, em um byte. O resultado é armazenado na variável cabecasHidra. A data de "nascimento" de Pégaso (valor fictício para fins desse exemplo) é representada pela string nascimentoPegaso, sendo convertida para um objeto DateTime usando Convert.ToDateTime na linha 28, e o resultado é armazenado em dataNascimento. A quantidade de ouro que o rei Midas possuía é representada como valor decimal, que na linha 31 convertemos para um double usando Convert.ToDouble e armazenamos na variável quantidadeOuro.

Como pudemos observar, cada método da classe Convert transforma valores de um tipo para outro. Isso nos permite, por exemplo, realizar operações matemáticas ou formatar e exibir esses valores de maneira mais adequada no contexto do programa.

Os métodos da classe Convert também podem converter dados informados pelo usuário em aplicações do tipo console, após sua leitura. Na figura 137 criamos um código no qual um usuário informa alguns dados relacionados à mitologia grega, e nós usaremos os métodos da classe Convert para tratar essas entradas.

```
1.  using System;
2.
3.  namespace MitologiaGrega
4.  {
5.     class Programa
6.     {
7.        static void Main(string[] args)
8.        {
9.           Console.WriteLine("Quantos trabalhos Hércules realizou?");
10.          int numeroDeTrabalhos = Convert.ToInt32(Console.ReadLine());
11.
12.          Console.WriteLine("A Medusa tinha cabelos belos antes de ser
amaldiçoada? (true/false)");
13.          bool tinhaCabeloBonito = Convert.ToBoolean(Console.ReadLine());
14.
15.          Console.WriteLine("Quantas cabeças a Hidra de Lerna tinha?");
16.          byte cabecasHidra = Convert.ToByte(Console.ReadLine());
17.
18.          Console.WriteLine("Em que ano Pégaso nasceu? (formato: YYYY-MM-DD)");
19.          DateTime dataNascimento = Convert.ToDateTime(Console.ReadLine());
20.
21.          Console.WriteLine("Quantas unidades de ouro o Rei Midas possuía?");
22.          double quantidadeOuro = Convert.ToDouble(Console.ReadLine());
23.
24.          // Resumindo as respostas
25.          Console.WriteLine($"\nHércules realizou {numeroDeTrabalhos}
trabalhos.");
26.          Console.WriteLine($"É {tinhaCabeloBonito} que Medusa tinha
cabelos belos antes de ser amaldiçoada.");
27.          Console.WriteLine($"A Hidra de Lerna tinha {cabecasHidra}
cabeças.");
28.          Console.WriteLine($"Pégaso nasceu em {dataNascimento.
ToShortDateString()}.");
29.          Console.WriteLine($"Rei Midas possuía {quantidadeOuro} unidades
de ouro.");
30.       }
31.    }
32. }
```

Figura 137 – Classe Convert: métodos na leitura de dados do Console

4.3.2 Numerics

A biblioteca **System.Numerics** é uma parte do .NET que fornece um conjunto de tipos numéricos indisponíveis nos tipos primitivos padrão, e são projetados para lidar com operações numéricas mais complexas ou que requeiram mais precisão ou escala do que tipos convencionais, como int, double e float.

Vamos discutir os principais componentes dessa biblioteca: BigInteger, Complex, vetores e matrizes.

BigInteger

Estrutura presente na biblioteca System.Numerics do C#, foi projetada para representar um inteiro arbitrariamente grande, eliminando efetivamente os limites que encontramos com tipos numéricos padrão como int, long e ulong. Em outras palavras, enquanto tipos como int e long têm valores mínimos e máximos determinados, um BigInteger pode crescer (ou diminuir) tanto quanto a memória do sistema permitir. Isso torna o BigInteger útil em cenários que requerem cálculos com números muito grandes, como em operações criptográficas, algoritmos de fatoração de números, simulações matemáticas e outros domínios que ultrapassam os limites dos tipos numéricos padrão.

O quadro 25 descreve as limitações dos tipos inteiros.

Quadro 25 – Valores mínimos e máximos de alguns tipos numéricos

Tipo de dado	Valor mínimo	Valor máximo
int	-2,147,483,648 (ou int.MinValue)	2,147,483,647 (ou int.MaxValue)
long	-9,223,372,036,854,775,808 (ou long.MinValue)	9,223,372,036,854,775,807 (ou long.MaxValue)
ulong	0 (ou ulong.MinValue)	18,446,744,073,709,551,615 (ou ulong.MaxValue)

Para o BigInteger a situação é um pouco diferente, dado que foi projetada para representar inteiros de tamanho arbitrário. Isso significa que não tem um valor mínimo ou máximo predefinido como os tipos numéricos padrão. Em teoria, um BigInteger pode crescer (ou diminuir) tanto quanto a memória do sistema permitir. Na prática, isso significa que o tamanho de um BigInteger é limitado apenas pela quantidade de memória disponível em seu sistema. No entanto, o uso excessivo de memória pode resultar em problemas de desempenho e causar exceções relacionadas à memória em seu programa.

Uma característica diferenciada do BigInteger é o fato de ser imutável. Ou seja, uma vez que um objeto BigInteger é criado, seu valor não pode ser alterado. Qualquer operação que pareça modificar o valor de um BigInteger (como adição, subtração ou multiplicação) na verdade retorna um novo objeto BigInteger com o resultado da operação.

A imutabilidade, embora possa parecer um obstáculo à primeira vista, oferece vantagens significativas em termos de previsibilidade e segurança do código. Como os objetos BigInteger não podem ser alterados após sua criação, os desenvolvedores podem confiar que o valor de um BigInteger não será alterado inesperadamente devido a efeitos colaterais em outras partes do código.

No entanto, o BigInteger vem com suas próprias considerações de desempenho. Por não ter um tamanho fixo, suas operações podem ser mais lentas se comparadas às que usam tipos numéricos padrão. Além disso, como pode crescer indefinidamente, talvez leve a um alto consumo de memória em operações que geram números extremamente grandes. Para muitos desenvolvedores e aplicações, os tipos numéricos padrão fornecem capacidade suficiente; mas em casos especiais – onde números além dos limites são necessários – o BigInteger preenche essa lacuna, proporcionando uma solução flexível e robusta para trabalhar com inteiros de tamanho arbitrário.

O quadro 26 enumera e descreve os principais métodos do BigInteger.

Quadro 26 – BigInteger: principais métodos

Categoria	Operação	Descrição
Operações aritméticas	Add	Adiciona dois valores BigInteger
	Subtract	Subtrai um valor BigInteger de outro
	Multiply	Multiplica dois valores BigInteger
	Divide	Divide um valor BigInteger por outro
	Remainder	Retorna o resto da divisão de um valor BigInteger por outro
	Negate	Inverte o sinal de um valor BigInteger
	Abs	Retorna o valor absoluto de um BigInteger
Operações de bit	ShiftLeft	Desloca um valor BigInteger para a esquerda por um número especificado de bits
	ShiftRight	Desloca um valor BigInteger para a direita por um número especificado de bits
	BitwiseAnd	Executa uma operação bitwise AND entre dois valores BigInteger
	BitwiseOr	Realiza uma operação bitwise OR entre dois valores BigInteger
Comparação	Compare	Compara dois valores BigInteger
	Equals	Determina se dois valores BigInteger são iguais
	Sign	Obtém um número que indica o sinal (positivo, negativo ou zero) do valor BigInteger
Outras operações	GreatestCommonDivisor	Retorna o maior divisor comum (MDC) de dois números
	ModPow	Executa a operação de exponenciação modular
	IsPowerOfTwo	Determina se um valor é uma potência de dois
	Log	Retorna o logaritmo de um número especificado

Convém mencionar que o .NET não oferece nenhuma biblioteca, similar ao BigInteger, para tipos de ponto flutuante.

O quadro 27 detalha as limitações dos tipos de ponto flutuante. É importante mencionar que os tipos float e double são representações binárias de ponto flutuante e, portanto, podem ter problemas de precisão ao representar números que não são exatamente representáveis em binário. O tipo decimal, por outro lado, é uma representação baseada em base 10 e é projetado especificamente para evitar problemas de precisão, especialmente útil em cálculos monetários e financeiros.

Quadro 27 – Valores mínimos e máximos de tipos numéricos (ponto flutuante)

Tipo de dado	Mínimo	Máximo	Precisão
float (single)	Aproximadamente –3.4 × 10^38 (ou float.MinValue)	Aproximadamente 3.4 × 10^38 (ou float.MaxValue)	Cerca de 7 dígitos decimais
double	Aproximadamente –1.7 × 10^308 (ou double.MinValue)	Aproximadamente 1.7 × 10^308 (ou double.MaxValue)	Cerca de 15-16 dígitos decimais
decimal	Aproximadamente –7.9 × 10^28 (ou decimal.MinValue)	Aproximadamente 7.9 × 10^28 (ou decimal.MaxValue)	Cerca de 28-29 dígitos decimais significativos

Como dissemos, a linguagem-padrão e sua biblioteca não incluem um equivalente para ponto flutuante de precisão arbitrária em C#; no entanto bibliotecas de terceiros oferecem essa funcionalidade. Uma das bibliotecas mais populares que fornecem tal capacidade é a MathNet.Numerics, cujo tipo BigFloat oferece operações de ponto flutuante com precisão arbitrária. É uma biblioteca open-source (de código aberto) voltada para computação numérica em .NET, que fornece ampla variedade de funcionalidades matemáticas, desde álgebra linear até otimização e integração.

A Microsoft fornece muitas bibliotecas e ferramentas para desenvolvedores, mas não todas; o MathNet.Numerics, por exemplo, não é seu produto. A comunidade .NET é rica e diversificada, com muitos contribuintes independentes e organizações que fornecem bibliotecas e ferramentas de alta qualidade, e o MathNet.Numerics é exemplo dessa contribuição da comunidade à ecossistema .NET. Um desenvolvedor interessado em usá-lo (ou qualquer outra biblioteca de terceiros) deve lembrar que sempre é boa prática verificar sua documentação, revisões e talvez até seu código-fonte, para garantir que ele atenda a suas necessidades e padrões de qualidade.

Complex

O tipo **Complex** da biblioteca System.Numerics representa um número complexo, um tipo de número que tem tanto uma parte real quanto uma parte imaginária. Essa estrutura é projetada para trabalhar com operações matemáticas que envolvem números complexos, abrindo as portas para uma variedade de aplicações, particularmente aquelas no domínio da matemática, física e engenharia.

A parte real e a parte imaginária de um número complexo são representadas como números de ponto flutuante de precisão dupla. Na matemática tradicional, a parte imaginária é representada com a letra i, mas em engenharia, especialmente em contextos de engenharia elétrica, a notação j é frequentemente usada. Em C#, a parte imaginária é representada com a letra i, conforme a convenção matemática. A estrutura Complex não apenas armazena números complexos, mas também oferece uma série de operações e métodos úteis para trabalhar com eles; por exemplo, você pode somar, subtrair, multiplicar ou dividir números complexos. Além das operações aritméticas básicas, há funções que permitem obter a magnitude (ou módulo) de um número complexo, seu ângulo ou fase, bem como outras propriedades e operações típicas associadas a números complexos, como o conjugado.

Outro aspecto fundamental do Complex é sua capacidade de interagir com outros tipos numéricos; ou seja, podemos facilmente converter tipos mais simples – como inteiros ou números de ponto flutuante – para o tipo complexo e vice-versa. Essa interconversão é útil em cenários onde uma função pode aceitar vários tipos de número, incluindo complexos.

Ao longo dos anos, a computação com números complexos tornou-se crucial em muitas áreas. A presença de um tipo de dados nativo para números complexos no C# (como a estrutura Complex) sublinha a flexibilidade e a capacidade da linguagem de se adaptar a uma variedade de necessidades de computação. Seja para resolver equações em domínios complexos ou modelar fenômenos em ciências naturais, Complex é uma estrutura inestimável no arsenal do desenvolvedor .NET.

Vetores

Além dos complexos, a biblioteca System.Numerics em C# oferece um conjunto especializado de estruturas projetadas para trabalhar com vetores, fundamentais não apenas em matemática e física, mas em gráficos de computador, simulações e muitos outros domínios técnicos e científicos. Vetores no contexto da System.Numerics implicam coleções ordenadas de números que representam uma direção e magnitude no espaço. O tratamento computacional de vetores permite operações como adição e subtração de vetores, escalar (multiplicar um vetor por um número) e até determinar o produto escalar ou produto vetorial de dois vetores.

A System.Numerics oferece variadas estruturas de vetores, especificamente otimizadas para diferentes dimensões e tipos de dados. A mais notável é a **Vector2**, projetada para representar vetores em espaço bidimensional, frequentemente usada em gráficos 2D. Da mesma forma, **Vector3** e **Vector4** destinam-se respectivamente a espaços tridimensionais e quadridimensionais, com Vector3 sendo um pilar em gráficos 3D e simulações.

Essas estruturas não são apenas armazenamentos simples de dados; são equipadas com um arsenal de métodos e operações que facilitam as tarefas matemáticas comuns associadas a vetores. Isso inclui (mas não se limita a) determinar a magnitude de vetores, normalizá-los, calcular distâncias e ângulos entre eles e muito mais. O que é notável desde a abordagem de C# à computação vetorial é a ênfase na eficiência. Essas estruturas são otimizadas para tirar proveito das instruções de vetorização oferecidas por muitos processadores modernos. Isso significa que operações com vetores podem ser realizadas de forma muito mais rápida, especialmente quando diversas operações precisam ser executadas em paralelo, como em gráficos 3D ou processamentos de imagem.

Além das estruturas dedicadas a dimensões específicas, a System.Numerics introduz o tipo genérico **Vector<T>**, que pode usar qualquer tipo numérico como seu componente e é otimizado para tirar o máximo proveito das capacidades de vetorização do hardware subjacente.

Matrizes

No universo da computação e da matemática, matrizes são uma ferramenta indispensável. No C#, a biblioteca System.Numerics não apenas reconhece a importância das matrizes, mas também oferece uma abordagem robusta e otimizada para trabalhar com elas, sob a forma da estrutura **Matrix4×4** e outras matrizes associadas. Em essência, matrizes são uma coleção bidimensional de números organizados em linhas e colunas, e comportam uma gama de aplicações, desde resolver sistemas de equações lineares até transformar gráficos computacionais e animações.

Em gráficos 3D a transformação de pontos e vetores usando matrizes é uma operação-padrão. Na System.Numerics a estrutura Matrix4×4 representa uma matriz 4×4, particularmente útil para operações em gráficos tridimensionais, sendo uma das ferramentas mais usadas em transformações 3D, como translação, rotação, escala e até projeções perspectivas. Sua eficácia e versatilidade a tornam fundamental para renderizar gráficos e simulações.

Na implementação de matrizes no C#, são notáveis a eficiência e riqueza de métodos embutidos nelas. Ao usar Matrix4x4, por exemplo, os desenvolvedores não precisam reinventar a roda nem mergulhar profundamente em álgebra linear, estando prontamente disponíveis funções para multiplicar matrizes, inverter, obter um determinante e criar matrizes específicas para diferentes transformações, com funcionalidades que economizam tempo e reduzem a possibilidade de erros.

Além do aspecto funcional, a performance é outro ponto-chave. As operações da matriz são otimizadas para proporcionar cálculos rápidos, aproveitando as capacidades de hardware quando disponíveis, garantindo que transformações e cálculos sejam executados de forma eficaz.

4.3.3 Math

A classe Math (abordada brevemente no tópico 3.3) é parte fundamental da biblioteca padrão .NET, servindo como repositório central para funções matemáticas comumente necessárias em aplicações de software. Embora a matemática possa ser vista por muitos como tópico desafiador e por vezes esotérico, a abordagem do C# ao fornecer essa classe é simplificar a vida do desenvolvedor, oferecendo uma gama de funções prontamente disponíveis, que lidam com cálculos complexos por trás dos bastidores.

O ponto central da classe Math é sua universalidade e eficiência. Muitos desenvolvedores, seja trabalhando em sistemas financeiros, jogos, análises científicas ou aplicações gráficas, encontrarão situações em que precisam de operações matemáticas precisas e otimizadas.

Em vez de escrever essas funções do zero – o que pode ser propenso a erros e ineficiências –, a classe Math fornece uma solução confiável e testada. Por exemplo, ao lidar com trigonometria, uma área da matemática que explora as relações entre os lados e ângulos de triângulos, não é incomum que desenvolvedores necessitem calcular senos, cossenos ou tangentes de ângulos. A classe Math tem funções dedicadas para esses cálculos, eliminando a necessidade de os desenvolvedores terem entendimento profundo de como essas funções são derivadas ou calculadas.

Além da trigonometria, a classe Math abrange outras áreas da matemática, como a aritmética, fornecendo funções para encontrar valores absolutos, potências e raízes quadradas, além do suporte para constantes matemáticas amplamente utilizadas, como pi e E. Outro ponto notável é sua capacidade de lidar com precisão e limitações numéricas. Em computação, trabalhar com números pode ser complicado devido à precisão finita e à representação binária. A classe Math ajuda a trabalhar com esses desafios, oferecendo funções que lidam com arredondamento, comparação e outras operações que podem ser afetadas por imprecisões numéricas.

4.4 Trabalhando com texto

Trabalhar com texto em C# envolve compreender as capacidades e particularidades da linguagem em relação à manipulação de strings. No tópico 3.5 vimos como utilizar as próprias strings, seus principais métodos e recursos, além da questão de sua imutabilidade. Esta característica, embora promova a integridade dos dados, pode inadvertidamente levar a problemas de desempenho quando se trata de

operações de manipulação de strings em grande escala; e é precisamente o motivo pelo qual o C# oferece o StringBuilder.

Projetado especificamente para quando strings precisam ser modificadas repetidamente, como em loops, o StringBuilder fornece uma maneira de manipular texto sem sobrecarga associada à criação de múltiplas instâncias de string. A natureza versátil do C# também se manifesta na variedade de métodos que oferece para manipular texto, tornando mais acessíveis e eficientes tarefas como pesquisar substrings, substituir caracteres ou segmentos de texto, e até mesmo alterar a capitalização.

A introdução da interpolação de string nas versões recentes do C# elevou ainda mais a capacidade de formatar texto, permitindo a desenvolvedores inserir diretamente variáveis e expressões em literais de string, facilitando a leitura e a escrita do código. Outro aspecto crucial ao trabalhar com texto em C# é a sensibilidade à cultura. O mundo é diversificado, com idiomas e escritas de muitas regras e particularidades. O C# reconhece essa diversidade, proporcionando operações de string sensíveis ao contexto cultural. Isso é evidente, por exemplo, nas operações de comparação de strings, onde a ordem das letras e a capitalização podem variar de acordo com a cultura.

No código da figura 138, exploramos diversos métodos de manipulação de strings em C# usando o contexto da mitologia grega. Começamos com o método String.Replace, crucial para substituir todas as ocorrências de uma substring específica por outra. No nosso exemplo, foi utilizado para atualizar as referências de Zeus para Júpiter na narrativa (linha 11). Em sequência, o método String.Contains foi usado para verificar se uma sequência específica de caracteres aparece na string, o que nos ajudou a identificar se Zeus era mencionado como rei na história (linha 14).

Ao falar sobre Afrodite, introduzimos os métodos String.ToLower e String.ToUpper, capazes de converter todos os caracteres da string em minúsculas ou maiúsculas, respectivamente (linhas 20 e 21). Isso nos permitiu ver o nome Afrodite tanto em sua forma original quanto em variações de caixa-alta e baixa. A partir daí, com a história de Hera, demonstramos os métodos String.StartsWith e String.EndsWith (linhas 24 e 28, respectivamente), extremamente úteis para verificar se uma string começa ou termina com uma sequência específica de caracteres.

O método String.Substring, empregado na narrativa de Zeus, tem a habilidade de extrair uma parte da string com base em um índice inicial e um comprimento (linha 33). Nesse caso, foi utilizado para pegar uma parte específica da história de Zeus que se refere ao seu domínio do trovão. Introduzimos nas linhas 37 e 38 o StringBuilder.Append, método fundamental para construir uma string a partir de várias adições, principalmente porque oferece uma saída eficiente sem criar novas instâncias de string a cada adição. No contexto da mitologia grega, utilizamos isso para construir uma string que nos fale sobre a riqueza das histórias e narrativas dos deuses.

Finalmente, chegamos aos métodos String.Join e String.Split. O primeiro (linha 42) permite combinar várias strings em uma única usando um delimitador específico – isso foi demonstrado ao juntar nomes de vários deuses com vírgulas. O último, por sua vez, faz o oposto, dividindo uma string em várias substrings com base em um delimitador (linha 44). No exemplo, foi usado para separar e imprimir individualmente o nome dos deuses que havíamos combinado anteriormente.

```
1.  using System;
2.  using System.Text;
3.  namespace DemonstracaoMitologiaGrega
4.  {
5.    class Programa
6.    {
7.      static void Main(string[] args)
8.      {
9.        // Demonstrando o método String.Replace()
10.       string historiaZeus = "Zeus é o deus do trovão. Zeus é o rei
do Olimpo.";
11.       string historiaAtualizadaZeus = historiaZeus.Replace("Zeus",
"Júpiter");
12.       Console.WriteLine(historiaAtualizadaZeus); // Júpiter é o deus
do trovão. Júpiter é o rei do Olimpo.
13.       // Demonstrando o método String.Contains()
14.       if (historiaZeus.Contains("rei"))
15.       {
16.         Console.WriteLine("Zeus é mencionado como rei.");
17.       }
18.       // Demonstrando String.ToLower() e String.ToUpper()
19.       string nomeAfrodite = "Afrodite";
20.       Console.WriteLine(nomeAfrodite.ToLower()); // afrodite
21.       Console.WriteLine(nomeAfrodite.ToUpper()); // AFRODITE
22.       // Demonstrando String.StartsWith() e String.EndsWith()
23.       string historiaHera = "Hera é a rainha dos deuses e esposa de
Zeus.";
24.       if (historiaHera.StartsWith("Hera"))
25.       {
26.         Console.WriteLine("A história começa com Hera.");
27.       }
28.       if (historiaHera.EndsWith("Zeus."))
29.       {
30.         Console.WriteLine("A história termina mencionando Zeus.");
31.       }
32.       // Demonstrando o método String.Substring()
33.       string subHistoria = historiaZeus.Substring(12, 21); // extrai
"deus do trovão"
34.       Console.WriteLine(subHistoria);
35.       // Demonstrando o método StringBuilder.Append()
36.       StringBuilder sb = new StringBuilder();
37.       sb.Append("A mitologia grega é rica em histórias. ");
38.       sb.Append("De Zeus a Afrodite, cada deus tem sua própria
narrativa.");
39.       Console.WriteLine(sb.ToString());
40.       // Demonstrando os métodos String.Join() e String.Split()
41.       string[] deuses = { "Zeus", "Hera", "Afrodite", "Ares", "Atena"
};
42.       string deusesUnidos = String.Join(", ", deuses);
43.       Console.WriteLine(deusesUnidos); // Zeus, Hera, Afrodite, Ares,
Atena
44.       string[] divindades = deusesUnidos.Split(", ");
45.       foreach (string deus in divindades)
46.       {
47.         Console.WriteLine(deus);
48.       }
49.     }
50.   }
51. }
```

Figura 138 – String e StringBuilder: métodos em ação

Além de utilizar os métodos, é importante entender os recursos de concatenação, interpolação e a formatação de texto. **Concatenação** é o processo de unir duas ou mais strings para formar uma única. Em muitos cenários, como ao criar mensagens personalizadas ou ao construir consultas dinâmicas, a concatenação se torna um recurso essencial. Embora seja uma ferramenta simples, é extremamente poderosa e versátil, permitindo aos desenvolvedores combinar informações de várias fontes diferentes em uma única sequência de texto.

Interpolação, por outro lado, oferece uma maneira de inserir valores de variáveis ou expressões diretamente dentro de uma string. Em vez de quebrar a string ou usar métodos tradicionais de concatenação, a interpolação em C# proporciona uma abordagem mais elegante e legível para incorporar conteúdo dinâmico em uma string. Ao fazer isso, os desenvolvedores podem criar mensagens ou outputs mais claros e personalizados, melhorando a experiência do usuário final e simplificando o próprio código.

Já a **formatação** de texto refere-se ao processo de ajustar sua aparência ou estrutura. Em C# isso geralmente envolve métodos que permitem aos desenvolvedores controlar como os dados são apresentados, seja especificando o número de casas decimais de um número ou determinando a disposição visual de um conjunto de informações. A formatação adequada é crucial para garantir que a informação seja facilmente compreendida, evitando ambiguidades ou mal-entendidos.

Em C# há várias maneiras de concatenar textos; uma das mais diretas é usar o operador +, como foi mostrado na descrição de Zeus (linha 14 da figura 139). Outra maneira é usar o método String. Concat (linha 19), útil quando temos várias strings que queremos juntar e demonstrado na relação entre Zeus e Hera.

Já a interpolação de strings pode ser realizada usando o símbolo **$** antes da string e colocando as variáveis ou expressões entre chaves **{}**. Como visto na frase sobre o atributo de Zeus (linha 24), a interpolação proporciona uma maneira limpa e legível de incorporar valores dinâmicos em uma string. O método String.Format, usado na descrição da deusa Atena (linha 30), é outra maneira de atingir um resultado similar, embora seja menos conciso do que a interpolação direta.

Como pudemos observar, concatenação, interpolação e formatação de texto são mais do que apenas técnicas; são facilitadores que permitem aos desenvolvedores comunicar informações de maneira clara e eficiente. A manipulação de texto em C# desempenha um papel inegável em uma variedade de aplicações reais, servindo como espinha dorsal de muitas operações essenciais.

Vamos explorar alguns exemplos, mapeando os sistemas reais com os principais métodos de manipulação de texto.

```
1.  using System;
2.
3.  namespace MitologiaGregaTexto
4.  {
5.     class Program
6.     {
7.        static void Main(string[] args)
8.        {
9.           // 1. Definição e inicialização de strings
10.          string nomeDeus = "Zeus";
11.          string titulo = "deus do trovão";
12.
13.          // 2. Concatenação usando o operador '+'
14.          string descricaoZeus = nomeDeus + " é o " + titulo + " e reina
no Olimpo.";
15.          Console.WriteLine(descricaoZeus);
16.
17.          // 3. Concatenação usando String.Concat()
18.          string esposaZeus = "Hera";
19.          string relacaoZeus = String.Concat(nomeDeus, " é casado com ",
esposaZeus, ".");
20.          Console.WriteLine(relacaoZeus);
21.
22.          // 4. Interpolação de string usando $ (C# 6.0 e versões
posteriores)
23.          string atributo = "força";
24.          string fraseInterpolada = $"{nomeDeus} é conhecido por sua
{atributo}.";
25.          Console.WriteLine(fraseInterpolada);
26.
27.          // 5. Usando String.Format() para formatação
28.          string deusa = "Atena";
29.          string dominioDeusa = "sabedoria";
30.          string descricaoDeusa = String.Format("{0} é a deusa da {1}.",
deusa, dominioDeusa);
31.          Console.WriteLine(descricaoDeusa);
32.
33.       }
34.    }
35. }
```

Figura 139 – Recursos de concatenação, interpolação e formatação de texto

Começando com sistemas de gerenciamento de conteúdo (CMS), plataformas renomadas como WordPress e Joomla, bem como soluções CMS personalizadas, são amplamente dependentes da manipulação textual. Tarefas cruciais são a formatação de conteúdo, busca de palavras-chave e substituição de texto. Métodos como String.Replace, String.Contains e String.Format são rotineiramente empregados para essas tarefas.

Em seguida, consideremos motores de busca, como Google e Bing, que indexam e pesquisam páginas da web e, assim, manipulam vastas quantidades de texto. O processamento de texto ajuda na

tokenização, no processamento e na classificação de texto usando algoritmos complexos. O String.Split e o String.IndexOf são métodos comumente usados.

Em sistemas de comércio eletrônico, como Amazon ou eBay, a manipulação de texto é essencial para gerenciar descrições de produtos, buscas e processamento de avaliações de usuários. Funções como String.ToLower ou String.ToUpper são valiosas ao realizar buscas insensíveis a maiúsculas e minúsculas. Aplicações de processamento de linguagem natural (NLP), como assistentes virtuais, se beneficiam enormemente da capacidade do C# de processar entradas de texto do usuário e interpretá-las usando métodos como String.StartsWith e String.EndsWith para compreender e responder a comandos. Ferramentas de desenvolvimento, como o Visual Studio (uma IDE popular), também se baseiam na manipulação de texto para fornecer realce de sintaxe, autocompletar e refatoração. Aqui, métodos como String.Substring e StringBuilder.Append são frequentemente utilizados.

Considerando também os sistemas de relatórios, o C# é usado para formatar, manipular e apresentar texto em layouts específicos. Métodos como String.PadLeft ou String.PadRight são úteis para alinhar texto de forma adequada nos relatórios. Aplicações de tradução se baseiam fortemente na capacidade de processar e alterar strings para fornecer traduções precisas. Aqui funções como String. Compare e String.Equals são vitais para garantir a precisão e a qualidade da tradução. Sistemas de mensagens dependem do C# para formatar mensagens, detectar e criar links clicáveis e processar emojis; e métodos como String.Compare são comuns para detectar padrões (como URLs) em mensagens.

Finalmente, plataformas de e-learning manipulam texto para apresentar quizzes, gerenciar fóruns de discussão e fornecer feedback. A capacidade de dividir, juntar e pesquisar strings usando métodos como String.Join ou String.Split é frequentemente necessária. Ao considerar todos esses sistemas, fica claro que a capacidade de C# em manipular texto não é apenas valiosa, mas fundamental para o funcionamento eficaz de muitos sistemas modernos.

4.5 Trabalhando com datas

Trabalhar com datas e horas em C# é um aspecto fundamental de muitos programas, desde aplicativos de agendamento até softwares financeiros. A linguagem C# fornece uma série de classes e estruturas para lidar com datas, horas e intervalos no namespace System.DateTime. No cerne desse processo está a estrutura DateTime, que serve para representar um instante específico, podendo expressar tanto uma data quanto uma hora específica. Por exemplo, com o DateTime podemos facilmente obter a data e hora atuais do sistema ou até mesmo definir uma data específica. Ele não apenas permite representar momentos, mas também fornece uma variedade de propriedades e métodos úteis, como informar sobre ano, mês e dia de uma data ou até mesmo sobre a hora, minuto e segundo de um momento específico. Além disso, é possível conhecer o dia da semana de uma determinada data ou adicionar e subtrair dias, meses e anos a ela.

Formatação é fundamental para apresentar datas e horas a usuários ou registrar arquivos, e o DateTime dá flexibilidade para formatá-las de diversas maneiras, tornando-as mais legíveis ou adaptadas a diferentes padrões regionais. É comum também comparar datas, seja para determinar a diferença entre elas ou ordená-las, e em C# podem ser facilmente comparadas usando operadores-padrão.

Além de simplesmente compará-las, muitas vezes precisamos saber o intervalo exato entre elas; para isso a linguagem nos apresenta o TimeSpan, que representa um intervalo e pode ser determinado subtraindo-se duas datas.

No entanto, a hora e a data em um local nem sempre se traduzem da mesma forma em outro. Isso é especialmente verdadeiro em um mundo globalizado, onde aplicações muitas vezes precisam considerar fusos horários. Enquanto o DateTime representa data e hora sem considerar um fuso horário específico, temos o DateTimeOffset, que traz essa perspectiva em relação ao tempo universal coordenado (UTC). Além disso, a manipulação adequada de zonas horárias e até mesmo do horário de verão é viabilizada pela classe TimeZoneInfo.

O quadro 28 apresenta detalhes do DateTime.

Quadro 28 – DateTime: principais propriedades e métodos

Tipo	Nome	Descrição
Propriedade	Year	Retorna o ano da data
	Month	Retorna o mês da data
	Day	Retorna o dia do mês da data
	Hour	Retorna a hora do dia
	Minute	Retorna o minuto da hora
	Second	Retorna o segundo do minuto
	DayOfWeek	Retorna o dia da semana
	DayOfYear	Retorna o dia do ano
	Date	Retorna a parte da data (sem a hora)
	TimeOfDay	Retorna a hora do dia
Métodos	Now	Retorna data e hora atuais do sistema
	Today	Retorna a data atual
	AddDays(double value)	Adiciona um número especificado de dias à data
	AddMonths(int value)	Adiciona um número especificado de meses à data
	AddYears(int value)	Adiciona um número especificado de anos à data
	AddHours(double value)	Adiciona horas à data
	AddMinutes(double value)	Adiciona minutos à data
	AddSeconds(double value)	Adiciona segundos à data
	Subtract(DateTime value)	Retorna a diferença entre duas datas como um TimeSpan
	ToString()	Converte o objeto DateTime em sua representação de string equivalente
	Parse(string s)	Converte a representação de string de uma data e hora em seu equivalente DateTime
	TryParse(string s, out DateTime result)	Tenta converter a representação de string de uma data e hora e retorna um valor que indica se a conversão foi bem-sucedida

A figura 140 apresenta um código-fonte que utiliza a mitologia grega como recurso didático para ilustrar a estrutura DateTime e seus principais aspectos.

```
1.  using System;
2.
3.  namespace MitologiaGrega
4.  {
5.    class Program
6.    {
7.      static void Main(string[] args)
8.      {
9.        // Criar datas específicas para eventos mitológicos
10.       DateTime nascimentoDeZeus = new DateTime(5000, 1, 1);
11.       DateTime guerraDeTroia = new DateTime(1200, 6, 10);
12.
13.       // Formatação de datas
14.       string zeusFormatado = nascimentoDeZeus.ToString("dd/MM/yyyy");
15.       Console.WriteLine($"Zeus nasceu em {zeusFormatado}.");
16.
17.       // Comparação de datas
18.       if (nascimentoDeZeus < guerraDeTroia)
19.       {
20.         Console.WriteLine("Zeus nasceu antes da Guerra de Troia.");
21.       }
22.       else
23.       {
24.         Console.WriteLine("Zeus nasceu depois da Guerra de Troia.");
25.       }
26.
27.       // Uso do DateTimeOffset
28.       DateTimeOffset festivalDeDionisio = new DateTimeOffset(800, 3,
20, 12, 0, 0, TimeSpan.FromHours(3));
29.       Console.WriteLine($"O festival de Dionísio ocorreu em
{festivalDeDionisio.DateTime} no fuso UTC+3.");
30.
31.       // Trabalhando com zonas horárias e horário de verão
32.       TimeZoneInfo athensZone = TimeZoneInfo.
FindSystemTimeZoneById("E. Europe Standard Time");
33.       bool isDaylight = athensZone.IsDaylightSavingTime(festivalDeDi
onisio.DateTime);
34.       Console.WriteLine($"No festival de Dionísio, Atenas estava no
horário de verão? {isDaylight}");
35.     }
36.   }
37. }
```

Figura 140 – DateTime: exemplo de utilização

O código começa criando duas datas significativas no contexto da mitologia grega: o nascimento de Zeus e a Guerra de Troia (linhas 10 e 11, respectivamente). A data de nascimento de Zeus é formatada e exibida em um padrão comum, mostrando como podemos facilmente alterar a representação visual de um DateTime. Em seguida, na linha 18 o programa compara as duas datas para determinar se Zeus nasceu antes ou depois da Guerra de Troia, o que demonstra como é simples comparar dois objetos DateTime usando operadores-padrão.

O código então apresenta o conceito de DateTimeOffset ao criar uma data e hora para o festival de Dionísio, que inclui informações sobre o fuso horário. Usamos um fuso horário de UTC+3, que pode representar uma localização na Grécia Antiga (linha 29). Finalmente, abordamos zonas horárias e horário de verão ao verificar se Atenas observava o horário de verão durante o festival de Dionísio; para isso utilizamos a classe TimeZoneInfo (propriedade IsDaylightSavingTime, na linha 33). Esse exemplo demonstra como as zonas horárias e o horário de verão podem influenciar o horário exato de um evento, o que é crucial em aplicações que lidam com múltiplos fusos horários.

Essas são apenas algumas das muitas propriedades e métodos disponíveis na estrutura DateTime, que proporcionam uma gama de funcionalidades para lidar com datas e horários, facilitando o desenvolvimento de aplicações robustas e precisas.

4.6 Conversões

Conversões em C# são uma parte fundamental da linguagem, visto que desenvolvedores frequentemente precisam transformar um tipo de dado em outro. Isso pode acontecer por uma variedade de razões, como atender à assinatura de um método ou processar informações de maneiras diferentes. No tópico 4.3 abordamos esse tema em detalhes e vimos que a conversão pode ser classificada em duas categorias principais: conversões implícitas e conversões explícitas.

Conversão implícita é realizada pelo compilador C# sem que o desenvolvedor precise indicá-la explicitamente. Isso ocorre quando não há risco de perder dados ou quando o tipo de destino é compatível com o tipo de origem. Em contraste, se houver potencial para perder dados ou o tipo de destino não for imediatamente compatível com o tipo de origem, o compilador exige uma conversão explícita. Isso é frequentemente chamado de **casting**.

Também foi apresentado que, além do casting, o C# fornece várias classes e métodos que auxiliam nas conversões, como Convert, TryParse e ToString. Por serem temas já vistos em detalhes, o foco deste capítulo será conversões relacionadas a tipos personalizados.

Desenvolvedores podem implementar métodos de conversão usando operadores **explicit** e **implicit**, permitindo definir regras personalizadas para conversões entre tipos. Classes personalizadas, no contexto da POO – e especificamente em linguagens como C# – são definições de tipos criadas pelo programador para representar entidades, conceitos ou estruturas específicas não fornecidas diretamente pelas bibliotecas-padrão da linguagem. Classes personalizadas não somente contêm dados (representados por campos ou propriedades), mas também comportamentos (representados por métodos). A capacidade de combinar dados e comportamento em uma única estrutura é uma das principais características da POO.

Ao trabalhar com tipos personalizados, muitas vezes precisamos converter uma instância de uma classe em uma instância de outra. A linguagem C# oferece a capacidade de definir operadores de conversão personalizados – métodos especiais que determinam como a instância de uma classe pode ser transformada em instância de outra. Operadores de conversão personalizados também são classificados em duas categorias: implícitos e explícitos. Os primeiros permitem uma conversão sem precisar de um

cast explícito, sendo utilizados se houver certeza de que a conversão não resultará em perda de dados ou se não houver ambiguidade na operação. Já os últimos requerem um cast para converter e são usados se houver potencial para perder dados ou ambiguidade.

Um detalhe importante é que, ao definir operadores de conversão personalizados em C#, as palavras-chave explicit e implicit devem ser seguidas pela palavra **operator**, pois isso distingue a declaração de ser um operador de conversão em vez de um modificador de acesso ou qualquer outra função em C#.

A figura 141 ilustra as duas categorias de conversão. Nesse código temos três classes personalizadas: Deus, SemiDeus e Humano. A classe Deus tem um operador de conversão explícito (linha 9) que permite converter um objeto Deus em um objeto SemiDeus; isso significa que um deus pode ser rebaixado a semideus, representando o conceito de um filho divino. No caso, adicionamos "Jr." ao nome para indicar que o semideus é um descendente. Por outro lado, a classe SemiDeus tem um operador de conversão implícito (linha 19) para um humano. Isso representa o fato de que semideuses, por terem sangue divino, muitas vezes desempenham papel heroico no mundo dos mortais.

No método Main começamos com Zeus, o rei dos deuses. Convertendo Zeus explicitamente em semideus, criamos seu filho (linha 38). Perceba o uso de abre-fecha parênteses para sinalizar o casting; esse filho é então convertido implicitamente (linha 41) em humano, resultando em um herói com ocupação baseada no domínio de seu progenitor divino. Note que nesse caso, por se tratar de conversão implícita, não foi necessário sinalizar o casting.

Esse exemplo ilustra a riqueza da linguagem C#, ao modelar conceitos abstratos e a flexibilidade dos operadores de conversão personalizados a fim de definir regras específicas quando se trabalha com tipos personalizados.

```
1.  public class Deus
2.  {
3.    public string Nome { get; set; }
4.    public string Dominio { get; set; }
5.
6.    // Conversão explícita de um Deus para um SemiDeus
7.    public static explicit operator SemiDeus(Deus deus)
8.    {
9.      return new SemiDeus { Nome = deus.Nome + " Jr.", Dominio = deus.
Dominio };
10.   }
11. }
12.
13. public class SemiDeus
14. {
15.   public string Nome { get; set; }
16.   public string Dominio { get; set; }
17.
18.   // Conversão implícita de um SemiDeus para um Humano
19.   public static implicit operator Humano(SemiDeus semiDeus)
20.   {
21.     return new Humano { Nome = semiDeus.Nome, Ocupacao = "Herói por
causa do parente " + semiDeus.Dominio };
22.   }
23. }
24.
25. public class Humano
26. {
27.   public string Nome { get; set; }
28.   public string Ocupacao { get; set; }
29. }
30.
31. class Program
32. {
33.   static void Main(string[] args)
34.   {
35.     Deus zeus = new Deus { Nome = "Zeus", Dominio = "Céu" };
36.
37.     // Conversão explícita para SemiDeus
38.     SemiDeus filhoDeZeus = (SemiDeus)zeus;
39.
40.     // Conversão implícita para Humano
41.     Humano heroi = filhoDeZeus;
42.
43.     Console.WriteLine($"{heroi.Nome} é um {heroi.Ocupacao}.");
44.   }
45. }
```

Figura 141 – Conversão entre tipos personalizados: exemplo mitológico

4.7 Breve histórico da linguagem C# e do .NET

Antes de lançar o C#, a Microsoft usava várias linguagens de desenvolvimento, sendo o Visual Basic (VB) e o Visual C++ as mais notáveis. Ambas tinham capacidades para POO, mas cada uma atendia a diferentes nichos e tinha suas próprias forças. O VB foi uma das primeiras linguagens de programação que tornaram o desenvolvimento para Windows acessível a uma ampla gama de programadores, popularizando-se por sua simplicidade e capacidade de desenvolver rapidamente aplicações GUI para Windows. Embora tivesse capacidades para POO, não era tão rigoroso nem sofisticado quanto linguagens como C++ ou Java.

Visual C++ é o ambiente de desenvolvimento da Microsoft para C++, uma linguagem poderosa, que suporta tanto a programação procedural quanto a orientada a objetos. Antes do .NET, o Visual C++ era a ferramenta principal para desenvolver aplicações de alto desempenho e sistemas para Windows. A plataforma Java, lançada em 1995, começou a ganhar tração no final dos anos 1990 e tornou-se uma ameaça competitiva à Microsoft. Java prometeu o mantra "Escreva uma vez, execute em qualquer lugar", ou seja, os desenvolvedores poderiam escrever seu código uma vez e executá-lo em qualquer máquina com Java.

Esse conceito foi atraente para muitas empresas, pois prometia reduzir os custos e a complexidade do desenvolvimento de software. Embora o Visual C++ fosse poderoso e flexível, não era tão amigável nem acessível quanto o Java em certos aspectos. Java tinha uma biblioteca-padrão rica, gerenciamento automático de memória (através de garbage collection) e uma forte orientação a objetos, tornando-o atraente para uma gama de desenvolvedores.

A ascensão do Java, especialmente no espaço empresarial, estimulou a Microsoft a desenvolver uma resposta. Embora a empresa inicialmente tentasse colaborar com a Sun Microsystems (os criadores do Java) e produzisse sua própria versão do Java chamada J++ (Visual J++), disputas legais e diferenças estratégicas entre as duas empresas levaram a Microsoft a criar sua própria plataforma e linguagem. O resultado foi o .NET Framework e o C#. Ou seja, enquanto o C++ (e o Visual C++) era e ainda é uma linguagem poderosa usada por muitos desenvolvedores, a Microsoft viu a necessidade de uma plataforma e linguagem que pudesse competir mais diretamente com Java em termos de produtividade do desenvolvedor, portabilidade e capacidades orientadas a objetos. Essa foi uma das principais motivações por trás do desenvolvimento do C# e do .NET Framework.

No início dos anos 2000, a Microsoft, ao reconhecer a ascensão das linguagens orientadas a objetos e a necessidade de simplificar o desenvolvimento de aplicações para Windows, iniciou o projeto ambicioso de criar uma nova estrutura de desenvolvimento. O .NET Framework foi lançado oficialmente em 2002 e introduziu uma variedade de inovações (Box, 2002). A ideia era fornecer uma plataforma única para desenvolver aplicações web, desktop, móveis e de serviços, em contraste direto com a abordagem tradicional, que exigia diferentes ferramentas e linguagens para diferentes tipos de aplicações.

Na raiz do .NET estava o conceito de código gerenciado, executado em uma máquina virtual chamada Common Language Runtime (CLR). Esse ambiente de execução permitiu que diferentes linguagens de programação fossem usadas de forma intercambiável, enquanto mantinham a interoperabilidade.

Com o .NET Framework, a Microsoft lançou uma nova linguagem de programação chamada C# (Hejlsberg; Torgersen, 2010). Criada por Anders Hejlsberg – uma das mentes por trás do Turbo Pascal e do Delphi –, o C# foi desenvolvido para ser uma linguagem moderna, orientada a objetos e de tipagem forte. Ele combinou muitos dos melhores recursos de linguagens anteriores, como Java e C++, mas também introduziu novas características que o tornaram distinto e poderoso.

Nos primeiros anos de seu lançamento, o .NET e o C# enfrentaram ceticismo. Muitos desenvolvedores relutaram em adotar uma nova plataforma e linguagem, especialmente considerando a dominância de Java no mercado. No entanto, a Microsoft continuou a investir pesado no desenvolvimento e melhoria do .NET e C#. Com cada versão subsequente, mais funcionalidades foram adicionadas, tornando a plataforma mais robusta e versátil (Metzgar, 2018).

Após o lançamento inicial do .NET em 2002, passaram-se alguns anos para a plataforma ganhar tração significativa; mas por volta de 2005, com o lançamento do .NET Framework 2.0 e do Visual Studio 2005, a Microsoft afirmou que havia mais de três milhões de desenvolvedores .NET em todo o mundo. Ao mesmo tempo, outras linguagens e plataformas, como Java, Python e JavaScript, também viram um crescimento significativo (Java foi uma forte concorrente do .NET durante os anos iniciais, por exemplo). No entanto, uma vantagem competitiva do .NET era sua integração profunda com o sistema operacional Windows e o pacote de produtos da Microsoft, tornando-o a escolha preferida de muitas empresas já investidas no ecossistema da empresa.

O lançamento do .NET Framework 3.0 em 2006 foi um marco, pois introduziu o Windows Presentation Foundation (WPF) e o Windows Communication Foundation (WCF), expandindo ainda mais seu alcance e capacidade. Enquanto a Microsoft continuava a evoluir sua plataforma, também reconheceu a importância do código aberto. Em termos de fatia de mercado, o C# rapidamente se tornou uma das dez principais linguagens de programação no índice TIOBE pouco depois de seu lançamento, e durante a década de 2010 manteve-se consistentemente entre as cinco principais (Sebesta, 2010).

O crescimento explosivo do Python durante a década de 2010, impulsionado por aplicações em ciência de dados e aprendizado de máquina, tornou-o uma das linguagens de programação mais populares. Embora o .NET tenha tentado entrar nesse espaço com iniciativas como ML.NET, o Python ainda domina essa área. Em 2014, sob a liderança do CEO Satya Nadella, a empresa fez um movimento sem precedentes ao anunciar o .NET Core como uma reimplementação de código aberto e multiplataforma do .NET, com milhões de linhas de código, o que indicava a crescente complexidade e robustez do sistema.

A decisão mostrou um claro reconhecimento da importância da comunidade de desenvolvimento e da necessidade de inovação aberta. A evolução do .NET Core culminou no lançamento do .NET 5 em 2020, que unificou os vários sabores do .NET e posicionou a plataforma no futuro. O C# também viu várias melhorias durante o período, com novas características adicionadas para tornar o desenvolvimento mais intuitivo e eficiente.

O C# tem experimentado uma série de atualizações e melhorias desde seu lançamento em 2002. Cada versão introduziu novas características que refletem as mudanças nas práticas de desenvolvimento e nas demandas do setor. O quadro 29 destaca as versões do C# e suas características.

Quadro 29 – Versões do C# e características

Versão	Ano	Características relevantes
C# 1.0	2002	Lançamento original com o .NET Framework 1.0
		Introduziu classes, structs, interfaces e eventos
C# 2.0	2005	Introdução de genéricos
		Anônimos e métodos inline chamados de "delegados anônimos"
		Palavra-chave yield para iteração simplificada e criação de enumeradores
		Tipos anuláveis
C# 3.0	2007	Introdução de Language-Integrated Query (LINQ)
		Tipos anônimos para criar objetos sem definir um tipo de classe
		Expressões lambda para escrever funções anônimas
		Extensões de métodos
C# 4.0	2010	Suporte dinâmico com o tipo dynamic
		Argumentos nomeados e parâmetros opcionais
		Suporte melhorado para COM e interoperação com o Office
C# 5.0	2012	Palavras-chave async e await para programação assíncrona simplificada
		Melhorias em capturar exceções
C# 6.0	2015	Propriedades autoimplementadas com inicializadores
		Expressão nameof para obter o nome de variáveis, tipos ou membros
		Operador null condicional (?.) para simplificar verificações nulas
C# 7.0	2017	Suporte para tuplas e desestruturação
		Locais de função
		Melhorias em pattern matching
C# 8.0	2019	Tipos de referência anuláveis
		Padrões de switch melhorados
C# 9.0	2020	Registros (records)
		Iniciação com propriedades init
C# 10	2021	Structs de registro
		Manipuladores de cadeia de caracteres interpolada
		Diretivas using globais

A versão atual, lançada no final de 2022, é o C# 11 e tem como principais novidades:

- atributos genéricos;
- suporte matemático genérico;
- numérico IntPtr e UIntPtr;
- novas linhas em interpolações de cadeia de caracteres;
- padrões de lista;

- conversão aprimorada do grupo de métodos para delegado;
- literais de cadeia de caracteres bruta;
- structs de padrão automático;
- correspondência de padrão Span<char> e ReadOnlySpan<char> em uma constante string;
- escopo nameof estendido;
- cadeia de caracteres UTF-8 literais;
- membros necessários;
- campos ref e variáveis ref scoped;
- tipos de locais de arquivo.

Com o C# foi lançado o .NET 7, que tem como destaques:

- desempenho;
- serialização de System.Text.Json;
- matemática genérica;
- expressões regulares;
- bibliotecas .NET;
- observabilidade;
- SDK .NET;
- geração de origem P/Invoke.

Ao olharmos para trás, podemos ver que o C# e o .NET têm sido uma jornada de inovação contínua, e sua trajetória é notável desde os primeiros dias de hesitação até a aceitação global e o reconhecimento como uma das principais plataformas de desenvolvimento. O sucesso dessas tecnologias não é apenas um testemunho da visão da Microsoft, mas também do empenho e paixão da vasta comunidade de desenvolvedores que abraçaram, moldaram e impulsionaram C# e .NET para seu atual patamar.

Resumo

Nesta unidade observamos que a linguagem C# emprega uma sintaxe claramente delimitada, utilizando chaves para definir o escopo de blocos de código, como funções e classes, e ponto e vírgula para terminar instruções. Palavras-chave como class são utilizadas com o objetivo de definir classes, enquanto void precede a declaração de métodos que não retornam valores. Identificadores, sejam eles nomes de variáveis, métodos ou classes, são definidos pelo usuário mas devem seguir certas regras, começando com uma letra ou sublinhado, seguido por letras, números ou mais sublinhados; e o uso das nomenclaturas e convenções é importante.

Vimos ainda que na declaração de variáveis em C# especifica-se primeiro o tipo de dado, seguido pelo identificador da variável. É também essencial destacar que C# é uma linguagem fortemente tipada, ou seja, o compilador verifica o tipo de cada variável e objeto no momento da compilação, prevenindo muitos erros. Além disso, métodos em C# são blocos de códigos que podem ou não retornar um valor e são definidos dentro de classes ou estruturas.

Para criar um método, utilizamos modificadores de acesso, como public ou private, seguidos pelo tipo de retorno e, por fim, pelo nome do método. Os parâmetros do método são definidos entre parênteses após o nome do método, sendo cada parâmetro precedido por seu tipo de dado. Para criar objetos em C#, o programador recorre à palavra-chave new, seguida pela chamada ao construtor da classe. Os objetos criados herdam propriedades e métodos da classe a que pertencem, sendo possível acessar e modificar seus valores e comportamentos de acordo com as regras de acesso definidas na classe.

O .NET Framework serve como ambiente de execução que oferece um grande conjunto de serviços, bibliotecas e recursos que facilitam o desenvolvimento, a execução e a manutenção de aplicações escritas em diferentes linguagens de programação, incluindo o C#. Dessa forma, o C# foi especialmente projetado para operar de maneira otimizada dentro desse framework, aproveitando suas funcionalidades e serviços para entregar aplicações de alta performance e seguras.

Ao desenvolver com C# no ambiente .NET, os programadores têm acesso a uma série de bibliotecas e APIs robustas, que abrangem desde operações de entrada e saída até o desenvolvimento de interfaces gráficas e serviços web. Esse conjunto vasto de recursos proporciona aos

desenvolvedores a capacidade de criar aplicações diversificadas, como aplicações web, móveis, desktop, jogos e muito mais, de forma eficaz e com alta produtividade. O C# e o .NET juntos podem utilizar paradigmas de POO, permitindo aos desenvolvedores modelar soluções de software intuitivas, escaláveis e manuteníveis.

A interação entre objetos, classes e métodos em C# é gerenciada e otimizada pelo .NET, que fornece os mecanismos necessários para executar o código e gerenciar a memória, melhorando a estabilidade e a performance das aplicações. Além disso, proporciona uma plataforma de execução independente de sistema operacional, ou seja, aplicações desenvolvidas em C# podem ser executadas em diferentes sistemas operacionais sem modificações significativas no código-fonte, o que se traduz em maior portabilidade e flexibilidade para aplicações criadas nessa linguagem, alargando os horizontes de onde e como as aplicações podem ser implantadas e executadas.

Exercícios

Questão 1. (FGV 2018, adaptada) Leia o código C# a seguir.

```
using System;

    public class X
    {
    public static int Test(int a, int b)
    {
      while (a != b) {
             if (a > b) {a -= b;}
             else {b -= a;}
      }
      return a;
}
static void Main(string[] args)
{
      int x = 36;
      int y = 7;
      Console.WriteLine(X.Test(x, y));
    }
}
```

O número produzido pela execução desse código é

A) 1.

B) 2.

C) 3.

D) 7.

E) 36.

Resposta correta: alternativa A.

Análise da questão

O código-fonte do enunciado, escrito em linguagem C#, define uma classe de nome `X` com dois métodos: um método estático de nome `Test` e outro `Main` para executar o programa. No método `Main` há um comando de impressão do retorno do método `Test` com dois argumentos: `x` e `y`. Essas variáveis assumem, respectivamente, os valores 36 e 7.

Esses dois valores são recebidos pelos parâmetros `a` e `b` do método `Test`. A partir daí um laço `while` testa se `a` é diferente de `b` e, caso verdadeiro, o bloco de comandos do laço é executado.

Dentro desse laço, é feito mais um teste condicional, que verifica se o valor de `a` é maior que o valor de `b`. Caso seja verdade, é executado o comando `a -= b;`, que equivale ao comando `a = a-b;`. Senão é executado o comando `b -= a;`, que equivale ao comando `b = b-a;`.

A tabela a seguir resume as iterações do laço `while`, com a evolução dos valores das variáveis `a` e `b` ao longo da execução do programa. A primeira linha, da "execução 0", apenas mostra os valores iniciais das variáveis.

Tabela 1

Execução while	Teste da condição a != b	Teste da condição a > b	a	b
0	-	-	36	7
1	36 != 7 (V)	36 > 7 (V)	29	7
2	29 != 7 (V)	29 > 7 (V)	22	7
3	22 != 7 (V)	22 > 7 (V)	15	7
4	15 != 7 (V)	15 > 7 (V)	8	7
5	8 != 7 (V)	8 > 7 (V)	1	7
6	1 != 7 (V)	1 > 7 (F)	1	6
7	1 != 6 (V)	1 > 6 (F)	1	5
8	1 != 5 (V)	1 > 5 (F)	1	4
9	1 != 4 (V)	1 > 4 (F)	1	3
10	1 != 3 (V)	1 > 3 (F)	1	2
11	1 != 2 (V)	1 > 2 (F)	1	1
12	1 != 1 (F)	-	1	1

Na 12ª execução, a condição do laço `while` torna-se falsa, fazendo com que o bloco de comando do laço não seja executado. Nesse cenário, as variáveis `a` e `b` mantêm os valores obtidos na 11ª iteração.

Desse modo, o retorno do método `Test`, que é o valor da variável `a`, produzirá a saída 1 no console.

Questão 2. (Fundatec 2023, adaptada) A plataforma .NET consiste em um framework de desenvolvimento que proporciona um vasto conjunto de bibliotecas e de ferramentas para auxiliar desenvolvedores a criar aplicações para Windows, web, mobile, entre outros. A plataforma fornece um ambiente em tempo de execução que executa o código e fornece serviços que facilitam o processo de desenvolvimento. Como é chamado esse ambiente?

A) JVM (Java Virtual Machine).

B) CLR (Common Language Runtime).

C) REC (Runtime Environment Compiler).

D) RVM (Runtime Virtual Machine).

E) CRL (Compiler Runtime Library).

Resposta correta: alternativa B.

Análise da questão

Common Language Runtime (CLR) é um ambiente em tempo de execução, da plataforma .NET, que gerencia a execução de programas escritos em linguagens de programação compatíveis com a plataforma, como a C#. O CLR desempenha várias funções essenciais, como:

- compilação just in time;
- gerenciamento de memória;
- controle de execução;
- proteção contra códigos maliciosos;
- gerenciamento de threads.